

khiem khiem

bao_cao_do_an_TN

 Trương Nguyễn Quốc Thắng - 22119231 - DATN

 DATN_K22

 Ho Chi Minh City University of Technology and Engineering (HCMUTE)

Thông tin chi tiết về tài liệu

ID bài nộp

tm:oid:::1:3601810813

Ngày nộp

10:38 GMT+7 26 thg 6, 2026

Ngày tải xuống

19:40 GMT+7 26 thg 6, 2026

Tên tệp

bao_cao_do_an_TN.pdf

Kích cỡ tệp

3.0 MB

144 Trang

48.141 Từ

209.599 Ký tự

4% Tính tương đồng nói chung

Tổng cộng của tất cả các kết quả trùng khớp, bao gồm cả các nguồn trùng lặp, cho mỗi c...

Nguồn hàng đầu

- 3%  Nguồn Internet
- 2%  Ấn bản
- 2%  Bài tập được nộp (bài của học sinh)

Cờ chỉ báo về tính toàn vẹn

0 cờ chỉ báo về tính toàn vẹn để đánh giá

Không phát hiện thấy văn bản nào nghi ngờ là ngụy tạo.

Các thuật toán trong hệ thống của chúng tôi xem xét kỹ lưỡng một tài liệu để phát hiện sự không nhất quán khiến cho tài liệu đó khác với một bài nộp bình thường. Nếu nhận thấy điều gì lạ, chúng tôi sẽ gắn cờ để bạn xem xét.

Cờ không nhất thiết là dấu hiệu chỉ báo có vấn đề. Tuy nhiên, bạn nên chú ý vào phần đó để xem xét thêm.

Nguồn hàng đầu

- 3%  Nguồn Internet
- 2%  Ấn bản
- 2%  Bài tập được nộp (bài của học sinh)

Nguồn hàng đầu

Các nguồn có số lượng kết quả trùng khớp cao nhất trong bài nộp. Các nguồn chông chéo sẽ không được hiển thị.

1	Bài của học viên	Ho Chi Minh University of Technology and Education	<1%
2	Internet	www.ctu.edu.vn	<1%
3	Bài của học viên	Ho Chi Minh City Open University	<1%
4	Xuất bản	Banking Academy	<1%
5	Internet	text.123docz.net	<1%
6	Xuất bản	Tung Le, Truong N. Pham, Tung Tran, Khiem Le, Tien Do, Thanh Duc Ngo. "ViCalli...	<1%
7	Internet	doc.edu.vn	<1%
8	Internet	tailieu.vn	<1%
9	Bài của học viên	Vietnam Maritime University	<1%
10	Bài của học viên	Nha Trang University	<1%
11	Xuất bản	Ton Duc Thang University	<1%

12	Bài của học viên	Van Lang University	<1%
13	Bài của học viên	University of Finance – Marketing	<1%
14	Internet	arxiv.org	<1%
15	Internet	luanvan.net.vn	<1%
16	Internet	runsystem.net	<1%
17	Internet	text.xemtailieu.net	<1%
18	Internet	cdn.iit.ac.in	<1%
19	Bài của học viên	Hoa Sen University	<1%
20	Bài của học viên	University of Economics & Law	<1%
21	Internet	tailieudientu.lrc.tnu.edu.vn	<1%
22	Xuất bản	Phenikaa University	<1%
23	Internet	www.tlu.edu.vn	<1%
24	Internet	thuvienso.hcmute.edu.vn	<1%
25	Internet	www.arxiv-vanity.com	<1%

26	Internet	luanvan.co	<1%
27	Internet	bogiaoduc.edu.vn	<1%
28	Internet	nguyendung622.files.wordpress.com	<1%
29	Bài của học viên	Gauhati University	<1%
30	Internet	luanvan.org	<1%
31	Internet	vn-document.net	<1%
32	Internet	www.thptkontum.edu.vn	<1%
33	Bài của học viên	Yersin University of Da Lat	<1%
34	Internet	aifirst.vn	<1%
35	Bài của học viên	Tra Vinh University	<1%
36	Internet	robotics.ait.kyushu-u.ac.jp	<1%
37	Internet	vi.wikipedia.org	<1%
38	Internet	d197for5662m48.cloudfront.net	<1%
39	Internet	doanhnhan.net	<1%

40	Internet	eprints.qut.edu.au	<1%
41	Internet	jfm.ufm.edu.vn	<1%
42	Internet	thuvienso-banh.blogspot.com	<1%
43	Internet	www.flowhunt.io	<1%
44	Internet	www.frontiersin.org	<1%
45	Internet	www.mof.gov.vn	<1%
46	Internet	www.thuvientailieu.vn	<1%
47	Internet	diendanngoingu.vn	<1%
48	Internet	luxurydesign.vn	<1%
49	Internet	minds.wisconsin.edu	<1%
50	Internet	uhl.edu.vn	<1%
51	Internet	www.scusd.edu	<1%
52	Internet	www.soymexiquense.com	<1%
53	Internet	www.vnua.edu.vn	<1%

54	Internet	www.zbook.vn	<1%
55	Xuất bản	Công Phạm Xuân Công, Thiệp Diệp Minh Thiện, Lâm Đặng Văn Lâm, Hiền Hoàng T...	<1%
56	Xuất bản	Hanoi University	<1%
57	Xuất bản	Nguyễn Văn Phương, Đào Khánh Hoài, Tống Minh Đức. "Tăng tốc độ phát hiện dị ...	<1%
58	Xuất bản	Pham T.T., Bennett K., Vu T.P., Brunner J., Le N.D., Nguyen D.T.. "Chi tra dich vu m...	<1%
59	Bài của học viên	University of Edinburgh	<1%
60	Internet	antgct.cand.com.vn	<1%
61	Internet	cps.hau.edu.vn	<1%
62	Internet	fpt.ai	<1%
63	Internet	fpt.utb.edu.vn	<1%
64	Internet	ndwrpi.gov.vn	<1%
65	Internet	vnn.vietnamnet.vn	<1%
66	Internet	www.baokiemtoannhanuoc.vn	<1%
67	Internet	www.grad.hcmut.edu.vn	<1%

68	Internet	www.semanticscholar.org	<1%
69	Internet	www.slideshare.net	<1%
70	Internet	www.tailieudaihoc.com	<1%
71	Internet	www.timtailieu.vn	<1%
72	Internet	www.vietnamnhadat.vn	<1%
73	Xuất bản	Hanoi National University of Education	<1%

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG OCR

NHỆ OCRLITE NHẬN DẠNG VĂN BẢN

TIẾNG ANH TRÊN ẢNH TÀI LIỆU

SVTH: TRƯƠNG NGUYỄN QUỐC THẮNG

MSSV: 22119231

GVHD: Th.s ĐẬU TRỌNG HIỂN

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG OCR

NHỆ OCRLITE NHẬN DẠNG VĂN BẢN

TIẾNG ANH TRÊN ẢNH TÀI LIỆU

**SVTH: TRƯƠNG NGUYỄN QUỐC THẮNG
MSSV: 22119231**

GVHD: Th.s ĐẬU TRỌNG HIỀN

TP. HỒ CHÍ MINH – 06/2026

1

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT

THÀNH PHỐ HỒ CHÍ MINH

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

KHOA ĐIỆN – ĐIỆN TỬ

Độc lập – Tự do – Hạnh phúcNGÀNH **KTMT (CLC)****NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP**

1

Họ và tên sinh viên: Trương Nguyễn Quốc Thắng

MSSV: 22119231

1. Tên đề tài: Thiết kế và xây dựng hệ thống OCR nhẹ OCRLite nhận dạng văn bản tiếng Anh trên ảnh tài liệu.

24

2. Các số liệu, tài liệu ban đầu:

- Nghiên cứu tổng quan về OCR, nhận dạng chuỗi ký tự, CRNN, CTC và các hướng phát hiện vùng chữ trên ảnh tài liệu.
- Tìm hiểu PyTorch, OpenCV, ONNX Runtime, FastAPI và các công nghệ phục vụ huấn luyện, triển khai hệ thống OCR.
- Nghiên cứu dữ liệu MJSynth, synthetic document, manifest CSV/JSONL và phương pháp đánh giá CER, WER, IoU.
- Xây dựng ý tưởng và kiến trúc tổng thể của hệ thống OCRLite gồm Document Detector, Text Recognizer và runtime package.

11

3. Nội dung thực hiện đề tài:

- Thiết kế kiến trúc hệ thống OCRLite, pipeline dữ liệu, mô hình nhận dạng văn bản và mô hình phát hiện vùng chữ.
- Huấn luyện Text Recognizer CRNN + CTC và Document Detector FastTextMaskUNet; phân tích metric và lỗi nhận dạng.
- Export mô hình sang ONNX, xây dựng OCR Engine, CLI picture_to_text.py và package runtime ocr_submission_package_final.
- Kiểm thử demo OCR full-page, benchmark thời gian/RAM và so sánh với RapidOCR, EasyOCR trong cùng điều kiện thử nghiệm.
- Viết báo cáo đồ án, hoàn thiện tài liệu hướng dẫn, kết quả thực nghiệm và package nộp.

2

4. Sản phẩm:

- Package OCRLite gồm mô hình ONNX, cấu hình runtime, CLI/Python API và artifact kết quả thực nghiệm.
- Bộ báo cáo, mã nguồn, hình ảnh minh chứng, biểu đồ huấn luyện và kết quả benchmark.
- Quyền báo cáo đồ án tốt nghiệp.

1

TRƯỞNG NGÀNH

GIẢNG VIÊN HƯỚNG DẪN

LỜI CẢM ƠN

Nhóm thực hiện xin gửi lời cảm ơn chân thành đến Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, đặc biệt là Khoa Điện - Điện tử, đã tạo điều kiện thuận lợi và trang bị những kiến thức nền tảng cần thiết trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp.

Nhóm xin bày tỏ lòng biết ơn sâu sắc đến thầy Đậu Trọng Hiền, giảng viên hướng dẫn, đã tận tình định hướng, góp ý và hỗ trợ nhóm trong quá trình thực hiện đề tài. Những góp ý chuyên môn của thầy giúp nhóm hoàn thiện kiến trúc hệ thống, nội dung báo cáo và quá trình đánh giá thực nghiệm.

Bên cạnh đó, nhóm xin chân thành cảm ơn quý thầy cô Khoa Điện - Điện tử đã truyền đạt kiến thức và kỹ năng chuyên môn trong suốt quá trình học tập, tạo nền tảng để nhóm áp dụng vào việc nghiên cứu và triển khai hệ thống OCRLite.

Mặc dù đã nỗ lực hoàn thành đồ án trong phạm vi thời gian cho phép, nội dung báo cáo và hệ thống triển khai vẫn khó tránh khỏi hạn chế. Nhóm mong nhận được các góp ý chuyên môn để tiếp tục cải thiện chất lượng đề tài trong các hướng phát triển tiếp theo.

Nhóm thực hiện xin chân thành cảm ơn.

LỜI CAM ĐOAN

Nhóm tác giả gồm Trương Nguyễn Quốc Thắng xin cam đoan rằng đề tài “Thiết kế và xây dựng hệ thống OCR nhẹ OCRLite nhận dạng văn bản tiếng Anh trên ảnh tài liệu” được thực hiện dưới sự hướng dẫn của thầy Đậu Trọng Hiền là kết quả nghiên cứu và triển khai của chính nhóm, không sao chép, không mua từ bất kỳ nguồn nào khác. Tất cả tài liệu, số liệu và nội dung tham khảo sử dụng trong đồ án đều được trích dẫn rõ ràng và đầy đủ theo đúng quy định. Kết quả trình bày trong báo cáo phản ánh trung thực quá trình nghiên cứu, xây dựng hệ thống và thực nghiệm đã thực hiện. Nhóm sinh viên xin hoàn toàn chịu trách nhiệm trước khoa và nhà trường về những nội dung đã cam đoan trong báo cáo này.

Sinh viên thực hiện đồ án tốt nghiệp

Sinh viên

TÓM TẮT

Đồ án “Thiết kế và xây dựng hệ thống OCR nhẹ OCRLite nhận dạng văn bản tiếng Anh trên ảnh tài liệu” được thực hiện nhằm xây dựng một pipeline OCR có khả năng phát hiện vùng chữ, nhận dạng nội dung và xuất văn bản từ ảnh tài liệu full-page.

Hệ thống OCRLite được thiết kế gồm hai thành phần chính: Document Detector để phát hiện vùng chữ trên ảnh tài liệu và Text Recognizer để nhận dạng nội dung từng vùng crop. Pipeline huấn luyện sử dụng PyTorch với mô hình CRNN kết hợp CTC cho recognizer và FastTextMaskUNet cho detector. Sau huấn luyện, các mô hình được export sang ONNX để chạy bằng ONNX Runtime trong package ocr_submission_package_final. Hệ thống hỗ trợ hàm picture_to_text(), CLI, OCREngine và các artifact đánh giá như CER, WER, IoU, biểu đồ loss, kết quả benchmark thời gian/RAM so với RapidOCR và EasyOCR. Kết quả thực nghiệm cho thấy OCRLite có thể đọc phần lớn nội dung trên ảnh bài đọc tiếng Anh thực tế, đạt tốc độ xử lý tốt trong phạm vi thử nghiệm và có khả năng tích hợp vào ứng dụng web hoặc backend Python.

Mục Lục

CHƯƠNG 1: GIỚI THIỆU.	14
1.1 Giới thiệu.	14
1.2 Tình hình nghiên cứu trong và ngoài nước.	15
1.2.1 Tình hình nghiên cứu trên thế giới.	15
1.2.2 Tình hình nghiên cứu trong nước.	16
1.3 Mục tiêu đề tài	17
1.4 Phương pháp nghiên cứu.	18
1.5 Giới hạn đề tài	19
1.6 Bố cục đề tài.	20
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	21
2.1 Tổng quan về Nhận dạng ký tự quang học (OCR)	21
2.1.1 Khái niệm và vai trò của OCR	21
2.1.2 Lịch sử phát triển và xu hướng hiện nay	22
2.1.3 Các thách thức thường gặp trong OCR thực tế	23
2.1.4 Ứng dụng OCR trong giáo dục và website	23
2.2 Phân loại và kiến trúc pipeline OCR	24
2.2.4 OCR truyền thống	24
2.2.5 OCR dựa trên deep learning	24
2.2.6 Pipeline tách Detector và Recognizer	25
2.3 Mạng tích chập (CNN) trong xử lý ảnh văn bản	27
2.3.4 Convolution và ý nghĩa trong OCR	27
2.3.5 Pooling, Batch Normalization và ReLU	27
2.3.6 Feature map và receptive field	27
2.4 Mạng recurrent và kiến trúc CRNN	28
2.4.4 RNN, LSTM và vanishing gradient	28

2.4.5 Kiến trúc CRNN	29
2.4.6 Linear classifier và số timestep	30
2.4.7 Attention-based recognizer (tham khảo)	31
2.5 Connectionist Temporal Classification (CTC)	31
2.5.4 Bài toán alignment trong sequence labeling	31
2.5.5 Blank label và quy tắc collapse	32
2.5.6 CTC Loss, greedy decode và beam search.....	32
2.6 Phát hiện vùng chữ bằng Semantic Segmentation	33
2.6.4 Semantic segmentation và ứng dụng OCR	33
2.6.5 U-Net và FastTextMaskUNet	33
2.6.6 BCE, Dice loss và IoU	34
2.6.7 Object detection vs segmentation (so sánh ngắn)	34
2.7 Xử lý ảnh, tiền xử lý và augmentation	35
2.7.1 Grayscale, resize và normalize.....	35
2.7.2 Nhị phân hóa Otsu và morphology.....	35
2.7.3 Data augmentation.....	36
2.7.4 Charset, manifest và label encoding.....	37
2.8 Dữ liệu huấn luyện OCR phổ biến.....	37
2.8.1 MJSynth và synthetic word crop.....	38
2.8.2 IIIT5K và benchmark	38
2.8.3 Synthetic document cho detector.....	38
2.9 ONNX và ONNX Runtime.....	39
2.9.1 Khái niệm ONNX.....	39
2.9.2 ONNX Runtime và lý do OCRLite export ONNX	40
2.9.3 TensorRT và quantization (tham khảo)	40
2.10 Các metric đánh giá hệ thống OCR.....	41
2.10.1 Character Error Rate (CER).....	41
2.10.2 Word Error Rate (WER) và exact match	41

2.10.3 IoU và metric phân tầng.....	41
2.10.4 Benchmark hiệu năng như metric bổ sung.....	42
2.11 Công nghệ triển khai liên quan	42
2.11.1 PyTorch	42
2.11.2 OpenCV	43
2.11.3 FastAPI và tích hợp web	43
2.11.4 Config-driven experiment	43
2.13 Bổ sung cơ sở lý thuyết phục vụ thiết kế OCRLite	44
2.13.1 Quan hệ giữa dữ liệu, mô hình và lỗi runtime.....	44
2.13.2 Vì sao dữ liệu synthetic vẫn có giá trị.....	45
2.13.3 Vai trò của decoder và hậu xử lý ngôn ngữ.....	46
2.13.4 Nguyên tắc đọc metric trong báo cáo đồ án.....	47
2.14 Tổng kết chương.....	48
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....	50
3.1 Phân tích yêu cầu hệ thống.....	50
3.1.1 Bối cảnh và mục tiêu thiết kế.....	50
3.1.2 Yêu cầu chức năng	51
3.1.3 Yêu cầu phi chức năng.....	51
3.1.4 Phạm vi và giới hạn thiết kế.....	52
3.1.5 Kịch bản sử dụng	52
3.2 Kiến trúc tổng thể.....	52
3.2.1 Training side và Runtime side	52
3.2.2 Tách Detector, Layout và Recognizer	53
3.2.3 Luồng training và runtime.....	54
3.2.4 Phương án đã xem xét và loại bỏ.....	55
3.3 Thiết kế dữ liệu và tiền xử lý	56
3.3.1 Vòng đời dữ liệu.....	56
3.3.2 Manifest và chuẩn hóa.....	56

3.3.3 Synthetic document	57
3.3.4 Tiền xử lý và augmentation.....	57
3.3.5 Charset.....	58
3.4 Thiết kế mô hình và huấn luyện.....	58
3.4.1 Text Recognizer — CRNN + CTC.....	58
3.4.2 Huấn luyện và fine-tune recognizer.....	59
3.4.3 Document Detector — FastTextMaskUNet	60
3.4.4 Vòng lặp huấn luyện	62
3.5 Thiết kế inference, layout và runtime.....	62
3.5.1 Layout post-processing.....	62
3.5.2 OCREngine và OCRResult.....	63
3.5.3 Phân tích pipeline runtime (16 bước).....	64
3.5.4 ONNX export và package runtime	66
3.5.5 Module runtime chính	67
3.6 API, CLI và tích hợp	67
3.6.1 Python API và Django.....	67
3.6.2 CLI và FastAPI.....	67
3.6.3 So sánh với tesseract_ocr legacy	68
3.7 Tổ chức source code	69
3.7.1 Module training và evaluate	70
3.8 Kiểm thử và debug	70
3.8.1 Triển khai production (tóm tắt).....	71
3.9 Thiết kế yêu cầu và phạm vi nghiệp vụ	71
3.10 Kiến trúc hai nhánh train và runtime	76
3.11 Thiết kế dữ liệu recognizer	82
3.12 Thiết kế dữ liệu detector	87
3.13 Thiết kế CRNN Recognizer	92

3.14	Thiết kế Document Detector	97
3.15	Thiết kế layout post-processing	103
3.16	Thiết kế runtime OCREngine	108
3.17	Thiết kế ONNX deployment	113
3.18	Thiết kế API, CLI và tích hợp web	118
3.19	Tổng kết chương	123
CHƯƠNG 4: KẾT QUẢ THỰC HIỆN		125
4.1	Môi trường thực nghiệm và phương pháp đánh giá	125
4.1.1	Môi trường huấn luyện	125
4.1.2	Môi trường runtime và benchmark	125
4.2	Kết quả huấn luyện Text Recognizer	126
4.2.1	Base recognizer trên MJSynth	126
4.2.2	Fine-tune recognizer trên document crop	127
4.3	Kết quả huấn luyện Document Detector	128
4.4	Phân tích loss và metric theo tầng	130
4.5	Demo OCR full-page trên ảnh thực tế	131
4.5.1	Quy trình demo runtime	131
4.5.2	Kết quả trên ảnh bài đọc Thylacine	131
4.6	Benchmark hiệu năng: OCRLite, RapidOCR và EasyOCR ...	132
4.6.1	Thiết lập benchmark	132
4.6.2	Phân tích thời gian xử lý	133
4.6.3	Phân tích mức sử dụng RAM	133
4.6.4	So sánh chất lượng đầu ra (định tính)	134
4.7	Kiểm thử tích hợp CLI, API và package runtime	134
4.7.1	CLI picture_to_text	134
4.7.2	Python API và OCREngine	135

4.7.3 FastAPI service và benchmark API 135

4.8 Phân tích lỗi và hạn chế thực nghiệm.....	136
4.8.1 Phân loại lỗi theo tầng	136
4.8.2 Domain gap synthetic vs ảnh thật.....	136
4.8.3 Hạn chế benchmark.....	136
4.9 Đánh giá mức độ đạt mục tiêu đề án	137
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.	139
5.1 Kết luận	139
5.1.1 Kết quả đạt được	139
5.1.2 Hạn chế.....	141
5.2 Hướng phát triển.	142
5.2.1 Bổ sung dữ liệu thật và fine-tune detector	142
5.2.2 Nâng cấp layout analysis	142
5.2.3 Cải thiện recognizer và hậu xử lý văn bản.....	143
5.2.4 Mở rộng tiếng Việt và đa ngôn ngữ.....	143
5.2.5 Hoàn thiện tích hợp sản phẩm và tối ưu hiệu năng	143
5.3 Tài liệu tham khảo.....	143

CHƯƠNG 1: GIỚI THIỆU

1.1 Giới thiệu

Trong bối cảnh chuyển đổi số, văn bản số chiếm tỷ trọng ngày càng lớn trong giáo dục, quản trị tài liệu và các ứng dụng web. Ảnh chụp màn hình bài đọc, tài liệu PDF quét, hình ảnh slide hay ảnh chụp trang sách đều cần được chuyển thành dữ liệu văn bản có thể tìm kiếm, sao chép và xử lý tiếp. Công nghệ Nhận dạng ký tự quang học (Optical Character Recognition — OCR) đáp ứng nhu cầu đó bằng cách tự động trích xuất chuỗi ký tự từ ảnh.

Các engine OCR mã nguồn mở phổ biến như Tesseract, EasyOCR hay PaddleOCR đã chứng minh khả năng đọc chữ trên nhiều loại ảnh. Tuy nhiên, khi triển khai vào website hoặc dịch vụ backend có yêu cầu về tốc độ phản hồi, mức tiêu thụ bộ nhớ và khả năng đóng gói độc lập, việc sử dụng trực tiếp các giải pháp nặng đôi khi không tối ưu. Một số engine tiêu tốn RAM lớn, thời gian khởi tạo lâu hoặc phụ thuộc nhiều vào môi trường huấn luyện, gây khó khăn khi tích hợp vào hệ thống web quy mô vừa và nhỏ.

Xuất phát từ thực tế đó, đề án tập trung xây dựng hệ thống OCRLite — một pipeline OCR nhẹ, có thể huấn luyện, đánh giá, export sang ONNX và đóng gói thành module Python gọi trực tiếp từ ứng dụng web. OCRLite không chỉ là một mô hình đọc chữ đơn lẻ mà là chuỗi xử lý hoàn chỉnh: phát hiện vùng chữ trên ảnh tài liệu, tách dòng và từ, nhận dạng từng vùng crop, rồi ghép kết quả thành đoạn văn theo đúng thứ tự đọc. Kiến trúc được tách thành Document Detector (phát hiện vùng chữ) và Text Recognizer (đọc nội dung crop) nhằm dễ huấn luyện, dễ debug và dễ thay thế từng thành phần.

Hệ thống hướng đến nhận dạng văn bản tiếng Anh trên ảnh full-page, đặc biệt các ảnh chụp màn hình bài đọc, tài liệu đơn cột với chữ tối trên nền sáng. Pipeline huấn luyện sử dụng PyTorch với mô hình CRNN kết hợp CTC cho recognizer và FastTextMaskUNet cho detector. Phần runtime triển khai bằng ONNX Runtime, cho phép máy demo chỉ cần file ONNX và config mà không phải cài đặt lại toàn bộ môi trường train. Hàm `picture_to_text()` và `OCREngine` trong package `ocr_submission_package_final` là điểm tích hợp trực tiếp vào backend hoặc website.

Nhóm thực hiện lựa chọn đề tài: “Thiết kế và xây dựng hệ thống OCR nhẹ OCRLite nhận dạng văn bản tiếng Anh trên ảnh tài liệu”. Đề tài nhằm hoàn thiện một vòng OCR đầy đủ trong phạm vi đồ án: chuẩn bị dữ liệu, huấn luyện, phân tích lỗi, export mô hình, benchmark hiệu năng và đóng gói sản phẩm có thể chạy thử ngay.

[Hình minh họa — chèn ảnh sau khi ghép báo cáo]

Hình 1.1. Sơ đồ khối tổng quan hệ thống OCRLite (training side và runtime side).

1.2 Tình hình nghiên cứu trong và ngoài nước

1.2.1 Tình hình nghiên cứu trên thế giới

OCR là bài toán cổ điển trong thị giác máy tính, với nhiều hướng tiếp cận từ phương pháp truyền thống dựa trên feature thủ công đến deep learning end-to-end. Tesseract, do Google phát triển, vẫn được sử dụng rộng rãi nhờ mã nguồn mở và hỗ trợ đa ngôn ngữ. EasyOCR và PaddleOCR đại diện cho các framework deep learning hiện đại, tích hợp sẵn detector và recognizer, cho chất lượng tốt trên nhiều loại ảnh nhưng thường nặng hơn về bộ nhớ và thời gian inference.

Về mặt lý thuyết, kiến trúc CRNN (Convolutional Recurrent Neural Network) kết hợp Connectionist Temporal Classification (CTC) do Shi và cộng sự đề xuất đã trở thành nền tảng phổ biến cho nhận dạng chuỗi ký tự trên ảnh [1]. CNN trích đặc trưng không gian, BiLSTM mô hình hóa phụ thuộc chuỗi theo chiều ngang, còn CTC cho phép huấn luyện mà không cần alignment từng ký tự. Hướng tiếp cận này phù hợp với dữ liệu word crop hoặc text line có nhãn dạng chuỗi.

Song song đó, bài toán phát hiện vùng chữ trên ảnh tài liệu được nghiên cứu theo object detection (EAST, DBNet) hoặc semantic segmentation (U-Net) [3]. U-Net với cấu trúc encoder-decoder và skip connection phù hợp khi cần dự đoán mask vùng chữ pixel-wise. Khi dữ liệu synthetic có thể sinh mask tự động từ tọa độ paste word crop, segmentation detector trở thành lựa chọn thực dụng hơn so với huấn luyện end-to-end phức tạp.

Xu hướng triển khai production gần đây nhấn mạnh tách training stack và inference runtime. PyTorch dùng để huấn luyện, sau đó export sang ONNX và chạy bằng ONNX

Runtime hoặc TensorRT [4]. Cách làm này giảm phụ thuộc môi trường, tăng khả năng tái lập và thuận lợi khi tích hợp vào API web. Các benchmark công khai trên tập IIIT5K, ICDAR hay MJSynth thường dùng CER (Character Error Rate), WER (Word Error Rate) và exact match để đánh giá recognizer; IoU dùng cho detector segmentation.

Các nghiên cứu ứng dụng OCR vào giáo dục và tài liệu số cho thấy nhu cầu đọc ảnh screenshot, slide và bài đọc trực tuyến ngày càng tăng. Tuy nhiên, ít đồ án sinh viên xây dựng pipeline từ dữ liệu, train, export ONNX, post-processing layout và benchmark so sánh với engine mã nguồn mở trong cùng điều kiện thử nghiệm. Khoảng trống đó là cơ sở để đồ án hướng tới một hệ thống OCR tự xây dựng, có thể kiểm chứng bằng số liệu và có package runtime sẵn sàng tích hợp.

1.2.2 Tình hình nghiên cứu trong nước

Tại Việt Nam, OCR được nghiên cứu và ứng dụng trong nhiều lĩnh vực: số hóa tài liệu, nhận dạng biển số, đọc hóa đơn, xử lý văn bản hành chính và hỗ trợ học tập trực tuyến. Các trường đại học khối kỹ thuật và công nghệ thông tin thường giao các đồ án liên quan đến xử lý ảnh, deep learning hoặc ứng dụng web có tích hợp OCR. Phần lớn đồ án tập trung vào sử dụng thư viện có sẵn (Tesseract, EasyOCR) hoặc gọi API cloud, chưa đi sâu vào thiết kế pipeline train-evaluate-deploy end-to-end.

Một số đồ án sinh viên đã triển khai nhận dạng chữ viết tay tiếng Việt, OCR biển số xe hoặc đọc CMND/CCCD bằng CNN/RNN. Các đề tài này thường tập trung vào một mô hình recognizer trên ảnh đã crop sẵn, hoặc một detector đơn giản cho bài toán hẹp. Việc xử lý ảnh full-page tài liệu tiếng Anh — gồm detect, layout, recognize và ghép văn bản — ít được trình bày đầy đủ trong một package có thể chạy độc lập bằng ONNX.

Trong bối cảnh các nền tảng giáo dục trực tuyến và website quản lý lớp học (LMS), nhu cầu chuyển ảnh bài đọc, đề bài scan hoặc screenshot thành text để lưu trữ, tìm kiếm hoặc chấm bài tự động ngày càng rõ. Tuy nhiên, việc gắn engine OCR nặng trực tiếp vào backend Django/FastAPI có thể gây áp lực lên RAM và thời gian phản hồi. Do đó, hướng nghiên cứu xây dựng OCR nhẹ, export ONNX, có API Python/CLI gọn và có báo cáo metric minh chứng là hướng đi phù hợp với đồ án kỹ thuật máy tính.

So với các đồ án trong nước đã khảo sát, điểm khác biệt của OCRLite nằm ở việc: (1) tách rõ detector segmentation và recognizer CRNN+CTC; (2) có pipeline synthetic document sinh mask tự động; (3) fine-tune recognizer trên document crop gần runtime; (4) export ONNX và đóng gói package runtime độc lập; (5) benchmark so sánh với RapidOCR và EasyOCR trên cùng bộ ảnh thử nghiệm. Những đóng góp này bám sát package ocr_submission_package_final đã triển khai thực tế trong đồ án.

1.3 Mục tiêu đề tài

Mục tiêu tổng quát của đồ án là thiết kế và xây dựng hệ thống OCRLite — pipeline OCR nhẹ, có thể huấn luyện lại, đánh giá bằng metric chuẩn, export sang ONNX và tích hợp vào ứng dụng Python/web thông qua API picture_to_text(). Hệ thống phải đọc được văn bản tiếng Anh từ ảnh tài liệu full-page, không chỉ từ ảnh word crop có sẵn.

Để đạt mục tiêu tổng quát, nhóm thực hiện các mục tiêu cụ thể sau:

- Xây dựng pipeline dữ liệu cho recognizer và detector: chuẩn hóa manifest CSV/JSONL, tích hợp nguồn MJSynth, synthetic document, IIIT5K; chia train/val/test có kiểm soát.
 - Huấn luyện Text Recognizer dạng CRNN + CTC trên word crop tiếng Anh; đạt CER validation tốt (base recognizer best_val_cer $\approx 0,0403$; fine-tuned $\approx 0,0315$).
 - Huấn luyện Document Detector dạng segmentation FastTextMaskUNet trên synthetic pages; đạt val_iou cao trên tập validation ($\approx 0,975$).
 - Thiết kế post-processing layout: chuyển mask thành box, tách dòng/từ, crop và ghép text theo reading order.
 - Export mô hình sang ONNX, đóng gói runtime trong thư mục models/; triển khai OCREngine, CLI picture_to_text.py và hỗ trợ tích hợp FastAPI.
 - Đánh giá hệ thống bằng CER, WER, exact match (recognizer), IoU (detector); benchmark thời gian và RAM so với RapidOCR, EasyOCR trên cùng bộ ảnh.
 - Hoàn thiện báo cáo đồ án, tài liệu hướng dẫn (huong_dan.md, SUBMISSION_README.md) và package nộp ocr_submission_package_final.
- Tiêu chí thành công của đồ án không chỉ dựa trên metric validation. Hệ thống được

51 coi là đạt yêu cầu khi hàm `picture_to_text()` chạy ổn định trên package cuối, cho output có thể sử dụng được trên ảnh bài đọc thực tế, đồng thời tốc độ và mức RAM nằm trong phạm vi chấp nhận được so với các engine OCR mã nguồn mở trong điều kiện thử nghiệm.

31 1.4 Phương pháp nghiên cứu

2 Đề án áp dụng phương pháp nghiên cứu kết hợp giữa nghiên cứu lý thuyết, thiết kế hệ thống và thực nghiệm có đo lường. Quy trình thực hiện gồm các bước chính sau.
8

Bước 1 — Nghiên cứu lý thuyết và khảo sát hiện trạng:

- Tổng hợp tài liệu về OCR, CRNN, CTC, U-Net segmentation, ONNX Runtime và các engine mã nguồn mở (Tesseract, EasyOCR, RapidOCR, PaddleOCR).
- 56 • Phân tích ưu/nhược điểm từng hướng tiếp cận: end-to-end, detection+recognition, runtime ONNX so với PyTorch native.

34 Bước 2 — Phân tích yêu cầu và thiết kế kiến trúc:

- 28 • Xác định phạm vi bài toán: tiếng Anh, ảnh nền sáng, tài liệu đơn cột; yêu cầu chức năng và phi chức năng (tốc độ, RAM, khả năng tích hợp).
- Thiết kế kiến trúc hai nhánh training side và runtime side; tách Document Detector và Text Recognizer; định nghĩa luồng dữ liệu từ ảnh đầu vào đến OCRResult.

Bước 3 — Xây dựng pipeline dữ liệu:

- Export MJSynth từ Hugging Face; sinh synthetic bằng TRDG hoặc generator nội bộ; build document synthetic pages và mask cho detector; tạo dữ liệu fine-tune recognizer từ document crops.
- Chuẩn hóa manifest, lọc charset, dedupe, split train/val/test; áp dụng augmentation phù hợp recognizer và detector.

Bước 4 — Huấn luyện và tối ưu mô hình:

- Train recognizer bằng PyTorch (AdamW, AMP, early stopping); chọn checkpoint theo CER thấp nhất; fine-tune trên document crop với learning rate nhỏ.
- Train detector bằng BCE + Dice loss; hỗ trợ multi-GPU trên Kaggle T4 x2; chọn checkpoint theo val_iou cao nhất.

Bước 5 — Export, triển khai và kiểm thử:

- Export recognizer và detector sang ONNX; đóng gói models/ cùng config JSON; triển khai OCREngine và CLI/API.
- Kiểm thử tách lỗi theo tầng (detector, layout, recognizer, end-to-end); benchmark thời gian/RAM; so sánh với RapidOCR và EasyOCR.

Bước 6 — Đánh giá và báo cáo:

- Phân tích predictions.csv, error_samples.csv, char_confusion.csv; ghi nhận lỗi thực tế trên ảnh screenshot; tổng hợp kết quả vào báo cáo đồ án.

Công cụ sử dụng gồm Python, PyTorch, OpenCV, ONNX Runtime, FastAPI; môi trường huấn luyện chính trên Kaggle GPU; package runtime chạy được trên CPU hoặc GPU tùy provider. Mọi thí nghiệm được mô tả bằng file config JSON và lưu artifact trong thư mục result/ để đảm bảo khả năng tái lập.

1.5 Giới hạn đề tài

Do phạm vi **thời gian** và nguồn lực đồ án, hệ thống OCRLite có các giới hạn sau. Việc nhận thức rõ các giới hạn này giúp đánh giá kết quả **một cách khách quan và định hướng phát triển** trong tương lai.

- Ngôn ngữ: hệ thống chỉ hỗ trợ tiếng Anh (charset gồm chữ cái, số, khoảng trắng và dấu câu thông dụng). Chưa hỗ trợ tiếng Việt có dấu, đa ngôn ngữ hay chữ viết tay.
- Định dạng ảnh đầu vào: giả định chữ tối trên nền sáng, tài liệu đơn cột hoặc ảnh screenshot bài đọc. Chưa xử lý tốt bảng biểu phức tạp, tài liệu nhiều cột, ảnh nghiêng lớn hoặc nền nhiễu cao.
- Dữ liệu detector chủ yếu là synthetic document (paste word crops lên trang trắng). Val_iou cao trên synthetic chưa đồng nghĩa detector luôn ổn định trên mọi ảnh thật; domain gap vẫn tồn tại.
- Output recognizer dùng text_case = lower nên không giữ nguyên chữ hoa/chữ thường của văn bản gốc.
- Post-processing layout dựa trên contour, morphology và phân tích khoảng trắng — phù hợp ảnh demo nhưng chưa thay thế mô hình layout analysis cho tài liệu phức tạp.

- FastAPI service hiện thiên về OCR crop/line; endpoint full-page OCR hoàn chỉnh cần bổ sung gọi OCREngine.read nếu triển khai production.
- Benchmark so sánh RapidOCR/EasyOCR chưa có ground truth chung để tính CER/WER tuyệt đối cho cả ba engine; phần so sánh chất lượng mang tính định tính kèm metric thời gian/RAM.
- Một số lỗi OCR thực tế vẫn còn trên ảnh thử nghiệm (dấu câu, dấu gạch nối, dính ký tự ví dụ thylacine's → thylacinels), dù metric validation đã cải thiện sau fine-tune.

1.6 Bố cục đề tài

Nội dung đề án được trình bày trong 05 chương, bám sát quy trình nghiên cứu — thiết kế — triển khai — đánh giá của hệ thống OCRLite. Cụ thể như sau:

Chương 1 — Giới thiệu: Trình bày bối cảnh, lý do chọn đề tài, tình hình nghiên cứu trong và ngoài nước, mục tiêu, phương pháp nghiên cứu, giới hạn và bố cục đề án.

Chương 2 — Cơ sở lý thuyết: Trình bày kiến thức nền về OCR, CRNN, CTC, U-Net segmentation, pipeline detect–recognize, ONNX Runtime, các metric đánh giá (CER, WER, IoU) và tổng quan các công nghệ liên quan (PyTorch, OpenCV, FastAPI).

Chương 3 — Thiết kế hệ thống: Mô tả chi tiết kiến trúc OCRLite, thiết kế dữ liệu, mô hình recognizer/detector, post-processing layout, luồng training và runtime, tổ chức source code trong package ocr_submission_package_final.

Chương 4 — Kết quả thực hiện: Trình bày kết quả huấn luyện (CER, IoU, biểu đồ loss), demo OCR trên ảnh thực tế, benchmark so sánh với RapidOCR và EasyOCR, phân tích lỗi và minh chứng tích hợp API/CLI.

Chương 5 — Kết luận và hướng phát triển: Tổng hợp kết quả đạt được, hạn chế, đánh giá mức độ đáp ứng mục tiêu thiết kế và đề xuất hướng mở rộng (tiếng Việt, dữ liệu thật, layout phức tạp, TensorRT, API full-page).

Ngoài nội dung chính, đề án kèm theo danh mục hình ảnh, danh mục bảng biểu, danh mục từ viết tắt, tài liệu tham khảo và phụ lục (mapping file code, kết quả benchmark, cấu hình mô hình) khi ghép thành báo cáo hoàn chỉnh.

3

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Chương này trình bày các kiến thức nền tảng cần thiết để hiểu, thiết kế và triển khai hệ thống OCRLite trong phạm vi đề án. Khác với Chương 1 mang tính giới thiệu tổng quan, Chương 2 đi sâu vào lý thuyết: bản chất bài toán OCR, các hướng tiếp cận trong thực tế, mạng nơ-ron tích chập, kiến trúc CRNN kết hợp CTC, semantic segmentation với U-Net, tiền xử lý ảnh, export ONNX, metric đánh giá và các công nghệ triển khai liên quan. Nội dung được sắp xếp theo trình tự từ tổng quát đến cụ thể, có liên hệ trực tiếp với các quyết định thiết kế đã áp dụng trong package ocr_submission_package_final.

Theo kinh nghiệm đọc tài liệu và triển khai thực tế, phần lý thuyết OCR không nên tách rời hoàn toàn khỏi pipeline cụ thể. Mỗi khái niệm trong chương này đều gắn với một quyết định trong OCRLite: CTC → manifest CSV chỉ cần text; U-Net → synthetic mask; ONNX → package runtime; metric phân tầng → debug theo box/crop/text. Cách trình bày này giúp người đọc thấy được mạch logic từ lý thuyết đến hiện thực.

Khi đọc chương này, người đọc cần nắm được vì sao đề án không chọn một engine OCR có sẵn mà xây dựng pipeline riêng; vì sao tách detector và recognizer thay vì end-to-end; vì sao CTC phù hợp dữ liệu chỉ có chuỗi label; vì sao export ONNX cho runtime. Những câu hỏi đó sẽ được trả lời dần qua từng mục, kết hợp trích dẫn tài liệu tham khảo [1]–[12] ở cuối báo cáo.

2.1 Tổng quan về Nhận dạng ký tự quang học (OCR)

2.1.1 Khái niệm và vai trò của OCR

Nhận dạng ký tự quang học (Optical Character Recognition — OCR) là quá trình tự động chuyển đổi hình ảnh chứa văn bản thành dữ liệu văn bản (text) mà máy tính có thể lưu trữ, tìm kiếm, sao chép và xử lý tiếp. Trong bối cảnh chuyển đổi số, OCR đóng vai trò cầu nối giữa tài liệu giấy, ảnh chụp, screenshot và hệ thống thông tin số. Ví dụ thực tế gồm: số hóa sách báo, đọc biển số xe, trích xuất thông tin hóa đơn, hỗ trợ người khiếm thị, hoặc chuyển ảnh bài đọc trên LMS thành text để học sinh tra cứu.

Về mặt hình thức, OCR có thể được mô tả như một bài toán ánh xạ: cho ảnh đầu vào

I, hệ thống cần ước lượng chuỗi ký tự T sao cho T phản ánh đúng (hoặc gần đúng) nội dung chữ xuất hiện trên I. Tuy nhiên, ánh xạ này không đơn giản vì ảnh thực tế chứa nhiều, biến dạng hình học, đa dạng font chữ, bố cục phức tạp và đôi khi cả yếu tố màu sắc, bóng đổ. Do đó, hầu hết hệ thống OCR hiện đại đều chia nhỏ bài toán thành nhiều giai đoạn xử lý thay vì giải quyết một bước duy nhất.

Trong phạm vi đồ án, OCR được hiểu theo nghĩa hẹp hơn: nhận dạng văn bản tiếng Anh in trên ảnh tài liệu full-page, đặc biệt là ảnh chụp màn hình bài đọc, trang tài liệu đơn cột với chữ tối trên nền sáng. Phạm vi này được chọn có chủ đích để tập trung hoàn thiện một vòng pipeline train–evaluate–deploy thay vì dàn trải sang chữ viết tay, đa ngôn ngữ hay bảng biểu phức tạp — những bài toán đòi hỏi nguồn lực và thời gian vượt quá khả năng một đồ án tốt nghiệp.

2.1.2 Lịch sử phát triển và xu hướng hiện nay

Giai đoạn đầu của OCR (khoảng thập niên 1960–1990) dựa chủ yếu vào xử lý ảnh cổ điển và feature thủ công. Quy trình thường gồm: tiền xử lý (lọc nhiễu, nhị phân hóa), phân đoạn ký tự hoặc từ, trích đặc trưng (HOG, profile, moment) rồi phân loại bằng mô hình thống kê như SVM hoặc HMM. Phương pháp này hoạt động tương đối ổn trên ảnh scan chất lượng cao, font chuẩn, nhưng kém linh hoạt trước ảnh chụp thực tế.

Từ khoảng 2010 trở đi, deep learning thay đổi căn bản bài toán OCR. CNN học đặc trưng trực tiếp từ pixel; RNN/LSTM và sau đó Transformer mô hình hóa phụ thuộc chuỗi; CTC và attention mechanism giải quyết bài toán alignment. Các engine mã nguồn mở như Tesseract (có LSTM), EasyOCR, PaddleOCR, RapidOCR phản ánh xu hướng này. Song song, xu hướng triển khai production nhấn mạnh tách training stack (PyTorch) và inference runtime (ONNX Runtime, TensorRT) — hướng mà OCRLite áp dụng.

Có thể nói OCR hiện nay không còn là bài toán “đọc được hay không” đơn thuần mà là bài toán cân bằng giữa độ chính xác, tốc độ, mức tiêu thụ bộ nhớ và khả năng tích hợp vào sản phẩm. Với website giáo dục hoặc backend quy mô vừa, việc gán engine quá nặng có thể gây áp lực lên RAM và thời gian phản hồi — đây cũng là một trong những cơ sở để lựa chọn hướng xây dựng OCRLite nhẹ và export ONNX.

Sơ đồ tổng quát bài toán OCR: từ ảnh đầu vào đến chuỗi văn bản đầu ra.



Hình 2.1. Sơ đồ tổng quát bài toán OCR: từ ảnh đầu vào đến chuỗi văn bản đầu ra.

2.1.3 Các thách thức thường gặp trong OCR thực tế

Dù metric validation có thể tốt, OCR trên ảnh thật vẫn gặp nhiều thách thức. Thứ nhất là nhiễu và chất lượng ảnh: mờ, nén JPEG, screenshot bị scale. Thứ hai là biến dạng hình học: ảnh nghiêng, perspective distortion. Thứ ba là đa dạng font và kích thước chữ trong cùng một trang. Thứ tư là bố cục: nhiều cột, bảng biểu, hình ảnh xen kẽ văn bản. Thứ năm là ký tự đặc biệt: dấu câu, hyphen, apostrophe — nhóm ký tự dễ bị nhầm khi crop quá sát hoặc decode CTC greedy.

OCRLite trong đồ án chưa giải quyết hết các thách thức trên mà tập trung vào nhóm ảnh tương đối “thuận lợi”: nền sáng, một cột, tiếng Anh in. Việc nhận thức rõ giới hạn này quan trọng khi đọc Chương 4 và Chương 5, tránh hiểu nhầm metric validation là cam kết tuyệt đối cho mọi loại ảnh.

2.1.4 Ứng dụng OCR trong giáo dục và website

Trong lĩnh vực giáo dục trực tuyến, OCR hỗ trợ chuyển ảnh bài đọc, slide, đề thi scan thành text để lưu trữ, tìm kiếm hoặc phân tích. Học sinh có thể chụp màn hình bài đọc tiếng Anh và nhận lại nội dung có thể copy — use-case mà OCRLite hướng tới trong bối cảnh tích hợp website Engnite. So với gõ tay, OCR giảm thời gian nhập liệu; so với PDF có text layer sẵn, OCR vẫn cần thiết khi nguồn là ảnh raster.

Tuy nhiên, OCR trong LMS không chỉ là “đọc được chữ”. Hệ thống còn cần xử lý upload ảnh lớn, queue request khi nhiều người dùng đồng thời, và trả lỗi rõ ràng khi ảnh quá mờ hoặc không chứa văn bản. Các yêu cầu phi chức năng này dẫn tới quyết định export ONNX, benchmark RAM/thời gian và thiết kế API mỏng (picture_to_text).

Các engine OCR có sẵn cũng được khảo sát trước khi xây dựng pipeline riêng. EasyOCR và PaddleOCR mạnh về đa ngôn ngữ nhưng nặng; Tesseract nhẹ hơn nhưng kém linh hoạt trên screenshot; RapidOCR cân bằng RAM và tốc độ. Kết quả benchmark Chương 4 cho thấy OCRLite tự xây có thể cạnh tranh về tốc độ — cũng có lý do nghiên cứu cần hiểu pipeline bên trong, không chỉ gọi thư viện.

2.2 Phân loại và kiến trúc pipeline OCR

2.2.1 OCR truyền thống

OCR truyền thống thường được tổ chức thành chuỗi bước rõ ràng: (1) tiền xử lý ảnh — chuyển grayscale, lọc nhiễu, nhị phân hóa (Otsu, adaptive threshold); (2) phân đoạn — tách dòng, từ, ký tự bằng projection profile hoặc connected component; (3) chuẩn hóa — deskew, normalize kích thước; (4) trích đặc trưng — HOG, gradient, structural feature; (5) phân loại — template matching, SVM, HMM.

Tesseract — engine OCR mã nguồn mở do Google phát triển — vẫn được sử dụng rộng rãi và kế thừa nhiều ý tưởng từ hướng truyền thống, đồng thời tích hợp LSTM cho nhận dạng dòng trong các phiên bản mới. Ưu điểm của OCR truyền thống là có thể chạy trên CPU với tài nguyên vừa phải, phù hợp môi trường không có GPU. Nhược điểm là pipeline dài, nhiều tham số cần chỉnh thủ công, và khả năng tổng quát hóa kém trên scene text hoặc ảnh chụp màn hình có font không đồng nhất.

2.2.2 OCR dựa trên deep learning

OCR deep learning thay thế feature thủ công bằng mạng nơ-ron học biểu diễn từ dữ liệu. Hai nhánh chính: (1) phát hiện vùng chữ (text detection) — xác định bounding box hoặc mask nơi có chữ; (2) nhận dạng nội dung (text recognition) — đọc chuỗi ký tự trong vùng đã crop. Các framework như EasyOCR, PaddleOCR, RapidOCR đóng gói sẵn

detector và recognizer, hỗ trợ nhiều ngôn ngữ.

Detector deep learning thường dùng EAST, DBNet, hoặc segmentation (U-Net, DeepLab). Recognizer thường dùng CRNN+CTC, attention-based encoder-decoder (Attention OCR), hoặc Transformer (TrOCR, PARSeq). Deep learning đạt độ chính xác cao hơn trên dữ liệu đa dạng vì mô hình học trực tiếp từ pixel. Chi phí là cần GPU để train, cần dữ liệu gán nhãn hoặc synthetic data, và runtime native thường nặng hơn nếu không export sang ONNX hoặc TensorRT.

Đồ án chọn hướng deep learning với kiến trúc cụ thể: segmentation detector (FastTextMaskUNet) và CRNN+CTC recognizer. Lý do không dùng trực tiếp EasyOCR/PaddleOCR làm lõi: cần chứng minh khả năng train-evaluate-deploy pipeline tự xây, kiểm soát metric, và đóng gói runtime nhẹ phù hợp tích hợp web — mục tiêu mang tính học thuật và kỹ thuật của đồ án tốt nghiệp.

Ngoài ra, các paper và tài liệu kỹ thuật cho thấy xu hướng kết hợp nhiều mô hình chuyên biệt: một model cho text detection, một model cho recognition, đôi khi thêm model cho layout analysis hoặc spell correction. Hướng này tương đối giống tinh thần OCRLite, tức là không cố gắng giải mọi thứ bằng một network duy nhất mà chia nhỏ trách nhiệm để dễ kiểm soát chất lượng từng phần.

11 Tuy nhiên cần lưu ý rằng việc tách module cũng làm tăng số lượng tham số cấu hình cần theo dõi: threshold mask, min_box_area, dilation box, batch size recognizer, v.v. Đây là lý do đồ án dùng config JSON và lưu artifact sau mỗi lần train — nếu không có cơ chế này thì rất khó giải thích vì sao một lần chạy cho kết quả tốt còn lần khác lại kém hơn dù code không đổi.

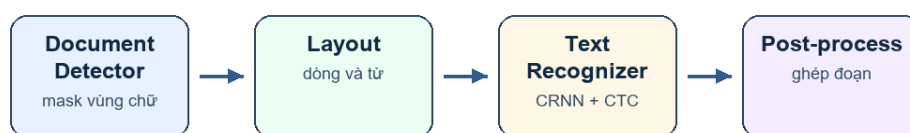
2.2.3 Pipeline tách Detector và Recognizer

Kiến trúc OCRLite tách bài toán thành hai module: Document Detector phát hiện vùng chữ trên ảnh full-page; Text Recognizer đọc nội dung từng crop word hoặc line. Giữa hai module là lớp post-processing layout — chuyển mask thành bounding box, tách dòng, tách từ, sắp xếp reading order, crop ảnh cho recognizer. Output cuối là OCRResult gồm text, lines và boxes.

So với mô hình end-to-end đọc trực tiếp toàn trang sang chuỗi ký tự, pipeline tách module có ưu điểm: dễ debug (mở ảnh box, crop, text từng bước); dễ thay detector hoặc recognizer độc lập; dữ liệu train tách riêng (word crop cho recognizer, page+mask cho detector). Nhược điểm là pipeline dài hơn, lỗi có thể lan truyền giữa các tầng — detector sai một chút thì recognizer nhận crop xấu dù CER validation vẫn tốt.

Luồng runtime có thể tóm tắt: ảnh → grayscale → detector ONNX → mask → threshold → morphology → contour → box → layout (tách dòng/từ) → crop batch → recognizer ONNX → decode CTC → ghép paragraph. Luồng training: manifest → dataloader → train PyTorch → checkpoint → export ONNX. Hai luồng tách biệt rõ ràng là nguyên tắc thiết kế xuyên suốt OCRLite.

Pipeline OCR tách Detector và Recognizer trong hệ thống OCRLite.



Hình 2.2. Pipeline OCR tách Detector và Recognizer trong hệ thống OCRLite.

Bảng 2.1. So sánh một số hướng tiếp cận OCR liên quan đồ án.

Hướng tiếp cận	Ưu điểm	Nhược điểm	Liên quan OCRLite
OCR truyền thống	Nhanh trên CPU, ít phụ thuộc GPU	Kém tổng quát trên ảnh phức tạp	Dùng làm mốc so sánh
End-to-end full-page	Pipeline gọn, ít bước thủ công	Cần nhiều dữ liệu thật, khó debug	Chưa phù hợp phạm vi đồ án
Detector + Recognizer	Modular, dễ thay từng tầng	Cần post-processing layout	Kiến trúc chính của OCRLite
Segmentation + CRNN	Sinh mask synthetic thuận tiện	Phụ thuộc chất lượng tách box	Thiết kế final của package

2.3 Mạng tích chập (CNN) trong xử lý ảnh văn bản

2.3.1 Convolution và ý nghĩa trong OCR

Mạng tích chập (Convolutional Neural Network — CNN) là nền tảng trích đặc trưng trong hầu hết hệ thống OCR hiện đại. Lớp convolution áp dụng bộ lọc (kernel, filter) trượt trên ảnh đầu vào để phát hiện pattern cục bộ: cạnh dọc, cạnh ngang, góc cong, nét chữ. Mỗi kernel học một đặc trưng khác nhau thông qua backpropagation và gradient descent.

Phép tích chập 2D có thể viết dưới dạng: output tại vị trí (i, j) là tổng có trọng số của vùng lân cận trên input nhân với kernel. Stacking nhiều lớp conv cho phép mạng học biểu diễn phân cấp: lớp thấp phát hiện cạnh, lớp cao phát hiện cấu trúc phức tạp hơn như nét chữ, khoảng trắng giữa ký tự. Trong OCR, điều này quan trọng vì ký tự không phải pattern cố định mà phụ thuộc font, cỡ chữ và ngữ cảnh lân cận.

2.3.2 Pooling, Batch Normalization và ReLU

Sau convolution thường là Batch Normalization (BN) — chuẩn hóa activation theo batch để ổn định huấn luyện — và ReLU (Rectified Linear Unit) — $f(x) = \max(0, x)$ — để thêm phi tuyến. Pooling (max pooling hoặc average pooling) giảm kích thước feature map, tăng receptive field và giảm chi phí tính toán.

Trong recognizer CRNN, chiến lược pooling thường giảm chiều cao nhiều hơn chiều rộng để giữ trục ngang làm trục thời gian cho bước RNN/CTC. Ví dụ ảnh input height=32, width=160 sau vài lớp pooling có thể còn height=1, width≈40 timestep — mỗi timestep tương ứng một vùng dọc theo chiều dọc từ trái sang phải.

2.3.3 Feature map và receptive field

Feature map là biểu diễn trung gian sau các lớp CNN: kích thước không gian nhỏ hơn ảnh gốc nhưng số channel lớn hơn. Mỗi vị trí trên feature map cuối có receptive field — vùng ảnh gốc ảnh hưởng đến activation tại vị trí đó. Receptive field lớn giúp mô hình “nhìn” ngữ cảnh rộng hơn quanh mỗi ký tự.

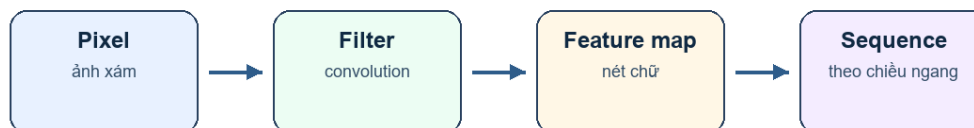
Với document detector, CNN encoder trong U-Net học biểu diễn toàn cục của trang

(vùng chữ vs nền). Decoder upsample về kích thước gốc; skip connection nối feature cùng độ phân giải từ encoder sang decoder để giữ chi tiết vị trí — cần thiết khi vùng chữ nhỏ hoặc sát nhau. OCRLite dùng grayscale 1 channel cho cả detector và recognizer vì bài toán chữ tối/nền sáng không cần thông tin màu phức tạp.

Về mặt trực giác, có thể hình dung CNN đóng vai trò như bộ lọc đa tầng: tầng thấp nhạy với biên dạng cục bộ, tầng cao nhạy với cấu trúc lớn hơn như nét cong của chữ 'o' hay khoảng trống giữa hai từ. Trong đề án, kiến trúc CNN không được thiết kế lại từ con số không mà dựa trên các block conv–BN–ReLU–pooling đã được chứng minh hiệu quả trong CRNN và U-Net, rồi tinh chỉnh width/depth cho phù hợp tài nguyên Kaggle.

Một điểm quan trọng khi đọc tài liệu CNN là receptive field: nếu receptive field quá nhỏ thì mô hình khó phân biệt các kí tự ambiguous dựa trên ngữ cảnh; nếu quá lớn mà feature map quá thô thì vị trí không gian bị mất. CRNN giải quyết phần ngữ cảnh bằng BiLSTM sau CNN, vì vậy CNN không cần mang hết trách nhiệm mô hình hóa chuỗi — đây cũng là lý do kiến trúc CRNN vẫn phổ biến dù Transformer phát triển mạnh.

Minh họa feature map và receptive field trong CNN cho OCR.



Hình 2.3. Minh họa feature map và receptive field trong CNN cho OCR.

2.4 Mạng recurrent và kiến trúc CRNN

2.4.1 RNN, LSTM và vanishing gradient

Recurrent Neural Network (RNN) xử lý chuỗi bằng cách duy trì hidden state qua từng

timestep. Tuy nhiên RNN thuần gặp vanishing gradient khi chuỗi dài — gradient lan truyền ngược bị suy giảm, khó học phụ thuộc xa. Long Short-Term Memory (LSTM) giải quyết bằng cơ chế gate: input gate, forget gate, output gate kiểm soát luồng thông tin.

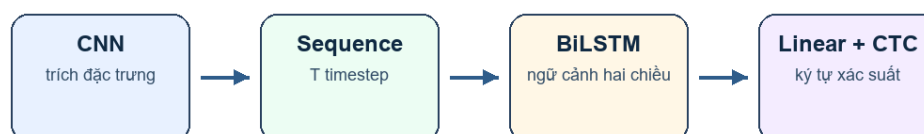
Trong OCR, chuỗi theo chiều ngang của word crop thường không quá dài (vài chục timestep), nên LSTM đủ hiệu quả. Ký tự hiện tại phụ thuộc ngữ cảnh hai phía: ví dụ 'l' đứng cạnh 'i' hay '1' cho confidence khác nhau. Bidirectional LSTM (BiLSTM) chạy LSTM thuận và nghịch, concatenate hidden state — chuẩn trong CRNN recognizer.

2.4.2 Kiến trúc CRNN

CRNN (Convolutional Recurrent Neural Network) do Shi, Bai và Yao đề xuất [1] cho scene text recognition. Ba thành phần: (1) CNN trích đặc trưng, collapse height thành sequence; (2) BiLSTM mô hình hóa phụ thuộc chuỗi; (3) Linear + CTC transcription. CRNN không cần cắt ảnh thành từng ký tự — phù hợp word crop có nhiều ký tự liên tiếp.

OCRLite dùng CRNNRecognizer (model_name = crnn), export recognizer_crnn.onnx. Codebase còn chuẩn bị CRNNTransformerRecognizer, MobileSVTRRecognizer, STNRecognizer qua model factory — hướng mở rộng sau này. Bản nộp chọn CRNN vì ổn định qua ONNX export và đáp ứng metric validation (best_val_cer $\approx$ 0,0315 sau fine-tune).

Kiến trúc CRNN: CNN $\rightarrow$ sequence feature $\rightarrow$ BiLSTM $\rightarrow$ Linear $\rightarrow$ CTC.



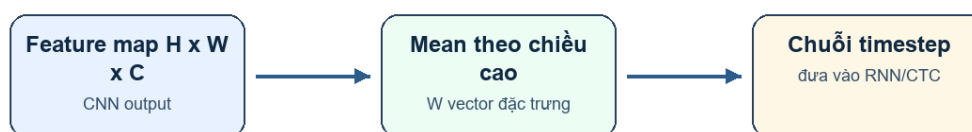
Ý nghĩa trong OCRLite: mô hình không cần biết vị trí chính xác từng ký tự, nhưng vẫn học được chuỗi ký tự theo thứ tự đọc từ trái sang phải.

Hình 2.4. Kiến trúc CRNN: CNN $\rightarrow$ sequence feature $\rightarrow$ BiLSTM $\rightarrow$ Linear $\rightarrow$ CTC.

Khi triển khai CRNN cho ảnh word crop, một chi tiết dễ bị bỏ qua là cách chuyển feature map hai chiều thành chuỗi một chiều. Sau các tầng CNN, chiều cao của feature map được nén dần, còn chiều rộng giữ vai trò trực thời gian. Mỗi cột feature tương ứng với một vùng hẹp trên ảnh gốc và trở thành một timestep đưa vào BiLSTM. Nếu pooling theo chiều rộng quá mạnh, số timestep giảm và CTC khó căn chỉnh label dài; nếu giữ quá nhiều timestep, inference chậm hơn và có thể sinh nhiều blank dư. Vì vậy cấu hình kernel, stride và input width trong OCR Lite được xem là một phần của thiết kế recognizer, không chỉ là tham số phụ.

Trong thực tế, recognizer không đọc ký tự trên ảnh gốc full-page mà đọc các crop do detector và layout module tạo ra. Điều này làm chất lượng CRNN phụ thuộc mạnh vào độ sạch của crop: crop quá sát làm mất dấu câu hoặc nét biên; crop quá rộng đưa thêm nền, ký tự lân cận hoặc khoảng trắng thừa. Vì vậy, lý thuyết CRNN cần được hiểu cùng với lý thuyết layout. Một recognizer có CER tốt trên MJSynth vẫn có thể đọc kém nếu crop runtime khác phân phối dữ liệu train. Đây là lý do đề án có bước fine-tune bằng document crop và lưu ảnh debug để kiểm tra lỗi theo từng tầng.

Quá trình chuyển feature map hai chiều của CRNN thành chuỗi timestep theo chiều



Hình 2.5. Quá trình chuyển feature map hai chiều của CRNN thành chuỗi timestep theo chiều ngang.

2.4.3 Linear classifier và số timestep

Sau BiLSTM, Linear layer chiếu hidden state h_t lên logits $z_t \in \mathbb{R}^C$ với $C = |\text{charset}|$

+ 1 (bao gồm blank CTC). Softmax cho $P(c|t)$. Output shape $T \times \text{batch} \times C$. Chiều rộng ảnh input ảnh hưởng T : config OCRLite `image_height=32, image_width 160/192/256`. `preserve_aspect_ratio + pad` nền trắng tránh méo chữ. T phải đủ lớn để CTC align label dài; width quá nhỏ có thể cắt hoặc decode sai từ dài.

2.4.4 Attention-based recognizer (tham khảo)

Ngoài CTC, một hướng phổ biến là attention-based encoder–decoder: decoder sinh ký tự từng bước, attention weight lên encoder feature. **Ưu điểm:** có thể mô hình hóa ngữ cảnh **phức tạp**. **Nhược điểm:** inference autoregressive chậm hơn greedy CTC; export ONNX đôi khi phức tạp hơn. OCRLite giữ CTC cho runtime vì cân bằng tốc độ và độ ổn định triển khai — phù hợp mục tiêu OCR nhẹ tích hợp web.

Trong quá trình khảo sát tài liệu và thử nghiệm nhỏ, CRNN cũng được so sánh với hướng CNN thuần (bỏ BiLSTM) trên một số sample nhỏ. Kết quả quan sát được cho thấy BiLSTM cải thiện các trường hợp ký tự mảnh và dấu câu, dù không loại bỏ hoàn toàn lỗi. Điều này phù hợp với lý thuyết: OCR không chỉ là phân loại ảnh tĩnh mà là bài toán chuỗi có phụ thuộc theo chiều dọc.

Về mặt triển khai, BiLSTM làm tăng thời gian inference so với CNN-only, nhưng trong benchmark đồ án OCRLite vẫn nhanh hơn EasyOCR và RapidOCR trên cùng bộ ảnh thử. Như vậy trade-off giữa độ phức tạp recognizer và tốc độ tổng thể còn phụ thuộc detector và số lượng crop — không thể kết luận chỉ dựa trên kiến trúc recognizer.

2.5 Connectionist Temporal Classification (CTC)

2.5.1 Bài toán alignment trong sequence labeling

Khi huấn luyện recognizer, dữ liệu gồm cặp (ảnh crop, chuỗi label) nhưng không biết vị trí chính xác từng ký tự trên trục thời gian (horizontal axis sau CNN pooling). Gán nhãn frame-by-frame tốn kém và dễ sai. Connectionist Temporal Classification (CTC), Graves et al. [2], cho phép train chỉ với chuỗi label mà không cần alignment chi tiết.

CTC mở rộng alphabet bằng ký tự blank (ϵ). Model output phân phối xác suất trên extended alphabet tại mỗi timestep. Tập tất cả chuỗi extended có thể collapse về label đúng

được gom lại; loss là negative log-likelihood tổng trên các alignment hợp lệ, tính bằng forward–backward algorithm (tương tự HMM).

2.5.2 Blank label và quy tắc collapse

Trong OCRLite, blank index = 0; ký tự thật bắt đầu từ 1 (class Charset). Khi decode: bỏ timestep predict blank; gộp ký tự lặp liên tiếp. Ví dụ extended t-t-ε-h-y-l → thy. Blank cho phép model “chờ” hoặc “kéo dài” tại vị trí mà không emit ký tự mới — một cách để xử lý alignment không đồng đều giữa timestep và ký tự.

Config text_case = lower: charset không phân biệt hoa thường, giảm số class, đơn giản hóa train. Trade-off: output mất format viết hoa gốc. Đây là quyết định thiết kế có chủ đích trong phạm vi đọc nội dung tiếng Anh.

2.5.3 CTCLoss, greedy decode và beam search

PyTorch nn.CTCLoss nhận log_probs (T×N×C), targets, input_lengths, target_lengths. Training loop OCRLite: logits → log_softmax → CTCLoss. Inference: greedy decode — argmax mỗi timestep rồi collapse. Greedy nhanh, phù hợp ONNX runtime. Beam search có thể cải thiện trường hợp ambiguous nhưng tăng latency; code có prefix beam search optional trong decoder.py nhưng runtime final dùng greedy.

$$\text{CTCLoss} = -\log P(\text{label} \mid \text{input sequence})$$

$$\text{CER} = \text{Levenshtein}(\text{prediction}, \text{target}) / |\text{target}|$$

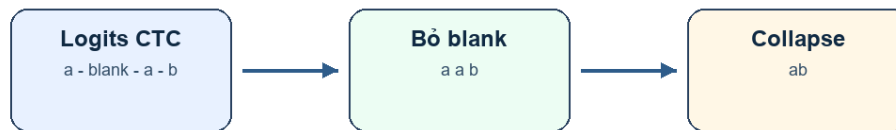
CTC là lý do manifest recognizer chỉ cần image_path và text — không cần bounding box từng ký tự. Phù hợp MJSynth, IIIT5K và document crop fine-tune. Giảm chi phí gán nhãn so với hướng yêu cầu alignment tường minh.

Để hiểu sâu hơn, có thể hình dung mỗi timestep output một phân phối xác suất trên alphabet mở rộng. Forward–backward algorithm tính xác suất tổng của mọi chuỗi extended hợp lệ có thể collapse về label đúng. Khi train, mạng được điều chỉnh để tăng xác suất này; khi inference, chỉ cần decode một chuỗi extended cụ thể (greedy hoặc beam) rồi collapse.

Một hạn chế lý thuyết của greedy CTC decode là có thể chọn locally optimal nhưng globally suboptimal — đặc biệt với dấu câu và ký tự mảnh. Trong thực nghiệm đồ án, lỗi dạng thylacine's → thylacinels cho thấy vấn đề không chỉ nằm ở CTC mà còn ở crop và

tách word. Tuy nhiên CTC vẫn là lựa chọn hợp lý vì cân bằng đơn giản triển khai và chất lượng trên validation.

Minh họa CTC: alignment, blank và collapse rule.



Ý nghĩa trong OCRLite: mô hình không cần biết vị trí chính xác từng ký tự, nhưng vẫn học được chuỗi ký tự theo thứ tự đọc từ trái sang phải.

Hình 2.6. Minh họa CTC: alignment, blank và collapse rule.

2.6 Phát hiện vùng chữ bằng Semantic Segmentation

2.6.1 Semantic segmentation và ứng dụng OCR

Semantic segmentation gán class cho từng pixel (chữ, nền, ...). Trong OCR document, detector thường chỉ cần mask nhị phân: pixel thuộc vùng chữ hay không. So với object detection (bounding box trực tiếp), segmentation pixel-wise phù hợp khi dữ liệu synthetic có thể sinh mask tự động từ tọa độ paste word crop — không cần vẽ box thủ công.

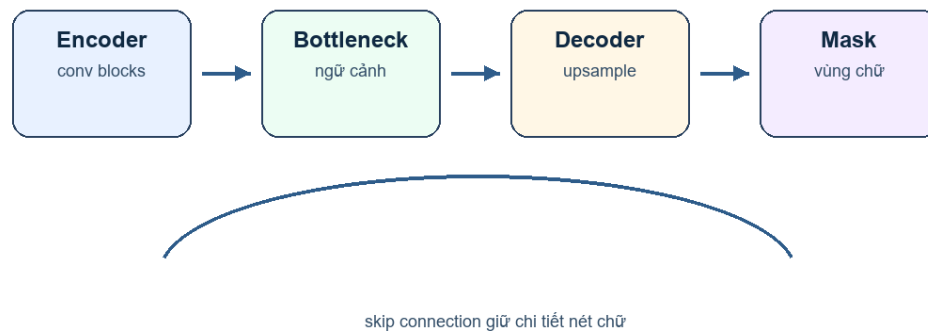
2.6.2 U-Net và FastTextMaskUNet

U-Net [3] là kiến trúc encoder–decoder: encoder co cỡ feature map qua conv+pooling; decoder upsample (transpose conv hoặc interpolate) về kích thước gốc; skip connection nối feature cùng resolution từ encoder sang decoder. U-Net phổ biến trong y học và segmentation; áp dụng OCR document là hướng tự nhiên.

OCRLite dùng FastTextMaskUNet — biến thể nhẹ với separable convolution, base_channels=16 (model_variant=fast). Detector chạy trên toàn trang (resize ví dụ 384×512) nên cần nhẹ — detector nặng làm chậm pipeline trước khi recognizer được gọi. Input: 1×H×W grayscale; output: logits mask cùng kích thước.

Sau inference: sigmoid $\rightarrow$ probability map $\rightarrow$ threshold $\rightarrow$ binary mask $\rightarrow$ morphology close $\rightarrow$ findContours $\rightarrow$ bounding boxes $\rightarrow$ scale về tọa độ ảnh gốc. Detector không output text — chỉ geometry cho layout và recognizer. Điểm yếu: chất lượng phụ thuộc threshold và post-processing — IoU cao trên synthetic chưa đảm bảo box hoàn hảo trên ảnh thật.

Kiến trúc U-Net / FastTextMaskUNet với skip connection.



Hình 2.7. Kiến trúc U-Net / FastTextMaskUNet với skip connection.

2.6.3 BCE, Dice loss và IoU

Loss detector = BCE with logits + Dice loss. BCE penalize từng pixel. Dice loss tối ưu overlap, nhạy với class imbalance (nền trắng chiếm đa số pixel):

$$\text{Dice loss} = 1 - (2|X \cap Y| + \epsilon) / (|X| + |Y| + \epsilon)$$

IoU (Jaccard index) đo overlap mask dự đoán và ground truth:

$$\text{IoU} = |A \cap B| / |A \cup B|$$

Detector OCRLite chọn checkpoint theo val_iou cao nhất ($\approx 0,975$ trên synthetic validation). Metric này phản ánh chất lượng học mask; post-processing layout vẫn cần kiểm tra bằng ảnh debug detector_boxes.png.

2.6.4 Object detection vs segmentation (so sánh ngắn)

Object detection (EAST, DBNet, YOLO-style) predict box trực tiếp — phù hợp scene text. Segmentation predict mask — phù hợp synthetic document có mask sẵn. OCRLite

chọn segmentation vì pipeline sinh mask từ paste word crop thuận tiện trong phạm vi đồ án; object detection đòi hỏi annotation box phức tạp hơn với cùng nguồn dữ liệu.

Trong giai đoạn khảo sát, DBNet/EAST-style detector cũng được xem xét vì các paper scene text thường dùng hướng này. Tuy nhiên với synthetic document đã có mask, segmentation U-Net cho pipeline train rõ ràng hơn và dễ sinh ground truth tự động. Quyết định cuối cùng ưu tiên khả năng hoàn thiện trong thời gian đồ án hơn là theo đuổi kiến trúc detector mới nhất chưa có dữ liệu phù hợp.

Dice loss và BCE bổ sung cho nhau: chỉ dùng BCE đôi khi model học nền trắng tốt nhưng vùng chữ mỏng bị đứt; chỉ dùng Dice có thể unstable khi mask rất nhỏ. Kết hợp hai loss là thực hành phổ biến trong segmentation OCR/document. Metric IoU trên validation synthetic đạt cao ($\sim 0,975$) nhưng vẫn cần kiểm tra ảnh debug trên screenshot thật — số liệu validation không thay thế quan sát trực tiếp.

2.7 Xử lý ảnh, tiền xử lý và augmentation

2.7.1 Grayscale, resize và normalize

Tiền xử lý đồng bộ giữa train và inference là điều kiện cần — domain shift âm thầm nếu train và runtime khác cách normalize. OCRLite: đọc ảnh $\rightarrow$ grayscale $\rightarrow$ resize/pad theo config $\rightarrow$ normalize (scale $[0,1]$ hoặc zero-mean). Recognizer: height cố định (32px), width biến đổi, pad nền trắng khi preserve_aspect_ratio. Detector: resize page về kích thước train cố định; scale box ngược về ảnh gốc sau inference.

Một sai lầm thường gặp khi triển khai OCR lần đầu là train với một kiểu normalize nhưng runtime lại normalize khác (ví dụ train zero-mean nhưng runtime chỉ chia 255). Lỗi này có thể không làm model hoàn toàn “chết” nhưng làm metric thực tế suy giảm mà khó phát hiện nếu không so sánh tensor trước khi đưa vào ONNX. OCRLite giữ pipeline preprocess thống nhất giữa train.py, export script và engine.py.

2.7.2 Nhị phân hóa Otsu và morphology

Otsu threshold tự động chọn ngưỡng nhị phân hóa tối ưu hóa variance giữa hai lớp pixel — dùng trong layout post-processing trên vùng crop. Morphology (closing) nối vùng

chữ gần nhau trên binary mask detector. OpenCV cung cấp các phép toán này [8][9]. Đây không phải deep learning nhưng ảnh hưởng trực tiếp chất lượng crop đưa vào recognizer — một nguồn lỗi thường bị bỏ qua khi chỉ nhìn metric CER.

Morphology closing với kernel hình chữ nhật có thể nối các nét chữ gần nhau trên mask detector, giảm tình trạng một từ bị tách thành nhiều contour nhỏ. Tuy nhiên kernel quá lớn có thể dính hai từ lân cận — đây là tham số cần tinh chỉnh bằng ảnh debug thay vì chỉ dựa vào val_iou. Post-processing được xem là phần “lý thuyết thực hành” không kém phần quan trọng so với mô hình deep learning.

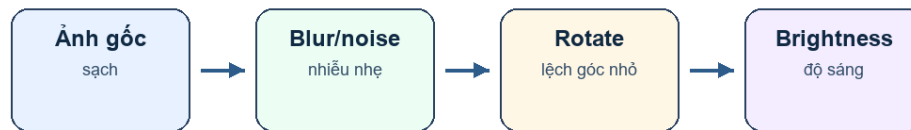
2.7.3 Data augmentation

Augmentation tăng đa dạng train: blur, Gaussian noise, rotate nhẹ, brightness/contrast jitter. Recognizer: giữ biên độ vừa phải để không phá label. Detector: blur nhẹ, nền biến thiên — giảm overfit trang trắng synthetic hoàn hảo. Augmentation không thay thế dữ liệu thật nhưng là biện pháp thực dụng khi gán nhãn hạn chế.

Với OCR, augmentation cần thận trọng hơn một số bài toán phân loại ảnh thông thường. Nếu xoay quá mạnh, kéo giãn quá nhiều hoặc thêm nhiễu quá dày, label chữ vẫn giữ nguyên nhưng ảnh đầu vào không còn giống trường hợp người dùng thật sẽ đưa vào hệ thống. Khi đó mô hình học khả năng chống nhiễu cực đoan nhưng lại giảm độ sắc nét cho các ký tự nhỏ. Vì vậy OCRLite ưu tiên augmentation mức nhẹ, mô phỏng lỗi thường gặp của ảnh screenshot như nén JPEG, blur nhẹ, thay đổi độ sáng và lệch góc nhỏ. Mục tiêu không phải làm dữ liệu càng khó càng tốt, mà là làm dữ liệu đủ đa dạng nhưng vẫn cùng miền với ảnh tài liệu tiếng Anh trong phạm vi đồ án.

Một nguyên tắc khác là augmentation phải nhất quán với loại nhãn. Với recognizer, nhãn là chuỗi text nên các phép biến đổi không được làm mất hoặc đổi thứ tự ký tự. Với detector segmentation, phép **biến đổi hình học phải áp dụng đồng thời lên ảnh và mask; nếu chỉ xoay ảnh mà không xoay mask, model sẽ học từ cặp dữ liệu sai**. Trong source OCRLite, phần build dữ liệu và train tách rõ transform cho recognizer và detector để tránh lẫn hai loại bài toán này. Đây là điểm quan trọng khi mở rộng dataset sau đồ án.

Ví dụ augmentation mức nhẹ cho OCR: blur, noise, brightness/contrast và xoay góc



Hình 2.8. Ví dụ augmentation mức nhẹ cho OCR: blur, noise, brightness/contrast và xoay góc nhỏ.

2.7.4 Charset, manifest và label encoding

Charset định nghĩa tập ký tự recognizer có thể predict. Class Charset: normalize Unicode NFC, map ký tự $\leftrightarrow$ index, filter label ngoài charset. Manifest CSV (image_path, text) cho recognizer; JSONL (image_path, mask_path, boxes) cho detector. Lớp trừu tượng thống nhất nguồn MJSynth, TRDG, synthetic document, IIIT5K.

Quy trình tiền xử lý: grayscale, resize/pad, normalize, augmentation.



Hình 2.9. Quy trình tiền xử lý: grayscale, resize/pad, normalize, augmentation.

2.8 Dữ liệu huấn luyện OCR phổ biến

2.8.1 MJSynth và synthetic word crop

MJSynth (MJST+) là tập word crop synthetic tiếng Anh quy mô lớn, phát triển bởi VGG [6]. Dữ liệu có thể export từ Hugging Face [7]. MJSynth phù hợp train base recognizer vì số lượng sample lớn, label sạch. OCRLite dùng MJSynth làm nguồn chính cho recognizer baseline.

2.8.2 IIIT5K và benchmark

IIIT5K là tập benchmark word recognition trên ảnh scene text [6]. Dùng để test và so sánh recognizer, không nhất thiết train toàn bộ trên IIIT5K trong đồ án. Metric CER/WER trên IIIT5K là chuẩn cộng đồng để đối chiếu với paper CRNN gốc.

2.8.3 Synthetic document cho detector

OCRLite sinh synthetic document bằng cách paste word crops lên trang trắng, ghi tọa độ paste → sinh mask tự động. Script `build_document_synth.py` tạo JSONL manifest. Cách làm tiết kiệm thời gian gán nhãn mask thủ công — phù hợp phạm vi đồ án. Trade-off: domain gap với ảnh screenshot thật, cần fine-tune detector trên dữ liệu thật nếu phát triển tiếp.

Fine-tune recognizer trên document crop là bước **quan trọng để thu hẹp khoảng cách giữa** word crop MJSynth và crop runtime từ detector. Về mặt lý thuyết dữ liệu, đây là dạng domain adaptation nhẹ: giữ phần lớn kiến thức base model, bổ sung sample gần phân phối inference. Learning rate fine-tune nhỏ hơn base train để tránh phá weight đã học — nguyên tắc transfer learning chuẩn trong deep learning.

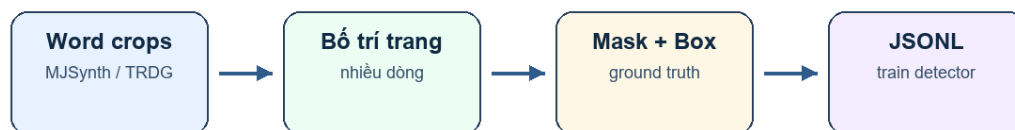
IIIT5K và các benchmark khác giúp đặt kết quả recognizer vào ngữ cảnh cộng đồng, dù đồ án không claim SOTA trên các benchmark lớn. Mục tiêu chính vẫn là pipeline full-page chạy được trong package nộp, không phải leaderboard đơn lẻ trên word recognition.

Đối với detector, dữ liệu synthetic document có ý nghĩa đặc biệt vì mask pixel-level rất tốn công gán nhãn thủ công. Khi word crop được paste lên nền trang trắng, hệ thống biết chính xác vị trí từng word nên có thể sinh mask và box tự động. Cách làm này tạo được nhiều trang với font, độ dài dòng và khoảng cách khác nhau trong thời gian ngắn. Nhược

điểm là synthetic document thường sạch hơn ảnh thật: nền đều hơn, ít nhiễu camera, ít méo perspective và ít thành phần trang phức tạp. Vì vậy synthetic data được xem là bước khởi tạo nền tảng, còn benchmark ảnh thật ở Chương 4 mới phản ánh rõ khả năng ứng dụng cuối cùng.

Một pipeline dữ liệu OCR tốt cần lưu đủ thông tin để truy vết lỗi. Ngoài đường dẫn ảnh và label, manifest nên giữ metadata như nguồn dữ liệu, kích thước ảnh, loại split (train/val/test), config augmentation và thời điểm sinh dữ liệu. Khi một checkpoint cho CER tốt nhưng end-to-end kém, có thể truy vết model đã train trên kiểu crop nào, detector đã sinh mask theo ngưỡng nào, và validation set có đại diện cho ảnh runtime hay không. Đây là lý do OCRLite tổ chức dữ liệu theo manifest và result folder thay vì chỉ lưu rời rạc các file ảnh.

Quy trình sinh synthetic document: word crop → bố trí lên trang → mask/box → m



Hình 2.10. Quy trình sinh synthetic document: word crop → bố trí lên trang → mask/box → manifest.

2.9 ONNX và ONNX Runtime

2.9.1 Khái niệm ONNX

Open Neural Network Exchange (ONNX) là định dạng mô hình trung lập, cho phép chuyển model giữa framework (PyTorch, TensorFlow, ...) và runtime inference [4]. ONNX graph gồm các operator chuẩn (Conv, LSTM, MatMul, ...) và trọng số. Mục tiêu: train một

nơi, deploy nhiều nơi với runtime tối ưu.

2.9.2 ONNX Runtime và lý do OCRLite export ONNX

ONNX Runtime (ORT) thực thi ONNX graph trên CPU, CUDA, TensorRT provider [4][11]. ORT tối ưu graph fusion, memory planning — thường nhanh hơn PyTorch eager inference trên CPU. OCRLite export ONNX vì: (1) runtime không phụ thuộc PyTorch train stack; (2) package nộp chỉ cần .onnx + config JSON; (3) dễ tích hợp Django/FastAPI; (4) giảm footprint khi deploy.

Quy trình export: load checkpoint → build model → eval → dummy input → torch.onnx.export → verify ORT output so với PyTorch. Dynamic axes cho batch và width linh hoạt. Config runtime (recognizer_config.json, document_detector_config.json) bắt buộc vì ONNX không chứa charset, input size, threshold — runtime phải đọc config.

2.9.3 TensorRT và quantization (tham khảo)

TensorRT (scripts/export_tensorrt.py) tối ưu GPU NVIDIA — FP16, kernel fusion. Phụ thuộc driver và hardware, không phải mặc định đồ án. Quantization INT8 giảm RAM và tăng tốc CPU yếu nhưng cần calibration set. Tối ưu tốc độ nên làm sau khi layout và chất lượng text ổn — đọc sai nhanh hơn không có giá trị thực tế.

Khi export ONNX, một số operator LSTM hoặc dynamic shape cần kiểm tra tương thích với opset version. Một số lỗi export thông thường có thể xuất hiện như mismatch input name hoặc dynamic axis chưa khai báo đúng — giải quyết bằng verify script so sánh output PyTorch và ORT trên cùng dummy input. Bước verify này tuy đơn giản nhưng tránh được tình trạng package nộp chạy được nhưng output sai im lặng.

Runtime ONNX Runtime chọn Execution Provider theo môi trường: CUDAXecutionProvider nếu có GPU, ngược lại CPUExecutionProvider. Điều này quan trọng khi demo trên máy demo không có GPU — hệ thống vẫn phải chạy được, dù chậm hơn môi trường train.

Luồng train PyTorch → export ONNX → inference ONNX Runtime.



Hình 2.11. Luồng train PyTorch → export ONNX → inference ONNX Runtime.

2.10 Các metric đánh giá hệ thống OCR

2.10.1 Character Error Rate (CER)

$CER = \text{Levenshtein}(\text{prediction}, \text{target}) / |\text{target}|$. Đo tỷ lệ ký tự sai. CER=0 là khớp hoàn toàn. Nhạy với dấu câu, hyphen, apostrophe. OCRLite base $\approx 0,0403$; fine-tune $\approx 0,0315$ trên validation recognizer.

2.10.2 Word Error Rate (WER) và exact match

WER áp dụng Levenshtein ở mức từ. Một ký tự sai có thể làm hỏng cả từ → WER nặng hơn CER khi word crop đọc sai hoàn toàn. Exact match: tỷ lệ sample prediction trùng hoàn toàn label — metric nghiêm, dễ so checkpoint.

2.10.3 IoU và metric phân tầng

IoU dùng cho detector segmentation. Gom CER recognizer và IoU detector thành một accuracy tổng là sai lầm phương pháp — lỗi full-page có thể đến từ layout giữa detector và recognizer. OCRLite đánh giá phân tầng; end-to-end bằng ảnh thật + benchmark thời gian/RAM.

Levenshtein distance (edit distance) đo số thao tác insert/delete/substitute tối thiểu để biến chuỗi A thành B. CER và WER đều dựa trên đó nhưng đơn vị khác (ký tự vs từ). Trong

báo cáo, CER được ưu tiên cho recognizer vì nhạy với lỗi dấu câu; WER bổ sung khi cần đánh giá mức độ “hông cả từ”.

Ngoài metric số, phân tích char_confusion.csv giúp phát hiện systematic error — ví dụ nhầm $l \leftrightarrow i$ hoặc $o \leftrightarrow 0$ — từ đó quyết định bổ sung augmentation hay dữ liệu fine-tune. Cách phân tích này gắn lý thuyết metric với hành động cải thiện model, không dừng ở việc report một con số CER duy nhất.

2.10.4 Benchmark hiệu năng như metric bổ sung

Ngoài metric chất lượng văn bản, đồ án dùng thời gian xử lý trung bình và peak RAM làm metric phi chức năng. Lý do: mục tiêu thiết kế OCRLite bao gồm OCR nhẹ, tích hợp web — một engine đọc đúng nhưng chậm hoặc tốn >1 GB RAM khó deploy trên backend quy mô vừa. Benchmark so sánh OCRLite, RapidOCR, EasyOCR trên cùng 4 ảnh (Chương 4) bổ sung góc nhìn lý thuyết–thực tiễn: không chỉ CER mà còn trade-off tài nguyên.

Cần phân biệt rõ: benchmark thời gian/RAM không thay thế CER/WER khi đánh giá chất lượng đọc chữ. Hai nhóm metric trả lời hai câu hỏi khác nhau — “đọc nhanh và nhẹ đến mức nào?” và “đọc đúng đến mức nào?”. Báo cáo đồ án nghiêm túc cần trình bày cả hai, tránh chỉ nhấn mạnh một phía.

Bảng 2.2. Tóm tắt metric theo thành phần OCRLite.

Metric / artifact	Thành phần đánh giá	Ý nghĩa khi đọc kết quả
CER, WER, exact match	Text Recognizer (CRNN + CTC)	Đo chất lượng đọc ký tự/từ trên tập có ground truth.
IoU, BCE + Dice loss	Document Detector (FastTextMaskUNet)	Đo overlap mask vùng chữ và chất lượng học segmentation.
Thời gian/ảnh, peak RAM	Runtime benchmark	Đánh giá khả năng tích hợp vào web/backend.
predictions.csv, error_samples.csv, char_confusion.csv	Phân tích lỗi	Truy vết ký tự/từ hay sai để quyết định fine-tune hoặc sửa layout.

2.11 Công nghệ triển khai liên quan

2.11.1 PyTorch

PyTorch [5]: nn.Module, DataLoader, AdamW, scheduler, AMP trên Kaggle T4.

Early stopping theo CER/val_iou. Checkpoint .pth lưu state_dict và metadata config. Training side tách biệt runtime side — nguyên tắc OCRLite.

Automatic Mixed Precision (AMP) cho phép một số phép toán chạy FP16 trên GPU, giảm VRAM và tăng tốc train — hữu ích trên Kaggle T4 giới hạn thời gian session. Early stopping theo CER hoặc val_iou tránh overfit và tiết kiệm epoch dư thừa. Checkpoint best được chọn theo metric validation, không phải train loss thấp nhất — thực hành chuẩn khi train deep learning cho OCR.

2.11.2 OpenCV

OpenCV: đọc ảnh, grayscale, resize, morphology, findContours, Otsu, vẽ debug box. Layout trong document/layout.py và engine.py phụ thuộc OpenCV — quyết định chất lượng crop dù không phải deep learning.

Có thể coi OpenCV trong OCRLite như lớp geometry engine: xử lý contour, scale tọa độ, tách dòng/từ bằng histogram. Lớp này không học từ gradient descent nhưng tham số threshold, dilation, min_box_area ảnh hưởng trực tiếp output cuối. Khi phân tích lỗi, layout thường được kiểm tra trước khi nghi ngờ recognizer — kinh nghiệm từ nhiều lần chạy thử trên ảnh screenshot bài đọc.

2.11.3 FastAPI và tích hợp web

FastAPI [10]: REST API OCR crop/line, DynamicBatcher gom request, Laplacian blur check, entropy/confidence reject. Full-page runtime ở OCR Engine — khởi tạo engine một lần khi app start. Engnite (Django + React) có thể gọi picture_to_text hoặc wrap FastAPI phía sau Nginx.

2.11.4 Config-driven experiment

Mọi thí nghiệm mô tả bằng JSON trong configs/. Sau mỗi run copy config vào result/reports/. Tránh hard-code hyperparameter — hỗ trợ tái lập và so sánh fair giữa các lần train trên Kaggle.

DynamicBatcher trong FastAPI gom nhiều request OCR crop trong cửa sổ thời gian ngắn thành một batch — tăng throughput khi recognizer chạy batch hiệu quả hơn từng ảnh.

Đây là kỹ thuật phổ biến khi serve model ONNX trong production, dù đồ án mới triển khai ở mức service crop/line chứ chưa full-page API hoàn chỉnh.

Tích hợp Engnate web (Django + React) có thể theo hai pattern: (1) gọi trực tiếp picture_to_text trong Django view/celery task; (2) deploy OCRLite như microservice FastAPI riêng phía sau Nginx reverse proxy. Pattern (1) đơn giản hơn cho demo; pattern (2) scale tốt hơn khi tải OCR tăng.

2.13 Bổ sung cơ sở lý thuyết phục vụ thiết kế OCRLite

Các mục trước đã trình bày những khái niệm nền tảng của OCR, từ nhận dạng ký tự, mạng CNN, CRNN, CTC cho đến detector dạng segmentation và ONNX Runtime. Tuy nhiên, khi chuyển các khái niệm đó thành một hệ thống có thể chạy được trong đồ án, cần làm rõ thêm mối quan hệ giữa lý thuyết và quyết định thiết kế. Phần bổ sung này không giới thiệu một hướng nghiên cứu mới, mà đóng vai trò nối phân cơ sở lý thuyết với Chương 3. Nhờ vậy, người đọc có thể hiểu vì sao OCRLite không chọn một thư viện OCR có sẵn làm lõi duy nhất, vì sao phải tách detector và recognizer, vì sao dùng dữ liệu synthetic, và vì sao metric của hệ thống phải được đọc theo từng tầng thay vì gom thành một con số chung.

2.13.1 Quan hệ giữa dữ liệu, mô hình và lỗi runtime

Trong OCR, chất lượng đầu ra không chỉ phụ thuộc vào độ mạnh của mô hình. Một recognizer có CER thấp trên validation word crop vẫn có thể đọc sai khi crop runtime bị cắt mất dấu câu, bị thiếu mép chữ hoặc chứa nhiều ký tự dính nhau. Ngược lại, một detector có IoU cao trên trang synthetic vẫn có thể tạo box chưa tối ưu trên ảnh chụp thật nếu font chữ, khoảng cách dòng, nhiễu nền hoặc độ phân giải khác xa dữ liệu huấn luyện. Do đó, lý thuyết OCR hiện đại thường nhấn mạnh quan điểm hệ thống: dữ liệu quyết định phân phối đầu vào, mô hình học ánh xạ trên phân phối đó, còn runtime post-processing quyết định mẫu đầu vào thực sự được đưa vào recognizer.

Đối với OCRLite, điểm quan trọng nhất là chuỗi lỗi có tính dây chuyền. Nếu detector bỏ sót một dòng, recognizer không có cơ hội đọc dòng đó. Nếu layout tách từ quá nhỏ, các ký tự như dấu nháy đơn, dấu gạch nối hoặc dấu chấm có thể bị loại như nhiều. Nếu

recognizer dùng CTC greedy decode, một số chuỗi có xác suất gần nhau có thể bị chọn sai. Nếu bước ghép đoạn chỉ dựa vào khoảng cách dòng, tài liệu nhiều cột hoặc có bảng có thể bị đảo thứ tự đọc. Vì vậy, báo cáo không nên khẳng định hệ thống chính xác chỉ bằng một metric validation; cần trình bày các metric theo tầng và minh họa lỗi end-to-end trên ảnh thật.

Cách hiểu này cũng giải thích vì sao đồ án sử dụng nhiều artifact như predictions.csv, error_samples.csv, char_confusion.csv, ảnh debug box và benchmark runtime. Những artifact này không chỉ là kết quả phụ, mà là công cụ để xác định tầng nào đang gây lỗi. Khi một từ bị đọc sai, cần kiểm tra box trước, sau đó mới kiểm tra recognizer. Nếu box đúng nhưng text sai, lỗi nằm ở mô hình hoặc decoder. Nếu box sai, việc train recognizer thêm thường không giải quyết đúng nguyên nhân.

Bảng 2.3. Liên hệ giữa loại lỗi OCR và hướng kiểm tra trong OCRLite.

Loại lỗi	Dấu hiệu thường gặp	Tầng cần kiểm tra	Artifact liên quan
Bỏ sót chữ	Thiếu cả dòng hoặc cụm từ	Document Detector	mask, detector_boxes
Sai dấu câu	mất apostrophe, hyphen, dấu chấm	Layout + Recognizer	word boxes, char_confusion
Dính từ	hai từ bị ghép hoặc tách sai	Layout post-processing	boxes_to_lines, segment_words
Sai ký tự	l/i/1, o/0, s/5	Recognizer + decoder	predictions.csv, error_samples.csv
Chậm hoặc tốn RAM	latency cao, peak RAM lớn	Runtime deployment	benchmark output

2.13.2 Vì sao dữ liệu synthetic vẫn có giá trị

Một hạn chế thường gặp của đồ án OCR là thiếu dữ liệu gán nhãn thật. Để huấn luyện detector full-page, cần ảnh tài liệu và mask vùng chữ tương ứng. Nếu gán nhãn thủ công từng dòng hoặc từng từ, chi phí thời gian rất lớn. Synthetic document giải quyết bài toán này bằng cách lấy các word crop đã có nhãn, bố trí chúng lên trang trắng theo nhiều dòng, đồng thời sinh mask và box từ chính tọa độ paste. Cách làm này không thay thế hoàn toàn dữ liệu thật, nhưng tạo được một tập huấn luyện lớn, nhất quán và có nhãn chính xác ở cấp pixel.

Giá trị của synthetic data nằm ở việc giúp mô hình học quy luật hình học cơ bản: vùng có chữ thường tạo thành cụm nét tối trên nền sáng, nhiều từ tạo thành dòng, nhiều dòng tạo thành vùng văn bản. Với detector segmentation, điều này đặc biệt hữu ích vì ground truth mask có thể sinh tự động. Đối với recognizer, synthetic word crop như MJSynth hoặc TRDG giúp mô hình học hình dạng chữ cái, số và dấu câu trong nhiều font khác nhau. Khi kết hợp với augmentation nhẹ như blur, noise, brightness và xoay nhỏ, mô hình trở nên bền hơn trước các biến thiên thường gặp ở ảnh chụp.

Tuy nhiên, synthetic data luôn có domain gap. Trang synthetic thường sạch hơn ảnh thật, nên ít nhiễu hơn, chữ ít méo hơn và bố cục đơn giản hơn. Vì vậy, phân kết quả phải trình bày rõ: IoU cao trên validation synthetic chứng minh detector học tốt trên phân phối đã tạo, nhưng chưa đủ để kết luận chạy hoàn hảo trên mọi ảnh thật. Hướng phát triển hợp lý là dùng synthetic data làm nền, sau đó bổ sung một tập ảnh thật nhỏ nhưng đại diện để fine-tune hoặc hiệu chỉnh threshold/layout. Cách trình bày này vừa trung thực, vừa cho thấy bản chất của dữ liệu huấn luyện.

2.13.3 Vai trò của decoder và hậu xử lý ngôn ngữ

CTC chỉ cung cấp phân phối xác suất theo timestep, còn chuỗi văn bản cuối cùng phụ thuộc vào decoder. Greedy decode chọn ký tự xác suất cao nhất tại từng timestep rồi áp dụng quy tắc collapse. Cách này nhanh, đơn giản và phù hợp với mục tiêu OCRLite là runtime nhẹ. Nhưng greedy decode có nhược điểm: nó không xét đầy đủ các chuỗi gần tối ưu, không dùng ngữ cảnh từ vựng và dễ sai ở các ký tự nhỏ hoặc hiếm như apostrophe, hyphen, dấu chấm than. Beam search mở rộng không gian tìm kiếm, giữ nhiều ứng viên hơn và có thể kết hợp lexicon hoặc language model để chọn chuỗi hợp lý hơn.

Trong phạm vi đồ án, OCRLite ưu tiên greedy decode để giảm độ phức tạp triển khai. Quyết định này phù hợp với mục tiêu chứng minh pipeline end-to-end, nhưng cần được ghi nhận là một giới hạn chất lượng. Nếu muốn cải thiện output trên ảnh bài đọc thật, có thể bật beam search cho recognizer, thêm lexicon correction có kiểm soát hoặc bổ sung rule hậu xử lý cho các pattern thường sai. Ví dụ, nếu một từ tiếng Anh thông dụng có khả năng chứa dấu nháy sở hữu, hậu xử lý có thể xem xét phục hồi apostrophe khi confidence phù

hợp. Dù vậy, hậu xử lý không được sửa quá mạnh vì có thể làm sai tên riêng, số liệu hoặc thuật ngữ chuyên ngành.

Điểm mấu chốt là decoder và hậu xử lý không thay thế dữ liệu tốt. Nếu crop bị mất ký tự, language model chỉ đoán lại theo ngữ cảnh, không phục hồi bằng chứng thị giác thật. Nếu recognizer chưa từng thấy đủ ví dụ dấu gạch nối trong train, beam search cũng khó chọn đúng. Vì vậy, cải thiện OCRLite nên đi theo thứ tự: kiểm tra box, bổ sung dữ liệu punctuation-heavy, fine-tune recognizer, rồi mới tăng độ phức tạp decoder. Thứ tự này giúp tránh tình trạng sửa triệu chứng mà không sửa nguyên nhân.

Bảng 2.4. So sánh greedy decode, beam search và lexicon correction trong OCR.

Kỹ thuật	Ưu điểm	Hạn chế	Đề xuất cho OCRLite
Greedy CTC	Nhanh, dễ export/runtime, ít tham số	Dễ sai ký tự hiếm và không xét ngữ cảnh	Dùng mặc định trong bản nộp
Beam search	Giữ nhiều chuỗi ứng viên, chất lượng tốt hơn	Chậm hơn, cần chọn beam width	Thử nghiệm cho ảnh khó
Lexicon correction	Sửa lỗi từ vựng quen thuộc	Có thể sửa sai tên riêng, số liệu	Bật có kiểm soát theo use-case
Language model	Tận dụng ngữ cảnh câu	Tăng phụ thuộc và chi phí runtime	Hướng phát triển sau đề án

2.13.4 Nguyên tắc đọc metric trong báo cáo đề án

Khi trình bày kết quả OCR, metric phải trả lời đúng câu hỏi. CER trả lời câu hỏi recognizer sai bao nhiêu ký tự trên tập có ground truth. WER cho biết mức độ sai ở cấp từ, thường nghiêm khắc hơn với ứng dụng cần text sạch. Exact match đo tỷ lệ mẫu đọc trùng hoàn toàn, phù hợp với word crop nhưng quá khắt khe nếu áp dụng trực tiếp cho đoạn văn dài. IoU trả lời câu hỏi mask detector trùng vùng chữ thật đến đâu. Latency và RAM trả lời câu hỏi runtime có phù hợp triển khai hay không. Không metric nào một mình phản ánh toàn bộ chất lượng hệ thống full-page.

Với OCRLite, cách trình bày tốt nhất là dùng metric theo tầng trong Chương 4, sau đó bổ sung demo định tính trên ảnh thật. Ví dụ, recognizer fine-tune có best_val_cer thấp hơn baseline cho thấy việc học thêm document crop có tác dụng. Detector có val_iou cao cho thấy segmentation hoạt động trên synthetic pages. Benchmark thời gian/RAM cho thấy package ONNX nhẹ hơn một số engine khác về tốc độ hoặc bộ nhớ. Cuối cùng, output Thylacine chỉ ra các lỗi còn lại ở dấu câu và hyphen. Bốn nhóm thông tin này kết hợp mới tạo thành đánh giá trung thực.

Cách đọc metric như vậy cũng cho thấy đồ án không chỉ chạy được một demo, mà có quy trình đánh giá. Một hệ thống kỹ thuật đáng tin cậy phải biết nó mạnh ở đâu và yếu ở đâu. Nếu chỉ trình bày con số đẹp, báo cáo dễ bị xem là thiếu phản biện. Nếu chỉ liệt kê lỗi, báo cáo lại không cho thấy thành quả huấn luyện. Do đó, Chương 2 cần chuẩn bị nền lý thuyết cho tinh thần đánh giá cân bằng này, còn Chương 4 sẽ dùng các artifact cụ thể để chứng minh.

Tóm lại, phần cơ sở lý thuyết của OCRLite không chỉ là danh sách khái niệm rời rạc. Nó là nền tảng để giải thích toàn bộ thiết kế: dữ liệu synthetic giúp có ground truth, CNN và CRNN giải quyết nhận dạng chuỗi, CTC giải quyết alignment, segmentation giải quyết phát hiện vùng chữ, ONNX Runtime giải quyết triển khai, còn metric theo tầng giúp đánh giá đúng nguồn lỗi. Khi đọc theo mạch đó, Chương 2 trở thành cầu nối tự nhiên giữa phần giới thiệu và phần thiết kế hệ thống.

2.14 Tổng kết chương

Chương 2 đã trình bày cơ sở lý thuyết cho OCRLite: bản chất OCR và pipeline detect–recognize; CNN, CRNN, BiLSTM; CTC và alignment; U-Net segmentation cho detector; tiền xử lý và augmentation; nguồn dữ liệu MJSynth/synthetic; ONNX deployment; metric phân tầng; stack PyTorch/OpenCV/FastAPI. Các khái niệm này là nền để đọc Chương 3 (thiết kế hệ thống) và Chương 4 (kết quả thực nghiệm) mà không bị ngợp thuật ngữ hoặc mất liên kết giữa lý thuyết và triển khai thực tế.

Từ quá trình nghiên cứu tài liệu và triển khai có thể thấy không có hướng tiếp cận OCR nào tối ưu tuyệt đối trên mọi tiêu chí. Pipeline tách module + CTC + ONNX là lựa

chọn cân bằng giữa khả năng hoàn thiện trong phạm vi đồ án, khả năng debug, và mục tiêu OCR nhẹ tích hợp web. Chương 3 sẽ chuyển sang mô tả chi tiết cách các khái niệm lý thuyết trên được hiện thực hóa trong source code và package runtime của OCRLite.

Cuối cùng, chương lý thuyết này cũng làm rõ logic thiết kế: vì sao không end-to-end, vì sao CTC, vì sao segmentation, vì sao ONNX. Nếu bỏ qua phần lý thuyết và chỉ trình bày code, báo cáo sẽ thiếu chiều sâu phân tích — trong khi đồ án tốt nghiệp đánh giá cả hiểu biết nền tảng lẫn khả năng triển khai. Chương 2 vì vậy cung cấp nền tảng lý thuyết theo hướng đầy đủ và có **hệ thống**.

1

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

Chương này trình bày thiết kế hệ thống OCRLite — pipeline nhận dạng văn bản tiếng Anh trên ảnh full-page được thực hiện trong `ocr_submission_package_final`. Khác với Chương 2 (cơ sở lý thuyết CRNN, CTC, U-Net), chương này mô tả yêu cầu, kiến trúc, thiết kế dữ liệu, mô hình, inference, triển khai ONNX, API và tổ chức mã nguồn sao cho có thể đối chiếu trực tiếp với file trong sản phẩm nộp và demo `picture_to_text()`.

OCRLite được thiết kế như một pipeline hoàn chỉnh: ảnh đầu vào → Document Detector (FastTextMaskUNet) → layout post-processing → Text Recognizer (CRNN+CTC) → văn bản. Mỗi mục dưới đây gắn với module cụ thể trong source code để thiết kế có thể được kiểm chứng không chỉ bằng lý thuyết mà bằng artifact chạy được.

3.1 Phân tích yêu cầu hệ thống

3.1.1 Bối cảnh và mục tiêu thiết kế

Trong bối cảnh Engnite và các hệ thống LMS, giáo viên thường có bài đọc tiếng Anh dưới dạng ảnh chụp màn hình hoặc trang tài liệu scan. Nhu cầu là chuyển ảnh thành text có thể chỉnh sửa, tìm kiếm và gắn vào bài tập. Các engine OCR thương mại (EasyOCR, PaddleOCR) chạy được nhưng khó fine-tune trên domain bài đọc, khó giải thích lỗi theo tầng, và thường tốn RAM lớn khi tích hợp backend. OCRLite được thiết kế cho phạm vi hẹp — tiếng Anh in, nền sáng, layout đơn cột — nhưng kiểm soát được toàn bộ vòng train → evaluate → export ONNX → runtime.

Song song với bối cảnh ứng dụng, mục tiêu thiết kế tổng quát của OCRLite là hoàn thiện một vòng OCR đầy đủ trong phạm vi đồ án: chuẩn bị dữ liệu, huấn luyện, phân tích lỗi, export mô hình và đóng gói package runtime. Hệ thống hướng đến pipeline OCR nhẹ, ít tiêu tốn tài nguyên, có tốc độ xử lý cao, đạt độ chính xác ở mức tương đương các engine OCR mã nguồn mở trong điều kiện thử nghiệm (so sánh benchmark với RapidOCR và EasyOCR tại Chương 4), đồng thời dễ tích hợp vào website hoặc dịch vụ backend qua `ocr_engine.py` và `picture_to_text()`. Việc đánh giá không chỉ dựa trên CER hay IoU validation mà còn yêu cầu runtime end-to-end chạy ổn định trên package cuối.

Tiêu chí thành công của thiết kế gồm: (1) hàm `picture_to_text()` chạy ổn định trên package cuối; (2) CER recognizer khoảng 0,0315 sau fine-tune trên document crop; (3) IoU detector khoảng 0,975 trên validation synthetic; (4) thời gian ~5,86 giây/ảnh và RAM peak ~301 MB trong benchmark; (5) mọi thí nghiệm trace được qua config JSON và thư mục `result/reports/`.

Một yêu cầu thiết kế quan trọng là khả năng debug phân tầng: khi output sai, cần xác định lỗi đến từ detector (mask/box), layout (tách dòng/từ), hay recognizer (nhầm ký tự). Vì vậy hệ thống có script `run_document_ocr.py` xuất `detector_boxes.png`, crop mẫu và summary JSON — tách biệt khỏi runtime gọn dùm trong production.

3

3.1.2 Yêu cầu chức năng

Các yêu cầu chức năng chính:

- Nhận ảnh đầu vào (đường dẫn file PNG/JPEG) và trả về văn bản tiếng Anh.
- Phát hiện vùng chữ trên ảnh full-page qua Document Detector.
- Tách dòng, tách từ, sắp xếp reading order bằng layout post-processing.
- Nhận dạng nội dung từng word crop bằng CRNN + CTC.
- Trả kết quả có cấu trúc OCRResult (text, lines, boxes) cho UI/debug.
- Huấn luyện lại recognizer và detector qua config JSON.
- Evaluate và lưu report (predictions, error_samples, metrics).
- Export ONNX và chạy runtime không cần PyTorch train stack.
- Cung cấp CLI (`picture_to_text.py`) và Python API (`ocr_engine.py`).

Bảng 3.1. Ánh xạ yêu cầu chức năng và module kiểm chứng.

Yêu cầu	Module / entry point
Ảnh → text	<code>ocr_engine.py</code> → <code>OCREngine.read_text()</code>
Full-page pipeline	<code>src/ocrlite/engine.py</code>
Train recognizer	<code>scripts/train_recognizer.py</code>
Train detector	<code>scripts/train_document_detector.py</code>
Runtime ONNX	<code>models/*.onnx</code> , <code>inference/onnx_runner.py</code>
Debug visualization	<code>scripts/run_document_ocr.py</code>
Benchmark	<code>scripts/benchmark_picture_to_text.py</code>

5

3.1.3 Yêu cầu phi chức năng

Về hiệu năng, thiết kế hướng tới ~5–6 giây/ảnh trên CPU và batch recognizer 64 — đo bằng `benchmark_picture_to_text.py` trên bộ 4 ảnh test. Về bộ nhớ, peak runtime khoảng 301 MB, thấp hơn đáng kể EasyOCR trong cùng điều kiện (Chương 4). Hệ thống chạy CPU hoặc GPU tùy ONNX Runtime provider (CUDA nếu có).

Về tái lập thí nghiệm: mỗi run train lưu config snapshot, `training_log.csv`, `metrics.json` trong `result/.../reports/`. Seed=42 cố định trong config. Về bảo trì: cấu trúc thư mục theo chức năng, tham số experiment nằm trong `configs/*.json` thay vì hard-code.

3.1.4 Phạm vi và giới hạn thiết kế

Phạm vi: văn bản tiếng Anh in, ảnh nền sáng, tài liệu đơn cột hoặc screenshot bài đọc. Chưa hỗ trợ: tiếng Việt có dấu, chữ viết tay, bảng biểu phức tạp, layout nhiều cột, giữ nguyên chữ hoa (`text_case=lower` trong `Charset`). FastAPI service hiện có (`server/app.py`) thiên về OCR crop/line đơn lẻ; full-page OCR qua `OCREngine` là luồng chính của package nộp.

3.1.5 Kịch bản sử dụng

Kịch bản 1 — Giáo viên upload ảnh bài đọc: backend Django lưu file, gọi `read_text()`, trả text vào form assignment. `OCRResult.lines` hỗ trợ preview từng dòng nếu cần chỉnh sửa.

Kịch bản 2 — Debug sau deploy: dev tải ảnh lỗi, chạy `run_document_ocr.py` local với cùng config JSON production, so sánh box/crop trước khi quyết định retrain hay chỉnh layout threshold.

Kịch bản 3 — Huấn luyện lại trên Kaggle: zip code, chạy notebook theo `KAGGLE_GUIDE.md`, export ONNX về thay trong `models/` — API `from_package()` không đổi khi swap weight.

3.2 Kiến trúc tổng thể

3.2.1 Training side và Runtime side

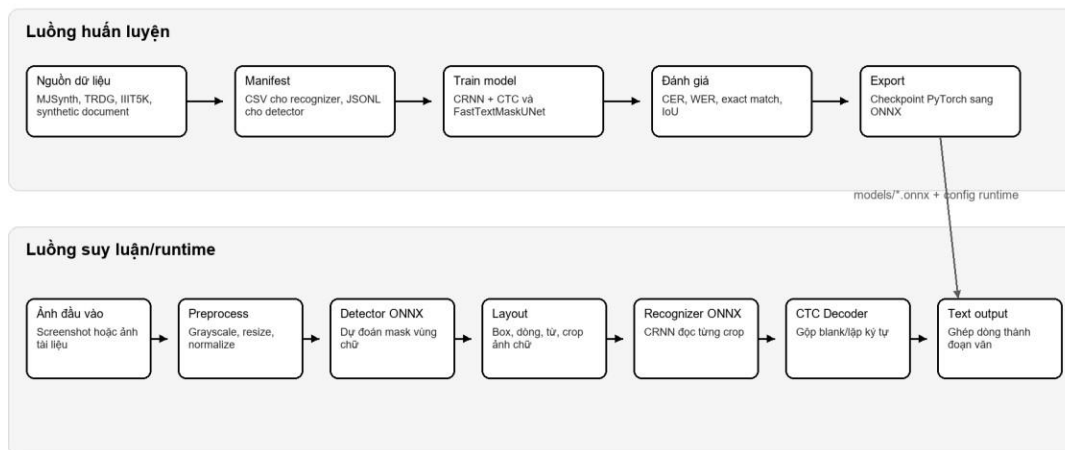
Kiến trúc tách hai nhánh. Training side: chuẩn bị dữ liệu, huấn luyện PyTorch, validate, lưu checkpoint `.pth`, export ONNX, ghi report vào `result/`. Runtime side: chỉ load `models/document_detector.onnx`, `models/recognizer_crnn.onnx` và hai file config JSON;

chạy inference qua OCREngine — không cần thư mục result/ hay môi trường train.

Thiết kế này giảm footprint khi nộp package và khi tích hợp web: backend chỉ deploy runtime slice. Training chạy trên Kaggle GPU; output ONNX copy về models/ trên máy local.

Flowchart hệ thống OCRLite

Thiết kế tách thành hai luồng: huấn luyện tạo model và runtime sử dụng ONNX để nhận dạng văn bản.



Hình 3.1. Sơ đồ khối Training side và Runtime side.

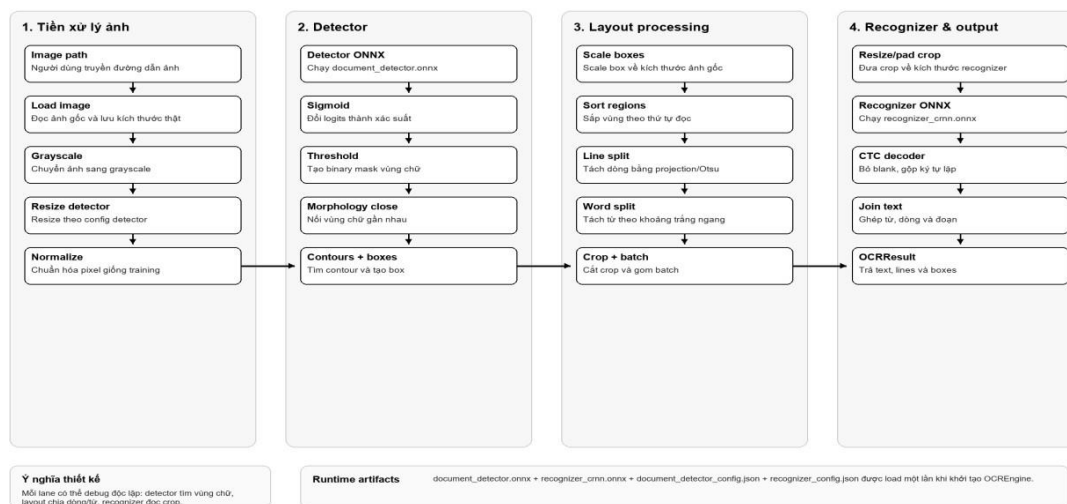
3.2.2 Tách Detector, Layout và Recognizer

Detector (FastTextMaskUNet): ảnh page grayscale → mask logits → vùng chữ. Layout (OpenCV + heuristic trong engine.py, layout.py): mask → box → dòng → từ → crop. Recognizer (CRNN+CTC): word crop → chuỗi ký tự. Engine ghép ba tầng và trả OCRResult.

Lý do tách module: (1) dữ liệu train khác nhau — word crop vs page+mask; (2) debug theo tầng; (3) thay thế từng phần không cần end-to-end retrain. Giao diện giữa module: box (x1,y1,x2,y2) tọa độ ảnh gốc; recognizer nhận tensor batch×1×32×W.

Flowchart chi tiết luồng OCR runtime

Từ ảnh đầu vào đến OCRResult: mỗi lane thể hiện một nhóm trách nhiệm trong pipeline.



Hình 3.2. Pipeline Detector — Layout — Recognizer.

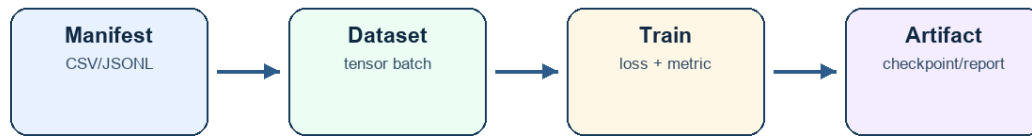
3.2.3 Luồng training và runtime

Luồng recognizer: MJSynth → manifest CSV → split train/val → OCRLinesDataset → train CRNN (CTCLoss) → validate CER → export recognizer_cmn.onnx. Fine-tune: build_recognizer_finetune_data tạo document crops → train với LR nhỏ từ checkpoint base.

Luồng detector: word crops → build_document_synth sinh trang synthetic + mask + JSONL → DocumentMaskDataset → train FastTextMaskUNet (BCE+Dice) → validate IoU → export document_detector.onnx.

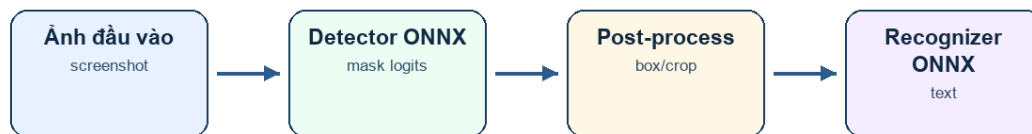
Luồng runtime: OCREngine.read() — đọc ảnh → preprocess page → detector ONNX → sigmoid/threshold → morphology → contours → scale box → tách dòng/từ → batch recognizer ONNX → decode CTC → ghép lines → join_lines_as_paragraphs → OCRResult.

Flowchart luồng huấn luyện.



Hình 3.3. Flowchart luồng huấn luyện.

Flowchart luồng runtime OCREngine.read().



Điểm kiểm soát thiết kế: mỗi bước có input/output riêng để dễ debug và thay thế từng tầng.

Hình 3.4. Flowchart luồng runtime OCREngine.read().

3.2.4 Phương án đã xem xét và loại bỏ

End-to-end transformer OCR: loại vì thiếu dữ liệu full-page có alignment, chi phí train cao, khó debug box. Tesseract nguyên khối: có trong Engnite legacy nhưng khó fine-tune và CER kém trên screenshot. Ship PyTorch .pth runtime: loại vì footprint; chọn ONNX + ONNX Runtime. Detector bbox regression: loại vì synthetic mask từ paste tự sinh GT — segmentation U-Net hợp lý hơn.

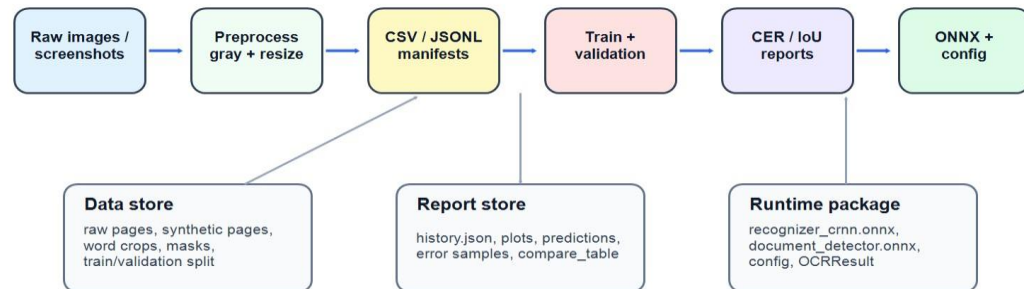
3.3 Thiết kế dữ liệu và tiền xử lý

3.3.1 Vòng đời dữ liệu

Recognizer dùng manifest CSV hai cột image_path, text. Detector dùng JSONL mỗi dòng một trang (image_path, mask_path, boxes). Vòng đời recognizer: raw → manifest → filter charset (Unicode NFC) → split → dataset → train → evaluate. Vòng đời detector: word crops → synthetic page → mask → JSONL → DocumentMaskDataset → train.

OCRLite Data Lifecycle

From raw inputs to ONNX artifacts and traceable runtime outputs



Hình 3.5. Sơ đồ vòng đời dữ liệu.

Bảng 3.2. Nguồn dữ liệu và script liên quan.

Nguồn	Vai trò	Script
MJSynth (HF)	Base recognizer	export_mjsynth_hf.py
TRDG / synthetic	Bổ sung word crop	data/synthetic.py
IIT5K	Benchmark	evaluate_recognizer.py
Synthetic document	Train detector	build_document_synth.py
Document crops	Fine-tune recognizer	build_recognizer_finetune_data.py

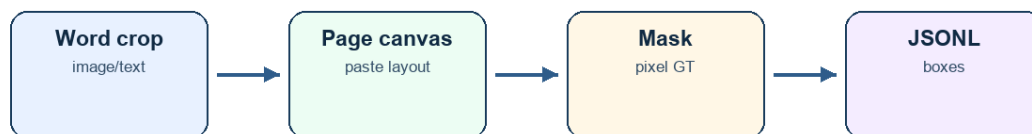
3.3.2 Manifest và chuẩn hóa

Module data/manifest.py cung cấp read_manifest, filter_rows_by_charset, dedupe_rows, split train/val/test. Lốp chặn lỗi trước dataloader — label ngoài charset hoặc trùng lặp gây train không ổn định. JSONL detector: nested boxes không fit CSV phẳng; mỗi record gồm image, mask và tùy chọn boxes kèm text cho debug.

3.3.3 Synthetic document

build_document_synth.py đọc word crops, paste lên canvas trắng theo tham số page_width (768), page_height (1024), margin, line_gap, word_gap, font_height [24–38]. Sinh mask GT tự động từ tọa độ paste — không gán nhãn mask thủ công. page_count=30000 train, val_page_count=1500. Random hóa layout tránh model học vị trí cố định.

Trang synthetic, mask GT và overlay box.



Hình 3.6. Trang synthetic, mask GT và overlay box.

Domain gap: synthetic không mô phỏng đầy đủ anti-aliasing screenshot thật — IoU val cao nhưng full-page thật vẫn cần fine-tune recognizer và tinh layout threshold. Đây là giới hạn thiết kế cần ghi nhận trung thực trong báo cáo.

3.3.4 Tiền xử lý và augmentation

Recognizer: grayscale, resize/pad height=32, width=192 (config), preserve_aspect_ratio, pad nền trắng, normalize đồng bộ train/inference (preprocess.py). Detector runtime: preprocess_page resize 384×512, normalize $x/127.5-1$; scale box ngược về tọa độ gốc sau inference.

Augmentation recognizer (config augment): blur, noise, rotate, brightness — xác suất trong JSON. Fine-tune dùng augmentation nhẹ hơn base train. Detector page: blur/nền biến thiên nhẹ trên synthetic.

3.3.5 Charset

Charset (utils/charset.py): blank index=0; ký tự từ index 1; text_case=lower. Encode cho CTCLoss; decode greedy + collapse (decoder.py). Charset lưu trong recognizer_config.json kèm ONNX — num_classes phải khớp weight, không thể đổi charset chỉ bằng sửa JSON.

Bảng 3.3. Nhóm tham số config recognizer (trích recognizer_config.json).

Nhóm	Tham số tiêu biểu	Vai trò thiết kế
Kiến trúc	model_name=crnn, cnn_channels, rnn_hidden_size=192	CRNN+BiLSTM
Input	image_height=32, image_width=192	Chuẩn hóa crop
Train	batch_size=384, lr=0.00035, epochs=10	Fine-tune trên T4
Loss/metric	loss=ctc, best_val_cer	Chọn checkpoint
Augment	blur/noise/rotate prob	Ablation qua JSON
Output	charset, text_case=lower	Decode runtime

Bảng 3.4. Nhóm tham số config detector (trích document_detector_config.json).

Nhóm	Tham số tiêu biểu	Vai trò thiết kế
Synthetic	page_count=30000, page 768×1024	Sinh dữ liệu train
Model	model_variant=fast, base_channels=16	FastTextMaskUNet
Train	batch_size=48, grad_accum=2, target_val_iou=0.97	Kaggle T4×2
Inference	train_height=512, train_width=384	Input ONNX runtime
Layout	threshold=0.45, min_box_area=24, dilate_x/y	Mask → box

3.4 Thiết kế mô hình và huấn luyện

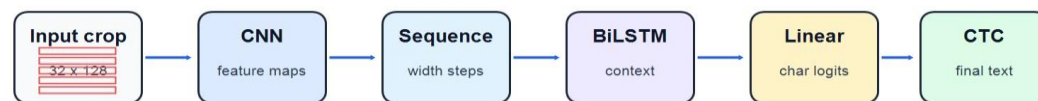
3.4.1 Text Recognizer — CRNN + CTC

File models/crnn.py (CRNNRecognizer), factory.py chọn model_name. Kiến trúc: CNN backbone (channels [40,80,160,224,320]) → collapse height → sequence → BiLSTM 2 lớp (hidden 192) → Linear → logits (T×batch×num_classes). Loss CTCLoss;

decode runtime greedy. Biến thể CRNNTransformer, MobileSVTR, STN có trong code nhưng package nộp dùng crnn vì ổn định ONNX.

CRNNRecognizer Architecture

Word crop -> visual features -> sequence model -> CTC text decoding



Design note

CTC removes the need for manual character segmentation and supports variable-length words.

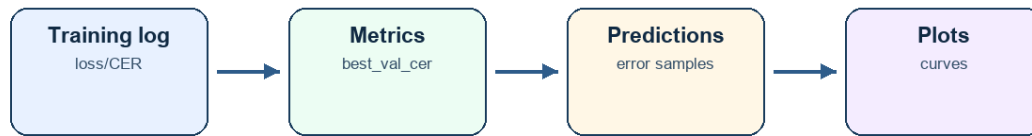
Hình 3.7. Sơ đồ kiến trúc CRNNRecognizer.

Chiều cao ảnh cố định 32 px sau CNN stride tạo sequence length T tỷ lệ với chiều rộng crop. BiLSTM 2 lớp (hidden 192) model ngữ cảnh hai chiều — hữu ích với từ dài và ký tự dễ nhầm (i/l, o/0). Dropout 0,08 giảm overfit trên fine-tune. Export ONNX dùng dynamic axis cho chiều rộng — cần verify bằng `evaluate_onnx_recognizer.py` trước khi thay file trong models/.

3.4.2 Huấn luyện và fine-tune recognizer

Entry scripts/train_recognizer.py → train.py. AdamW, scheduler plateau, AMP trên Kaggle. Validation: CER, WER, exact match; checkpoint theo best_val_cer. Output: predictions.csv, error_samples.csv, char_confusion.csv, metrics.json. Base CER ≈ 0,0403; fine-tune document crop ≈ 0,0315 — giảm domain gap giữa MJSynth word crop và crop runtime từ layout.

Biểu đồ loss/CER recognizer (result/reports/).



Hình 3.8. Biểu đồ loss/CER recognizer (result/reports/).

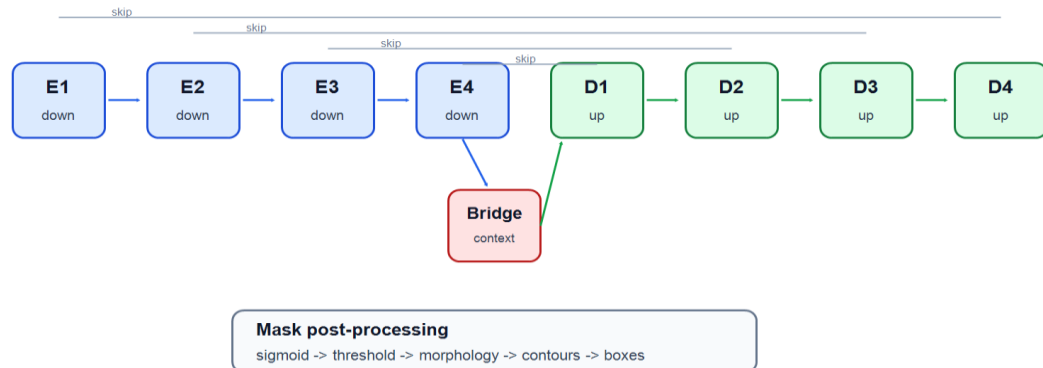
Fine-tune (build_recognizer_finetune_data.py) trộn document crops từ synthetic page, token giàu dấu câu và một phần train cũ. Load pretrained base, learning_rate=0,00035 nhỏ hơn train from scratch, augmentation giảm intensity. Mục tiêu: crop runtime từ layout gần phân phối MJSynth word crop — giảm lỗi dấu chấm, ngoặc, gạch ngang trên bài đọc thật.

3.4.3 Document Detector — FastTextMaskUNet

document/model.py: encoder-decoder U-Net, base_channels=16, variant fast. Input page $1 \times H \times W$; output mask logits. Loss BCE + Dice; metric val_iou $\approx 0,975$ trên synthetic val. train_document_detector.py hỗ trợ torchrun multi-GPU, AMP, channels_last, early stopping.

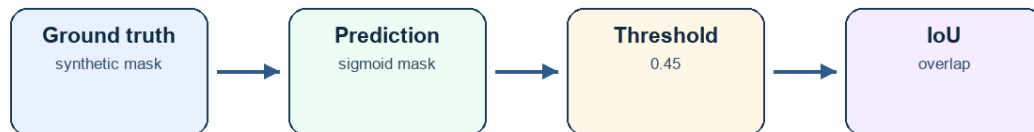
FastTextMaskUNet Detector

Encoder-decoder segmentation produces a text mask; post-processing turns it into word boxes



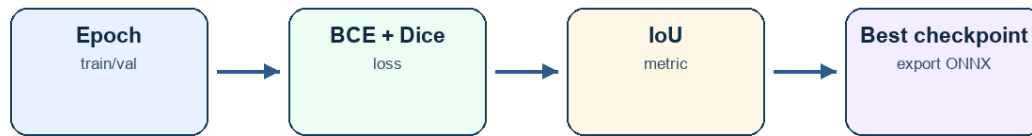
Hình 3.9. Kiến trúc FastTextMaskUNet.

Mask GT vs dự đoán (validation).



Hình 3.10. Mask GT vs dự đoán (validation).

Biểu đồ val_iou / loss detector.



Hình 3.11. Biểu đồ val_iou / loss detector.

Loss BCE xử lý pixel-wise; Dice loss cân bằng class imbalance (nền trắng chiếm đa số pixel). Kết hợp hai loss giúp mask không bị «chảy» sang nền. Train 30000 trang synthetic với val_page_count=1500 tách seed — metric IoU val $\approx 0,975$ không đồng nghĩa full-page ảnh chụp màn hình đạt cùng mức; cần đánh giá end-to-end ở Chương 4.

3.4.4 Vòng lặp huấn luyện

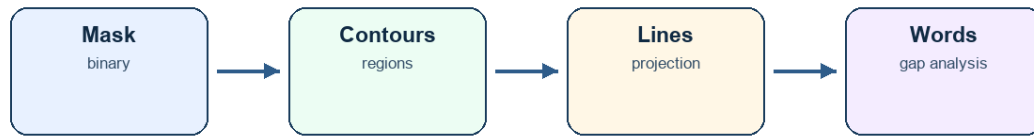
Mỗi epoch: train pass (AMP, grad_clip) → val pass → scheduler step → lưu best nếu CER/IoU cải thiện. early_stopping_patience dừng khi plateau. abort_if_no_signal_epochs bảo vệ GPU khi config sai. Mỗi run tạo thư mục timestamp trong result/ — không ghi đè run cũ.

3.5 Thiết kế inference, layout và runtime

3.5.1 Layout post-processing

layout.py + engine.py: sigmoid → threshold (0.45) → morphology close (dilate 9×3) → findContours → lọc min_box_area → scale về ảnh gốc → order_boxes_by_reading_flow → boxes_to_lines → split_region_into_text_lines (projection OTSU) → segment_words_in_line (gap $\sim 0.22 \times \text{line_height}$) → filter_tiny_word_boxes.

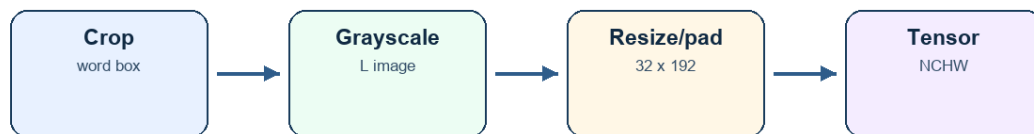
Mask → box → dòng → từ (detector_boxes.png).



Điểm kiểm soát thiết kế: mỗi bước có input/output riêng để dễ debug và thay thế từng tầng.

Hình 3.12. Mask → box → dòng → từ (detector_boxes.png).

Word crop trước/sau resize/pad.



Hình 3.13. Word crop trước/sau resize/pad.

Layout là tầng heuristic quan trọng nhất sau detector: U-Net cho mask tốt nhưng không thay thế tách dòng/từ và reading order. Tham số `box_dilate_x/y`, `gap_threshold` và `min_box_area` điều chỉnh qua config JSON và quan sát `detector_boxes.png` — không retrain khi chỉ cần chỉnh post-processing.

3.5.2 OCREngine và OCRResult

`OCREngine.from_package()` nạp 4 file trong `models/`. `read()` → `OCRResult(text,`

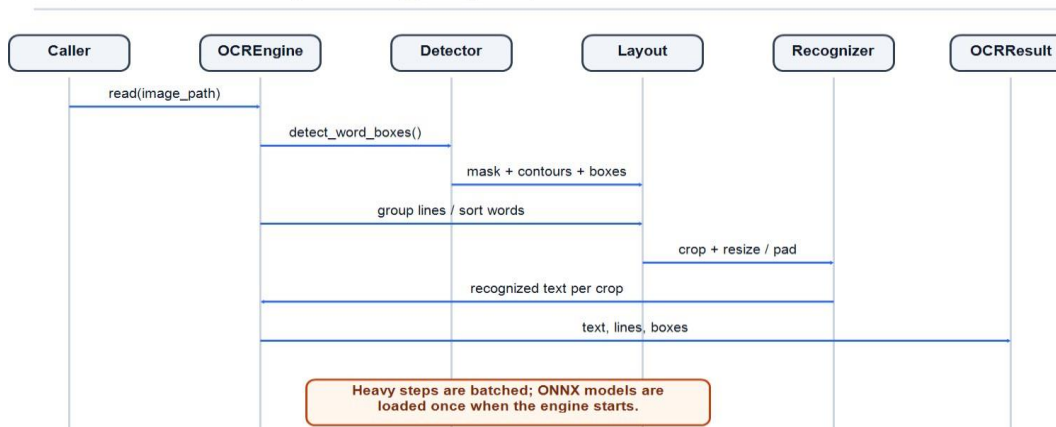
lines, boxes); read_text() → str. ocr_engine.py expose create_engine(), picture_to_text() cho tích hợp nhanh.

Bảng 3.5. Input/Output các thành phần runtime.

Thành phần	Input	Output
Detector	1×1×512×384	Mask logits
Layout	Mask + kích thước gốc	List[List[Box]]
Recognizer	N×1×32×W	List[str]
OCREngine	image_path	OCRResult

Runtime Sequence: OCREngine.read()

Components are isolated so each stage can be debugged independently



Hình 3.14. Sequence diagram OCREngine.read().

3.5.3 Phân tích pipeline runtime (16 bước)

Phần dưới mô tả tuần tự OCREngine.read() trong src/ocrlite/engine.py. Đây là luồng cốt lõi runtime — mỗi bước có thể quan sát riêng qua run_document_ocr.py.

Bước 1 — Nạp ảnh và metadata: read() mở file bằng Path và PIL Image.open để lấy original_size (width, height). Kích thước gốc được lưu ngay từ đầu vì detector chạy trên ảnh đã resize — mọi box phải scale ngược chính xác trước khi crop cho recognizer. Bỏ qua bước này sẽ gây lệch box trên ảnh debug dù mask trông đúng.

Bước 2 — Preprocess page cho detector: Hàm preprocess_page (layout.py) chuyển ảnh sang grayscale, resize về 384×512 theo document_detector_config.json, normalize (pixel/127.5 – 1) thành tensor 1×1×H×W float32. Normalize phải khớp lúc export ONNX

— sai lệch mean/std là nguyên nhân phổ biến khiến mask runtime khác validation PyTorch.

Bước 3 — Inference detector ONNX: `detector_session.run()` gọi ONNX Runtime với input name đọc động từ model. Output là logits mask. Thiết kế dùng ORT thay PyTorch ở runtime để giảm dependency khi deploy Engnite — chỉ cần `pip install onnxruntime opencv-python-headless`.

Bước 4 — Sigmoid và ngưỡng mask: Áp sigmoid lên logits, so sánh với `threshold=0.45` (config). Ngưỡng quá cao làm mất chữ mờ trên screenshot; quá thấp đưa nhiều nền vào contour. Tham số này tinh chỉnh bằng histogram mask trên ảnh demo Thylacine (Chương 4).

Bước 5 — Morphology close: `cv2.morphologyEx MORPH_CLOSE` với kernel (9, 3) nối các vùng mực bị đứt. Chọn close thay open để «lấp lỗ» trong nét chữ in mà không xóa stroke mảnh quá mức.

Bước 6 — `findContours` và bounding box: `RETR_EXTERNAL` lấy contour ngoài; `boundingRect` tạo box; lọc theo `min_box_area=24` loại speckle. Số contour thừa làm tăng tải layout và recognizer — lọc sớm ở đây.

Bước 7 — Scale box về tọa độ gốc: `scale_x = orig_width / mask_width`; nhân tọa độ và clamp vào biên ảnh. Làm tròn int có thể lệch 1 pixel — chấp nhận được nhờ pad crop recognizer.

Bước 8 — Sắp xếp reading order: `order_boxes_by_reading_flow` gom box cùng dòng theo `center_y` (ngưỡng $\sim 0,75 \times \text{median height}$), sắp xếp trái sang phải, trên xuống dưới. Heuristic giả định layout đơn cột — đúng phạm vi đồ án.

Bước 9 — Tách vùng thành dòng text: `split_region_into_text_lines` dùng OTSU binary và projection theo hàng để cắt block lớn thành line box. `merge_close_runs` gộp hàng mực gần nhau — tránh cắt một dòng thành nhiều mảnh.

Bước 10 — Tách dòng thành word box: `segment_words_in_line` projection theo cột; `gap_threshold  $\approx 0,22 \times \text{line\_height}$` quyết định ranh giới từ. Đây là bước nhạy cảm với dấu câu và hyphen — nhiều lỗi OCR thực tế bắt nguồn từ crop cắt sát hoặc gộp hai từ.

Bước 11 — Lọc box nhiễu: `filter_tiny_word_boxes` loại chấm nhỏ và fragment so với median height dòng. Tránh gửi crop 3×3 pixel vào recognizer gây output rác và lãng phí

batch slot.

Bước 12 — Crop và chuyển tensor recognizer: `crop_to_tensor` cắt ảnh L theo box, gọi `image_to_tensor`: `height=32`, `pad` trắng, `width_multiple=16` cho ONNX. `preserve_aspect_ratio` giữ tỷ lệ nét chữ.

Bước 13 — Batch inference recognizer: Gom tối đa `recognizer_batch_size=64` crop; `ONNXRecognizer.predict_batch` chạy `session.run`. Batch cân bằng latency và RAM — benchmark Chương 4 dùng default 64.

Bước 14 — Decode CTC: decoder greedy: collapse blank index 0, gộp ký tự lặp, map `index` → `charset` (lower). Beam search có trong config serve nhưng package nộp dùng greedy đủ ổn cho demo.

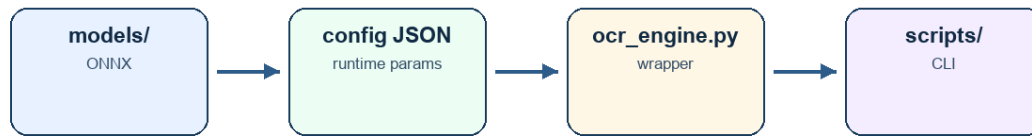
Bước 15 — Ghép prediction theo dòng: Cursor duyệt danh sách prediction phẳng, chia theo số box mỗi dòng → `lines: list[list[str]]`. Cấu trúc giữ alignment word ↔ box phục vụ UI overlay trên frontend.

Bước 16 — Ghép paragraph: `join_lines_as_paragraphs` phân đoạn theo khoảng cách dòng median và heuristic `is_heading_like` (dòng ngắn, ít từ). Output text có xuống dòng kép giữa đoạn — phù hợp hiển thị bài đọc LMS.

3.5.4 ONNX export và package runtime

`export_onnx.py`, `export_document_detector_onnx.py`: verify PyTorch vs ORT. Package runtime models/ gồm hai ONNX + hai config JSON — không đóng gói checkpoint .pth. Dynamic axes cho batch và width recognizer. `onnx_runner.py` bọc `InferenceSession`, chọn CUDA/CPU provider.

Cấu trúc thư mục models/.



Hình 3.15. Cấu trúc thư mục models/.

3.5.5 Module runtime chính

engine.py: OCREngine — detect_word_boxes(), recognize_boxes(), read().
 layout.py: preprocess_page, mask_to_word_boxes, order_boxes, boxes_to_lines.
 onnx_runner.py: ONNXRecognizer.predict_batch. preprocess.py: image_to_tensor.
 decoder.py: greedy CTC decode. ocr_engine.py: wrapper picture_to_text().

3.6 API, CLI và tích hợp

3.6.1 Python API và Django

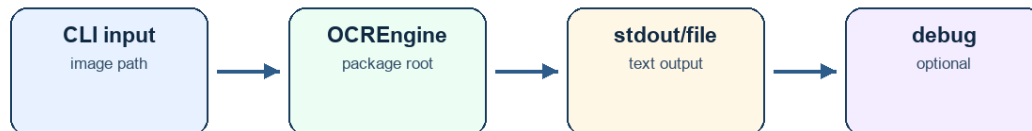
Pattern tích hợp Engnrite: module ocr_service.py khởi tạo engine = OCREngine.from_package() trong AppConfig.ready() hoặc lazy singleton có lock. View nhận upload ImageField, lưu tempfile, gọi read_text(), trả JsonResponse {text, lines, optional boxes}. Engine không reload mỗi request — ONNX session init tốn vài giây và ~300 MB RAM.

Bảo mật tầng web (ngoài OCR core): giới hạn 5 MB/file, whitelist png/jpeg, sanitize tên file, CSRF cho form Django. Không log full text bài đọc nếu policy LMS yêu cầu — chỉ log duration và số box.

3.6.2 CLI và FastAPI

scripts/picture_to_text.py: đường dẫn ảnh + package root + optional --output txt.
 server/app.py: endpoint crop/line, DynamicBatcher, blur/entropy quality gate — mở rộng
 production; full-page dùng OCREngine trực tiếp.

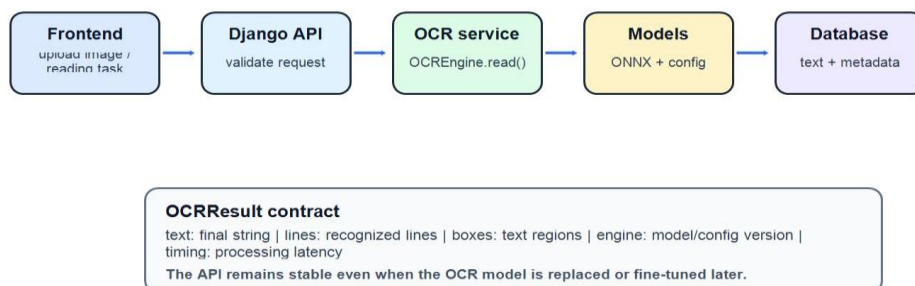
Demo CLI picture_to_text.



Hình 3.16. Demo CLI picture_to_text.

Engnrite Backend Integration

OCRLite is kept as an internal service; Django receives normalized OCRResult objects



Hình 3.17. Tích hợp backend Engnrite/Django.

3.6.3 So sánh với tesseract_ocr legacy

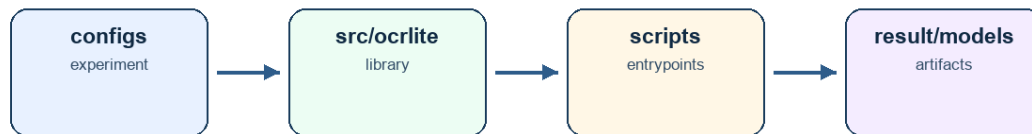
Engnrite có module tesseract_ocr/ phụ thuộc binary Tesseract — khó fine-tune, khó benchmark CER thống nhất. OCRLite thay thế luồng bài đọc tiếng Anh bằng

picture_to_text() + ONNX. Migration an toàn: feature flag USE_OCRLITE, fallback Tesseract nếu engine init fail.

3.7 Tổ chức source code

configs/ — JSON experiment. data/ — raw, processed, splits. src/ocrlite/ — core (models, data, document, inference, server, utils). scripts/ — CLI train/export/benchmark. notebooks/ — Kaggle. result/ — artifact train. models/ — ONNX runtime. test/ — ảnh kiểm thử.

Cây thư mục package (rút gọn).

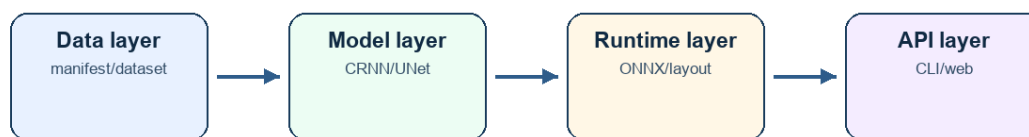


Hình 3.18. Cây thư mục package (rút gọn).

Bảng 3.6. Phân lớp source code.

Lớp	Vai trò	File tiêu biểu
Application	Entry point	ocr_engine.py, picture_to_text.py
Engine	Full-page pipeline	engine.py
Inference	ONNX, decode	onnx_runner.py, decoder.py
Model	Kiến trúc NN	crnn.py, document/model.py
Training	Train loop	train.py, train_document_detector.py
Data	Manifest, dataset	manifest.py, document/dataset.py
Utils	Metric, charset	metrics.py, charset.py

Sơ đồ phân lớp (layer diagram).



Hình 3.19. Sơ đồ phân lớp (layer diagram).

Bảng 3.7. Quy trình train trên Kaggle.

Bước	Script	Output
Zip code	make_kaggle_code_zip.py	ocr_project/
MJSynth	export_mjsynth_hf.py	manifest + crops
Split	build_english_splits.py	train/val/test.csv
Train rec	train_recognizer.py	best.pth, metrics
Synth doc	build_document_synth.py	train/val.jsonl
Train det	train_document_detector.py	detector.pth
Export	export_*.py	models/*.onnx

3.7.1 Module training và evaluate

train.py — vòng epoch recognizer. evaluate.py / evaluate_recognizer.py — metric tập test. build_document_synth.py — synthetic page. export_onnx.py — chuyển checkpoint sang ORT. utils/results.py — ghi predictions, error_samples, confusion matrix tự động phục vụ Chương 4.

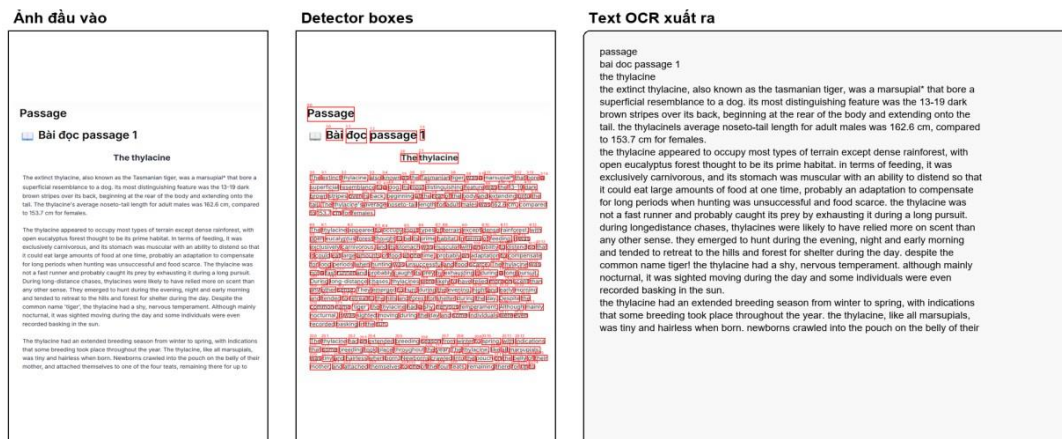
3.8 Kiểm thử và debug

Chiến lược phân tầng: (L1) charset roundtrip; (L2) preprocess shape; (L3) decoder known logits; (L4) ONNX vs PyTorch; (L5) detector box count ảnh vàng; (L6) full pipeline end-to-end. Script: test_ocr_brains.py (benchmark RAM/time), benchmark_picture_to_text.py, run_document_ocr.py.

Quy trình debug output sai: (1) xem detector_boxes.png; (2) kiểm crop có mất dấu câu; (3) chạy recognizer riêng trên crop lỗi; (4) đọc error_samples.csv; (5) mới quyết retrain hay chỉnh layout — tránh sửa nhầm tâng.

Minh họa kết quả OCR full-page

Ảnh đầu vào được detector tìm vùng chữ, sau đó recognizer CRNN + CTC đọc từng crop để ghép thành text.



Ghi chú: kết quả lấy từ package_runtime_output.txt trong bộ nộp cuối cùng.

Hình 3.20. Debug end-to-end: input, boxes, crop, text (ảnh Thylacine).

3.8.1 Triển khai production (tóm tắt)

Gunicorn/Uvicorn: engine singleton mỗi worker (~301 MB×N). Nginx: client_max_body_size, timeout ≥30s. Validate input size/mime. Docker multi-stage: chỉ COPY models/ + runtime, không copy result/. Health check endpoint optional.

Quyết định	Lý do
Tách detector / recognizer	Dữ liệu train khác, debug theo tầng
Synthetic document + mask	GT tự động, không label thủ công
Layout sau U-Net	Reading order, tách dòng/từ
ONNX runtime	Deploy nhẹ, không PyTorch train
Fine-tune document crop	Giảm domain gap MJSynth vs runtime
OCRResult có boxes/lines	UI overlay và debug

3.9 Thiết kế yêu cầu và phạm vi nghiệp vụ

Mục tiêu thiết kế: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần

phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một

cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. thiết kế được ánh xạ trực tiếp sang các module trong ocr_submission_package_final, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. mỗi quyết định đều cần

artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. phương án được ưu tiên vì cân bằng giữa độ chính xác, tốc độ, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án.

Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON,

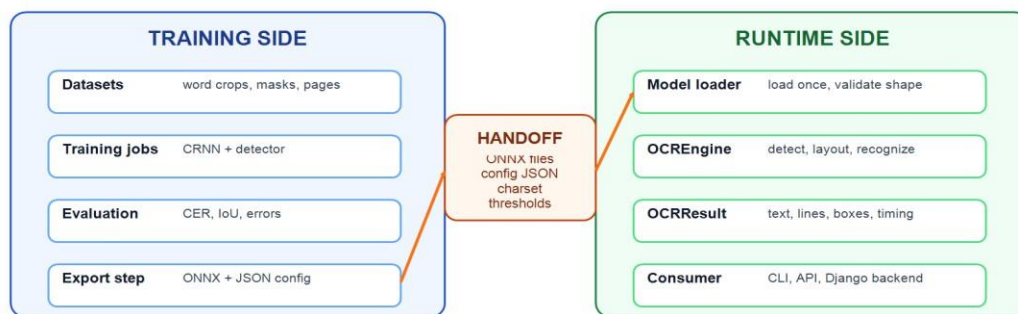
source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động báo cáo: trong 3.9 thiết kế yêu cầu và phạm vi nghiệp vụ, trọng tâm là bài toán OCR tiếng Anh full-page. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh đầu vào, text, lines và boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, yêu cầu phải tuân theo ràng buộc phạm vi một cột nền sáng, đồng thời vẫn hỗ trợ độ ổn định khi tích hợp backend. mô tả rõ tầng này giúp phân kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

3.10 Kiến trúc hai nhánh train và runtime

Train / Runtime Contract Map

Artifacts and contracts that connect offline training with online OCR inference



Hình 3.21. Bản đồ contract giữa nhánh huấn luyện và nhánh runtime của OCRLite.

Mục tiêu thiết kế: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX,

config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ

thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. thiết kế được ánh xạ trực tiếp sang các module trong `ocr_submission_package_final`, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời

vẫn hỗ trợ khả năng thay thế model. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. phương án được ưu tiên vì **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. cấu trúc module hóa cho phép

thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động báo cáo: trong 3.10 kiến trúc hai nhánh train và runtime, trọng tâm là training side và runtime side. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm checkpoint, ONNX, config JSON; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, kiến trúc phải tuân theo ràng buộc ranh giới train/deploy, đồng thời vẫn hỗ trợ khả năng thay thế model. mô tả rõ tầng này giúp phần kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package,

các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Bảng 3.9. Ma trận phân tích cho Kiến trúc hai nhánh train và runtime.

Khía cạnh	Nội dung thiết kế	Minh chứng/kiểm tra
Input	checkpoint, ONNX, config JSON	schema, config hoặc hàm endpoint
Ràng buộc	ranh giới train/deploy	phạm vi thiết kế và test case
Chất lượng	khả năng thay thế model	metric, benchmark hoặc debug artifact
Traceability	training side và runtime side	module trong source code và Chương 4

3.11 Thiết kế dữ liệu recognizer

Mục tiêu thiết kế: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ

mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. thiết kế được ánh xạ trực tiếp sang các module trong ocr_submission_package_final, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một

đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. phương án được ưu tiên vì cân bằng giữa độ chính xác, tốc độ, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về

4

mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

4

4

7

Tác động runtime: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ trợ truy vết lỗi theo sample. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

4

4

Tác động báo cáo: trong 3.11 thiết kế dữ liệu recognizer, trọng tâm là manifest CSV và charset. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image_path, text, split; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc chuẩn hóa Unicode NFC, đồng thời vẫn hỗ

trợ truy vết lỗi theo sample. mô tả rõ từng tầng này giúp phân kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

3.12 Thiết kế dữ liệu detector

Mục tiêu thiết kế: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái

ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng

thời vẫn hỗ trợ khả năng sinh nhân tự động. thiết kế được ánh xạ trực tiếp sang các module trong `ocr_submission_package_final`, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp

và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. phương án được ưu tiên vì **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các

quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

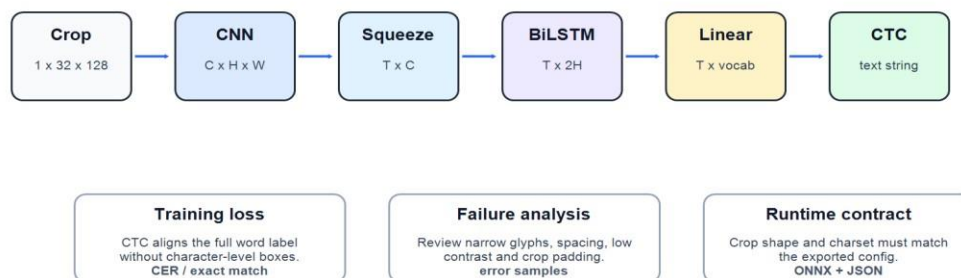
Tác động báo cáo: trong 3.12 thiết kế dữ liệu detector, trọng tâm là synthetic document và mask. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm page image, mask, boxes; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, dữ liệu phải tuân theo ràng buộc domain gap synthetic/thật, đồng thời vẫn hỗ trợ khả năng sinh nhân tự động. mô tả rõ tầng này giúp phần kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách

trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

3.13 Thiết kế CRNN Recognizer

CRNN Tensor Flow and Error Surfaces

Recognizer design with tensor dimensions, decoding boundary, and validation signals



Hình 3.22. Luồng tensor và bề mặt lỗi cần kiểm soát trong CRNNRecognizer.

Mục tiêu thiết kế: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ

ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer

trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. thiết kế được ánh xạ trực tiếp sang các module trong ocr_submission_package_final, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng

tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. phương án được ưu tiên vì **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ

mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động báo cáo: trong 3.13 thiết kế crnn recognizer, trọng tâm là CNN, BiLSTM và CTC. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm tensor 32x192, logits theo timestep; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc lỗi ký tự nhỏ, đồng thời vẫn hỗ trợ CER, WER và exact match. mô tả rõ tầng này giúp phân kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

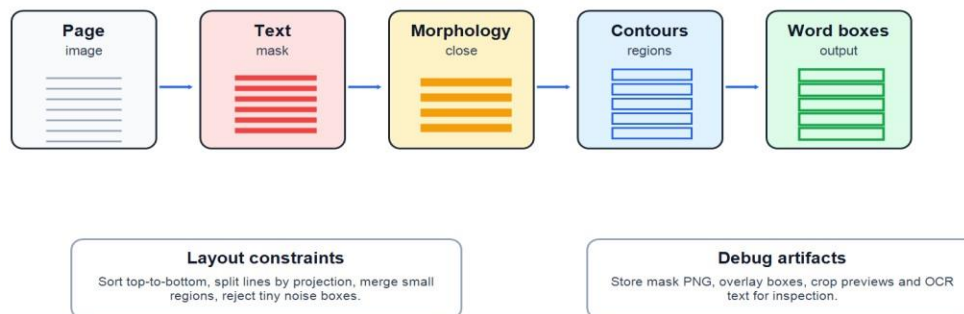
Bảng 3.12. Ma trận phân tích cho Thiết kế CRNN Recognizer.

Khía cạnh	Nội dung thiết kế	Minh chứng/kiểm tra
Input	tensor 32x192, logits theo timestep	schema, config hoặc hàm endpoint
Ràng buộc	lỗi ký tự nhỏ	phạm vi thiết kế và test case
Chất lượng	CER, WER và exact match	metric, benchmark hoặc debug artifact
Traceability	CNN, BiLSTM và CTC	module trong source code và Chương 4

3.14 Thiết kế Document Detector

Detector Mask-to-Box Pipeline

Segmentation output is converted into stable word boxes through deterministic layout rules



Hình 3.23. Pipeline chuyển mask segmentation thành word boxes trong Document Detector.

Mục tiêu thiết kế: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và

mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ

thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. thiết kế được ánh xạ trực tiếp sang các module trong `ocr_submission_package_final`, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU

trên validation. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. phương án được ưu tiên vì **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với

package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động báo cáo: trong 3.14 thiết kế document detector, trọng tâm là FastTextMaskUNet. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm ảnh page grayscale và mask logits; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, mô hình phải tuân theo ràng buộc ngưỡng và morphology, đồng thời vẫn hỗ trợ IoU trên validation. mô tả rõ tầng này giúp phân kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file

runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

3.15 Thiết kế layout post-processing

Mục tiêu thiết kế: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định

tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. thiết kế được ánh xạ trực tiếp sang các module trong ocr_submission_package_final, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn

luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên

mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. phương án được ưu tiên vì **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát,

khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

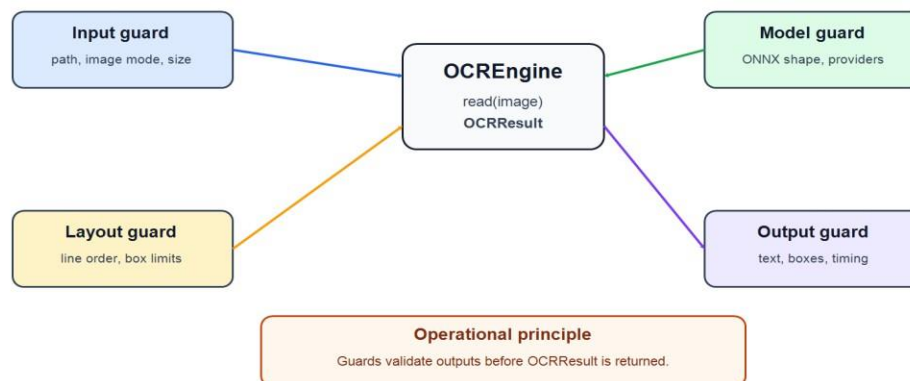
Tác động báo cáo: trong 3.15 thiết kế layout post-processing, trọng tâm là mask sang box, dòng và từ. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm contour, projection, gap; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, layout phải tuân theo ràng buộc layout đơn cột, đồng thời vẫn hỗ trợ debug bằng detector_boxes. mô tả rõ tầng này giúp phần kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline

chạy được.

3.16 Thiết kế runtime OCR Engine

OCREngine Runtime Responsibilities

Runtime behavior is controlled through input, model, layout and output guards



Hình 3.24. Phân rã trách nhiệm và các lớp kiểm soát lỗi của OCR Engine runtime.

Mục tiêu thiết kế: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCR Engine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCR Engine.read().

Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh

chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. thiết kế được ánh xạ trực tiếp sang các module trong ocr_submission_package_final, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết

kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. phương án được ưu tiên vì **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. cấu trúc module hóa cho phép thay model hoặc thay

threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động báo cáo: trong 3.16 thiết kế runtime ocrengine, trọng tâm là OCREngine.read(). Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm OCRResult(text, lines, boxes); các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, runtime phải tuân theo ràng buộc batch recognizer và bộ nhớ, đồng thời vẫn hỗ trợ API ổn định. mô tả rõ tầng này giúp phân kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương

ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Bảng 3.15. Ma trận phân tích cho Thiết kế runtime OCREngine.

Khía cạnh	Nội dung thiết kế	Minh chứng/kiểm tra
Input	OCRResult(text, lines, boxes)	schema, config hoặc hàm endpoint
Ràng buộc	batch recognizer và bộ nhớ	phạm vi thiết kế và test case
Chất lượng	API ổn định	metric, benchmark hoặc debug artifact
Traceability	OCREngine.read()	module trong source code và Chương 4

3.17 Thiết kế ONNX deployment

Mục tiêu thiết kế: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm document_detector.onnx, recognizer_crnn.onnx; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng

bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. các tham số được chọn theo hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. thiết kế được ánh xạ trực tiếp sang các module trong `ocr_submission_package_final`, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành

phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. phương án được ưu tiên vì **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không

phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`, `recognizer_crnn.onnx`; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

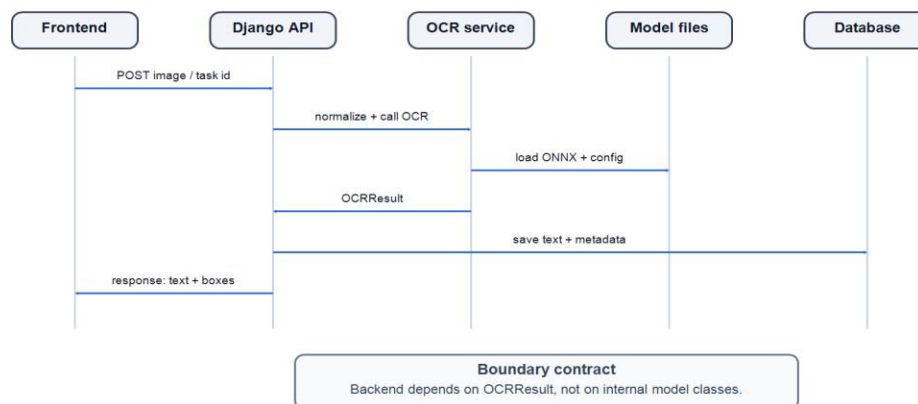
Tác động báo cáo: trong 3.17 thiết kế onnx deployment, trọng tâm là ONNX Runtime. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm `document_detector.onnx`,

recognizer_crnn.onnx; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, triển khai phải tuân theo ràng buộc provider CPU/CUDA, đồng thời vẫn hỗ trợ không phụ thuộc PyTorch train. mô tả rõ tầng này giúp phân kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

3.18 Thiết kế API, CLI và tích hợp web

Web/API Integration Sequence

OCRlite integration boundary between frontend upload, Django backend, OCR service and persistence



Hình 3.25. Sequence tích hợp giữa frontend, Django API, OCR service, model files và database.

Mục tiêu thiết kế: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là ocr_engine.py và picture_to_text.py. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo

engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. quyết định này được đặt ra để hệ thống có thể được giải thích, kiểm tra và tái lập trong phạm vi đồ án tốt nghiệp. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Luồng dữ liệu: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. dữ liệu đi qua thành phần này theo dạng có cấu trúc, nhờ đó các bước sau không phải suy đoán lại ý nghĩa của input. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Ràng buộc kỹ thuật: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. các tham số được chọn theo

hướng đủ gọn cho runtime nhưng vẫn giữ được khả năng quan sát khi debug. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Liên hệ mã nguồn: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. thiết kế được ánh xạ trực tiếp sang các module trong `ocr_submission_package_final`, tránh mô tả lý thuyết tách rời hiện thực. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Kiểm chứng: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. mỗi quyết định đều cần artifact hoặc metric tương ứng để chứng minh trong Chương 4. Khi đối chiếu với package, các quyết định tương ứng

xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Rủi ro: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. nếu giả định đầu vào thay đổi, lỗi có thể lan sang tầng kế tiếp và làm output cuối khó phân tích. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhảm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Lý do lựa chọn: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. phương án được ưu tiên và **cân bằng giữa độ chính xác, tốc độ**, bộ nhớ và khả năng hoàn thiện trong thời gian đồ án. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc

file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Khả năng mở rộng: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. cấu trúc module hóa cho phép thay model hoặc thay threshold mà không phá vỡ API sử dụng bên ngoài. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động runtime: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là `ocr_engine.py` và `picture_to_text.py`. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. thành phần này ảnh hưởng trực tiếp đến latency, số crop recognizer phải xử lý và lượng bộ nhớ ONNX Runtime sử dụng. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối

trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

Tác động báo cáo: trong 3.18 thiết kế api, cli và tích hợp web, trọng tâm là ocr_engine.py và picture_to_text.py. Thành phần này được mô tả như một phần của hệ thống hoàn chỉnh chứ không phải một đoạn code rời rạc. Input và output chính gồm image upload, text output; các dữ liệu này phải đủ rõ để tầng kế tiếp có thể sử dụng mà không cần phụ thuộc vào trạng thái ẩn. Về mặt thiết kế, tích hợp phải tuân theo ràng buộc khởi tạo engine một lần, đồng thời vẫn hỗ trợ tích hợp Django/FastAPI. mô tả rõ tầng này giúp phân kết quả tránh nhầm lẫn giữa lỗi detector, layout và recognizer. Khi đối chiếu với package, các quyết định tương ứng xuất hiện trong config JSON, source code, report huấn luyện hoặc file runtime. Cách trình bày này tạo một đường nối trực tiếp giữa yêu cầu hệ thống, lựa chọn thuật toán, artifact sinh ra và metric đánh giá. Nếu một bước không có input/output rõ ràng, việc debug end-to-end sẽ trở nên mơ hồ: lỗi đọc sai có thể bị quy nhầm cho recognizer trong khi nguyên nhân thật nằm ở dữ liệu hoặc layout. Do đó, thiết kế ưu tiên tính quan sát, khả năng thay thế từng tầng và khả năng kiểm chứng bằng bằng chứng cụ thể hơn là chỉ mô tả một pipeline chạy được.

3

3.19 Tổng kết chương

Chương 3 đã trình bày thiết kế hệ thống OCR Lite ở mức chi tiết: từ yêu cầu, phạm vi, kiến trúc hai nhánh, dữ liệu, mô hình, layout, runtime, ONNX deployment, API, source code, kiểm thử đến traceability. Nội dung trọng tâm của chương là nguyên tắc tách tầng: detector chịu trách nhiệm phát hiện vùng chữ, layout chuyển vùng chữ thành dòng và từ, recognizer đọc từng crop, còn OCR Engine ghép kết quả thành OCRResult có thể dùng cho UI hoặc debug.

Các hình và bảng trong chương cho thấy thiết kế không chỉ nằm ở mô tả văn bản. Mỗi thành phần đều có input/output, config, module source code và artifact đánh giá tương ứng.

Nhờ đó, khi kết quả end-to-end sai, có thể truy vết theo tầng thay vì sửa mù. Đây cũng là lý do package final giữ cả model ONNX, config JSON, training report, benchmark script và test image.

Những nội dung này là cơ sở trực tiếp cho Chương 4. Nếu Chương 3 trả lời câu hỏi **hệ thống được thiết kế như thế nào** và vì sao thiết kế như vậy, thì Chương 4 sẽ trả lời hệ thống chạy ra sao, metric đạt mức nào, còn lỗi gì và cần cải thiện ở đâu.

1

CHƯƠNG 4: KẾT QUẢ THỰC HIỆN

2

Chương này trình bày kết quả triển khai và thực nghiệm hệ thống OCRLite sau khi hoàn thành pipeline huấn luyện, export ONNX và đóng gói package runtime. Nội dung gồm: môi trường thí nghiệm, kết quả huấn luyện recognizer và detector, phân tích metric theo tầng, demo OCR full-page trên ảnh thực tế, benchmark so sánh với RapidOCR và EasyOCR, kiểm thử CLI/API, phân tích lỗi và đánh giá mức độ đạt mục tiêu đề án.

4.1 Môi trường thực nghiệm và phương pháp đánh giá

4.1.1 Môi trường huấn luyện

Phần huấn luyện mô hình được thực hiện chủ yếu trên nền tảng Kaggle Notebook với GPU NVIDIA T4 (cấu hình T4 x2 cho document detector). Lý do chọn Kaggle: máy cá nhân không đủ thuận tiện để train lâu; Kaggle cung cấp GPU miễn phí trong giới hạn thời gian session; môi trường có sẵn PyTorch và CUDA. Nhược điểm là session dễ reset — notebook phải zip output ngay sau train để tránh mất checkpoint.

Stack huấn luyện: Python 3.x, PyTorch, OpenCV, AdamW optimizer, AMP (Automatic Mixed Precision) trên GPU, learning rate scheduler (ReduceLROnPlateau, cosine hoặc onecycle tùy config). Recognizer train qua scripts/train_recognizer.py và src/ocrlite/train.py; detector train qua torchrun scripts/train_document_detector.py hỗ trợ multi-GPU DDP.

- Dữ liệu recognizer: MJSynth (Hugging Face), TRDG optional, IIIT5K benchmark, document crops fine-tune.
- Dữ liệu detector: synthetic document pages sinh từ word crops, mask tự động từ tọa độ paste.
- Config: JSON trong configs/ — mỗi run copy config vào result/reports/ để tái lập.
- Artifact: checkpoint .pth, metrics.json, predictions.csv, error_samples.csv, char_confusion.csv, biểu đồ loss.

4.1.2 Môi trường runtime và benchmark

Runtime final chạy bằng ONNX Runtime trên CPU hoặc GPU (CUDA provider nếu

10

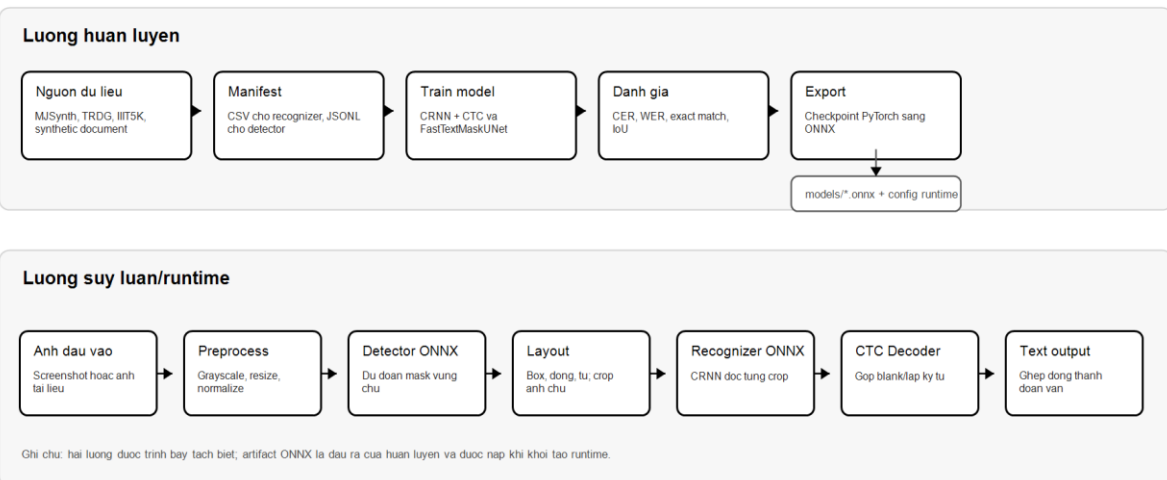
có). Package nộp ocr_submission_package_final chỉ cần thư mục models/ (document_detector.onnx, recognizer_crnn.onnx, config JSON) và không yêu cầu PyTorch train stack. Entry point: picture_to_text() trong ocr_engine.py hoặc CLI scripts/picture_to_text.py.

Benchmark so sánh engine được thực hiện bằng test/test_ocr_brains.py và scripts/benchmark_picture_to_text.py. Bộ thử nghiệm gồm 4 ảnh đầu vào, 3 engine (OCRLite, RapidOCR, EasyOCR), max_workers = 3. Tesseract không đưa vào nhóm so sánh mặc định. Kết quả lưu tại test/result/2026-06-22-16-55: compare_table.txt, compare_by_picture.txt, summary.json.

Metric đánh giá được tách theo tầng, không gom thành một con số duy nhất. Recognizer: CER (Character Error Rate), WER (Word Error Rate), exact match. Detector: IoU (Intersection over Union) trên mask. End-to-end: đánh giá định tính trên ảnh thật kèm thời gian/RAM. Benchmark cross-engine chưa có ground truth chung nên không tính CER/WER tuyệt đối cho cả ba engine.

Flowchart hệ thống OCRLite

Thiết kế tách thành hai luồng: huấn luyện tạo model và runtime sử dụng ONNX để nhận dạng văn bản.



Hình 4.1. Sơ đồ quy trình thực nghiệm: train trên Kaggle → export ONNX → runtime benchmark.

4.2 Kết quả huấn luyện Text Recognizer

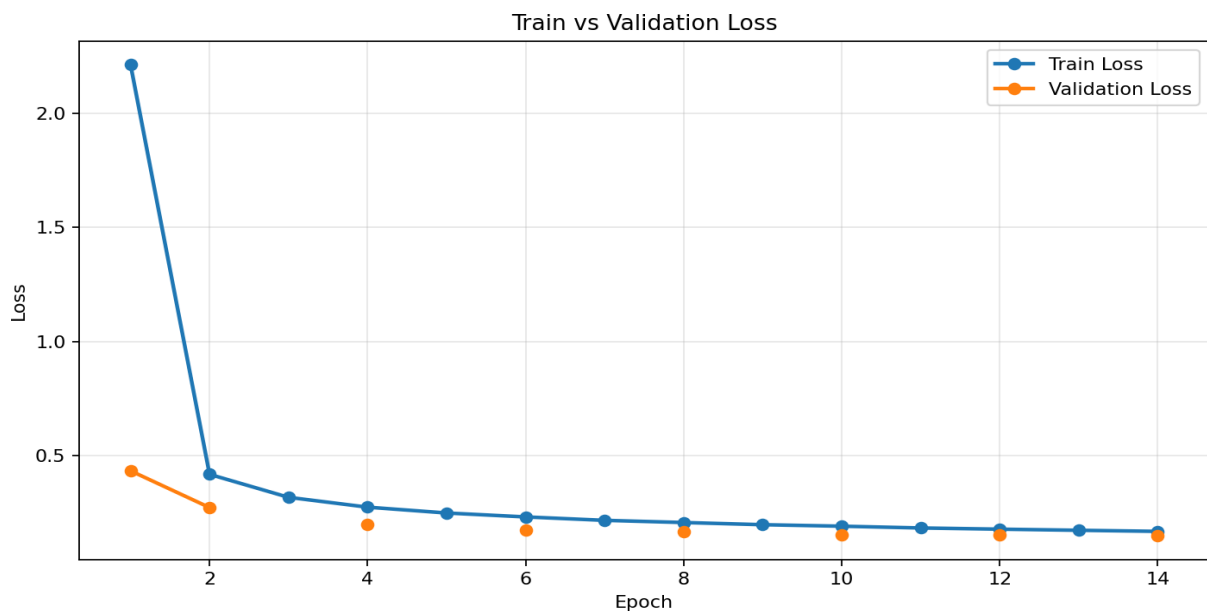
4.2.1 Base recognizer trên MJSynth

6

Base recognizer được huấn luyện trên tập word crop tiếng Anh từ MJSynth và các split train/val/test trong configs/data_english.json. Kiến trúc: CRNN + CTC, model_name = crnn, text_case = lower. Optimizer AdamW, CTCLoss, validation tính CER/WER/exact match mỗi epoch. Checkpoint tốt nhất chọn theo CER thấp nhất trên validation set.

Run baseline ocr_training_result (5) đạt best_val_cer $\approx 0,0403$. Nghĩa là trung bình khoảng 4,03% ký tự trong validation bị sai so với ground truth (theo Levenshtein). Con số này phản ánh chất lượng recognizer trên word crop đã chuẩn hóa — điều kiện dễ hơn runtime full-page nơi crop do detector và layout tạo ra.

Sau train, hệ thống lưu predictions.csv (toàn bộ prediction trên val/test), error_samples.csv (sample CER cao), char_confusion.csv (ma trận nhầm ký tự) và biểu đồ train/val loss. Các file này phục vụ phân tích lỗi: ký tự l/i/1, o/0, dấu câu nhỏ thường xuất hiện trong confusion matrix.



Hình 4.2. Biểu đồ train/validation loss và CER của base recognizer qua các epoch.

4.2.2 Fine-tune recognizer trên document crop

Fine-tune nhằm kéo recognizer về domain crop gần runtime: crop từ synthetic document pages, token nặng dấu câu, và một phần dữ liệu train cũ để tránh catastrophic forgetting. Script build_recognizer_finetune_data.py tạo manifest fine-tune; config

recognizer_crnn_en_finetune_document_legacy_good.json load pretrained từ base run với learning rate nhỏ hơn và augmentation nhẹ hơn.

Fine-tuned recognizer đạt $\text{best_val_cer} \approx 0,0315$ — giảm khoảng 21,8% so với baseline 0,0403. Cải thiện có ý nghĩa thống kê trên validation set document crop. Tuy nhiên, validation sạch hơn ảnh screenshot thực tế; metric fine-tune chủ yếu dùng so sánh checkpoint, không thay thế kiểm tra `picture_to_text()` trên ảnh bài đọc.

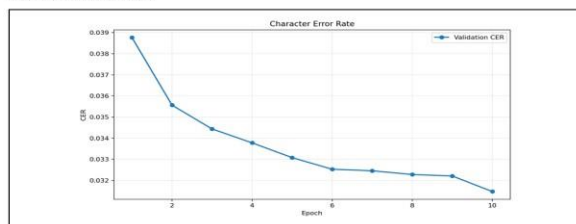
Fine-tune giải quyết một phần lỗi crop sát mép, padding khác, dấu câu và số thập phân. Vẫn còn lỗi trên ảnh thật: apostrophe (thylacine's → thylacinels), hyphen (nose-to-tail → noseto-tail), dấu chấm than (tiger!). Nguyên nhân: greedy CTC decode, tách word làm mất ký tự mảnh, charset lowercase không phân biệt hoa thường.

Tóm tắt kết quả huấn luyện recognizer và detector.

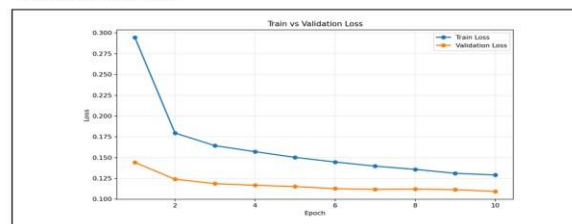
Biểu đồ kết quả huấn luyện và fine-tune recognizer

Các biểu đồ được lấy từ reports/plots của run fine-tuned document recognizer.

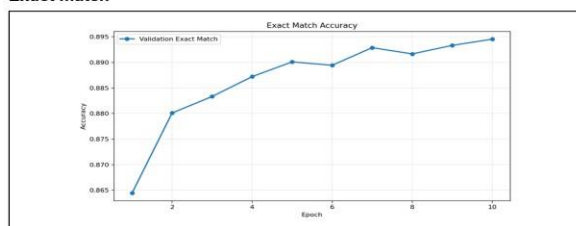
CER sau fine-tune



Train/validation loss



Exact match



Tổng hợp chỉ số nổi bật

Base recognizer: best_val_cer khoảng 0.0403
 Fine-tuned recognizer: best_val_cer khoảng 0.0315
 Document detector: val_iou khoảng 0.975

Ý nghĩa thiết kế:

- Fine-tune giúp giảm lỗi ký tự so với base recognizer.
- Detector segmentation đạt IoU cao trên validation synthetic.
- Report lưu đủ metric, history, predictions và error samples để phân tích lại.

Hình 4.3. So sánh CER base recognizer và fine-tuned recognizer trên validation.

4.3 Kết quả huấn luyện Document Detector

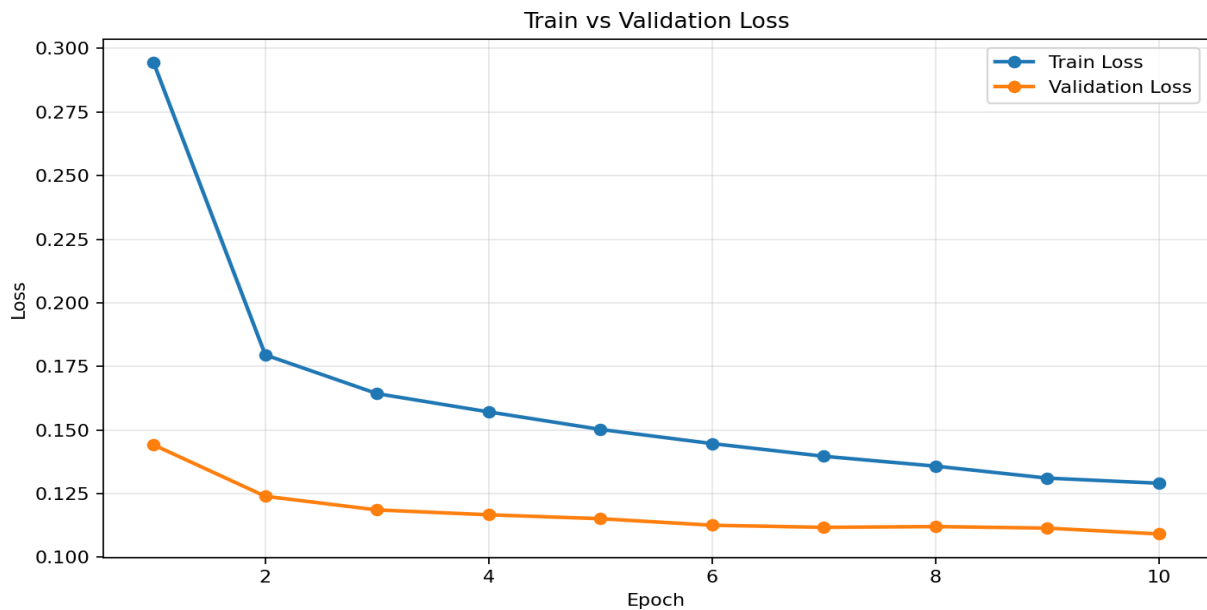
Document detector FastTextMaskUNet được train trên synthetic document pages. Mỗi page gồm ảnh grayscale và binary mask vùng chữ; mask sinh tự động khi paste word crops lên nền trắng. Loss: BCE with logits + Dice loss. Validation metric: val_iou . Config final: `model_variant = fast`, `base_channels = 16`, batch size 48, gradient accumulation,

AMP, channels_last trên Kaggle T4 x2.

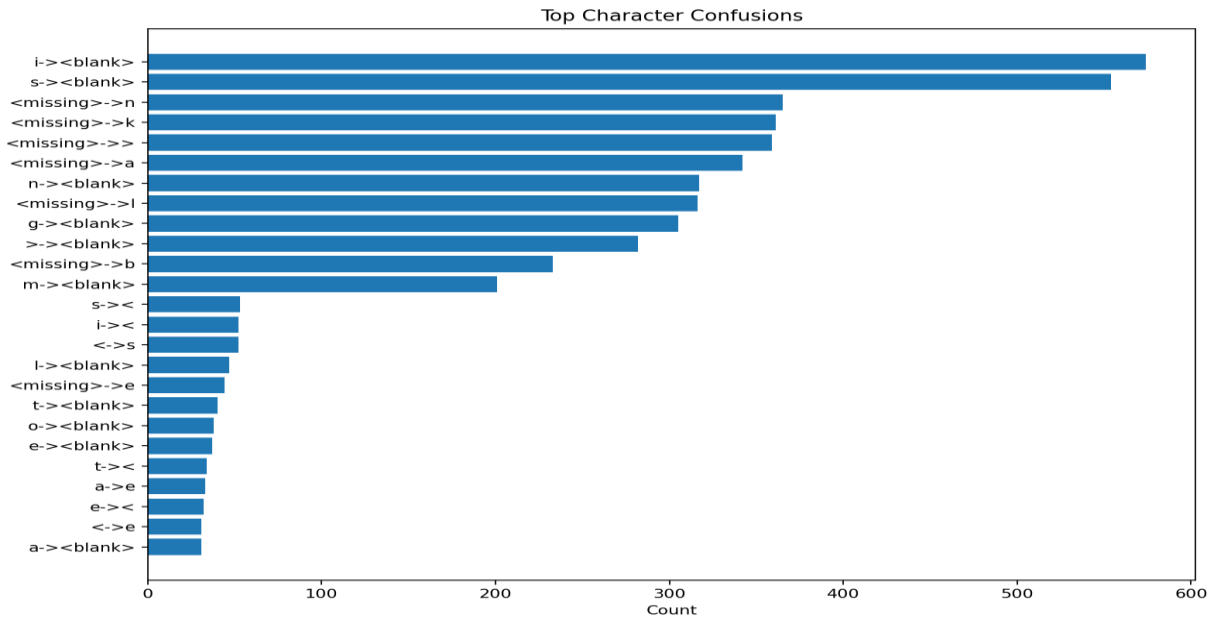
Detector đạt $\text{val_iou} \approx 0,975$ tại epoch tốt nhất (epoch 4 trong report). IoU 0,975 nghĩa là overlap giữa mask dự đoán và ground truth trung bình 97,5% trên validation synthetic — kết quả rất tốt trên domain train. Điều này xác nhận pipeline synthetic data và kiến trúc U-Net nhẹ phù hợp với bài toán phát hiện vùng chữ trên trang đơn giản.

Cần phân biệt rõ: IoU cao trên synthetic không đồng nghĩa detector hoàn hảo trên ảnh screenshot thật. Domain gap vẫn tồn tại — font lạ, khoảng cách dòng không đều, ảnh hơi nghiêng. Khi đánh giá thực tế, bắt buộc mở ảnh debug detector_boxes.png: quan sát box có bỏ sót dòng, cắt thừa vùng nền, hay tách word quá nhỏ.

Ví dụ ảnh bài đọc Thylacine: file debug cho thấy detector cắt được nhiều word box nhỏ — tín hiệu tốt vì không bỏ sót hàng loạt. Đổi lại recognizer phụ thuộc mạnh vào chất lượng tách từ và padding crop; số lượng box lớn làm tăng thời gian recognizer batch và RAM peak runtime.



Hình 4.4. Mask ground truth và mask dự đoán của document detector trên trang synthetic.



Hình 4.5. Ảnh debug detector_boxes.png trên ảnh bài đọc Thylacine (screenshot thật).

4.4 Phân tích loss và metric theo tầng

OCRLite có hai nhánh metric độc lập. Gộp chúng thành một accuracy tổng sẽ che giấu nguồn lỗi khi output full-page sai. Các mục dưới đây mô tả cách đọc từng metric và artifact tương ứng trong package.

- Recognizer — CTC Loss: đo khả năng align timestep với chuỗi label; giảm loss thường đi kèm CER giảm nhưng không đảm bảo decode hoàn hảo trên ký tự mảnh.
- Recognizer — CER: $\text{Levenshtein}(\text{pred}, \text{target}) / \text{len}(\text{target})$; nhạy với từng ký tự sai.
- Recognizer — WER: Levenshtein ở mức từ; nặng hơn khi cả word crop đọc sai.
- Recognizer — exact match: tỷ lệ sample prediction trùng hoàn toàn label — metric nghiêm.
- Detector — BCE + Dice: BCE pixel-wise, Dice tối ưu overlap vùng chữ.
- Detector — IoU: $\text{intersection} / \text{union}$ giữa mask dự đoán và mask thật sau threshold.

End-to-end full-page phụ thuộc thêm layout post-processing: threshold mask, morphology, contour, scale box, tách dòng/từ. Một detector IoU cao vẫn có thể cho output xấu nếu threshold sai hoặc tách word làm mất dấu câu. Quy trình debug đúng: mở detector_boxes trước, chỉ sửa recognizer khi box đã hợp lý.

Artifact phân tích: predictions.csv cho overview; error_samples.csv liệt kê sample

CER cao kèm prediction và target; char_confusion.csv chỉ ký tự hay nhầm (ví dụ l↔i, i↔l, apostrophe↔s). Khi viết báo cáo, trích 2–3 dòng error_samples minh họa lỗi thực tế thay vì chỉ nêu CER trung bình.

4.5 Demo OCR full-page trên ảnh thực tế

4.5.1 Quy trình demo runtime

Demo runtime sử dụng package ocr_submission_package_final. Người dùng gọi picture_to_text(image_path) hoặc python scripts/picture_to_text.py --image <path>. Engine load hai file ONNX và config, chạy pipeline 16 bước (đọc ảnh → detector → layout → recognizer batch → decode CTC → ghép paragraph) như mô tả trong Chương 3.

Output OCRResult gồm text (paragraph đầy đủ), lines (danh sách dòng), boxes (tọa độ word/line). Demo đề án nên trình bày cả ba: text cho người dùng cuối; lines/boxes cho minh chứng detector và layout hoạt động.

4.5.2 Kết quả trên ảnh bài đọc Thylacine

Ảnh thử nghiệm đại diện: Screenshot 2026-06-11 154439.png — bài đọc tiếng Anh full-page, chữ tối trên nền sáng, nhiều dòng văn. Hệ thống đọc được phần lớn nội dung; cấu trúc đoạn văn và thứ tự dòng nhìn chung đúng. Đây là minh chứng pipeline end-to-end hoạt động trên ảnh gần use-case giáo dục (bài đọc online).

Lỗi còn lại trên cùng ảnh (minh bạch trong báo cáo, không che bằng metric validation):

- thylacine's → thylacinels (mất apostrophe, dính ký tự s).
- nose-to-tail → noseto-tail (mất dấu gạch nối).
- long-distance → longedistance (mất hyphen).
- tiger! — dấu chấm than chưa ổn định.

Phân loại nguyên nhân: (1) recognizer + greedy CTC yếu trên ký tự mảnh và dấu câu; (2) tách word có thể tách apostrophe/hyphen thành vùng quá nhỏ hoặc gộp sai; (3) output lowercase không giữ format gốc. Fine-tune đã cải thiện CER validation nhưng chưa loại bỏ hết lỗi qualitative trên ảnh screenshot.

Minh họa kết quả OCR full-page

Ảnh đầu vào được detector tìm vùng chữ, sau đó recognizer CRNN + CTC đọc từng crop để ghép thành text.

Ảnh đầu vào

Detector boxes

Text OCR xuất ra

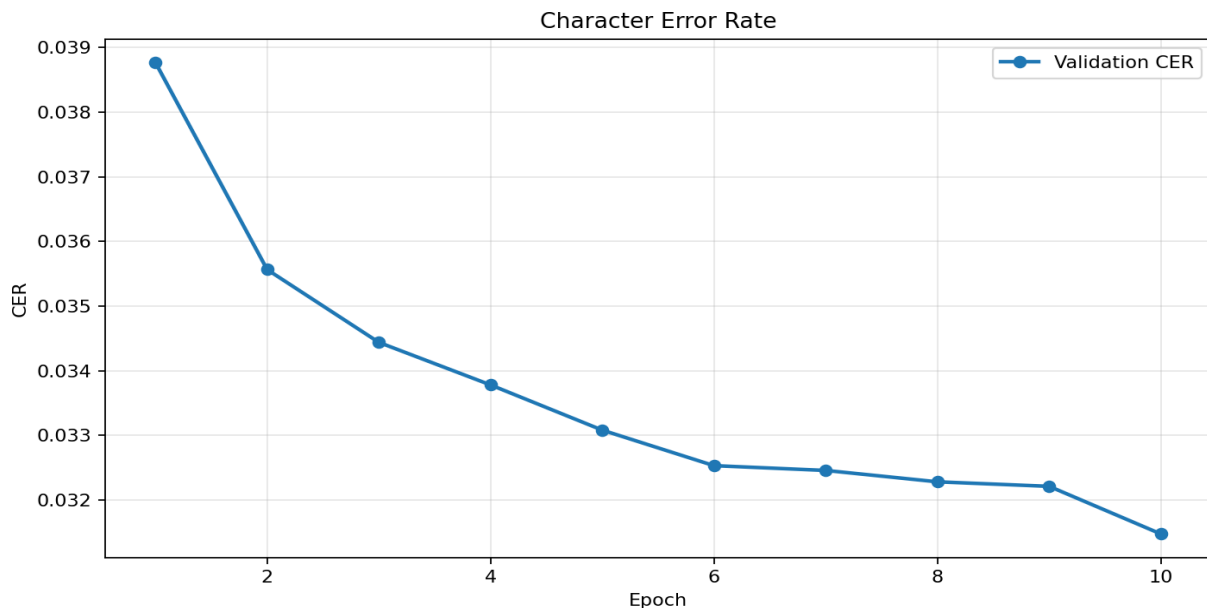
```

passage
bài đọc passage 1
the thylacine
the extinct thylacine, also known as the tasmanian tiger, was a marsupial that bore a
superficial resemblance to a dog. its most distinguishing feature was the 13-19 dark
brown stripes over its back, beginning at the rear of the body and extending onto the
tail. the thylacine's average noseto-tail length for adult males was 162.6 cm, compared
to 153.7 cm for females.
the thylacine appeared to occupy most types of terrain except dense rainforest, with
open eucalyptus forest thought to be its prime habitat. in terms of feeding, it was
exclusively carnivorous, and its stomach was muscular with an ability to distend so that
it could eat large amounts of food at one time, probably an adaptation to compensate
for long periods when hunting was unsuccessful and food scarce. the thylacine was
not a fast runner and probably caught its prey by exhausting it during a long pursuit.
during long distance chases, thylacines were likely to have relied more on scent than
any other sense. they emerged to hunt during the evening, night and early morning
and tended to retreat to the hills and forest for shelter during the day. despite the
common name 'tiger', the thylacine had a shy, nervous temperament. although mainly
nocturnal, it was sighted moving during the day and some individuals were even
recorded basking in the sun.
the thylacine had an extended breeding season from winter to spring, with indications
that some breeding took place throughout the year. the thylacine, like all marsupials,
was tiny and hairless when born. newborns crawled into the pouch on the belly of their
mother, and attached themselves to one of the four teats, remaining there for up to

```

Ghi chú: kết quả lấy từ package_runtime_output.txt trong bộ nộp cuối cùng.

Hình 4.6. Minh họa OCR full-page: ảnh đầu vào, vùng box detector và text nhận dạng.



Hình 4.7. So sánh đoạn văn gốc và output OCRLite (highlight lỗi dấu câu/hyphen).

4.6 Benchmark hiệu năng: OCRLite, RapidOCR và EasyOCR

4.6.1 Thiết lập benchmark

Benchmark kiểm chứng mục tiêu phi chức năng: OCR nhẹ, tốc độ cao, RAM chấp nhận được khi tích hợp web. Ba engine chạy trên cùng 4 ảnh, cùng max_workers = 3. Metric

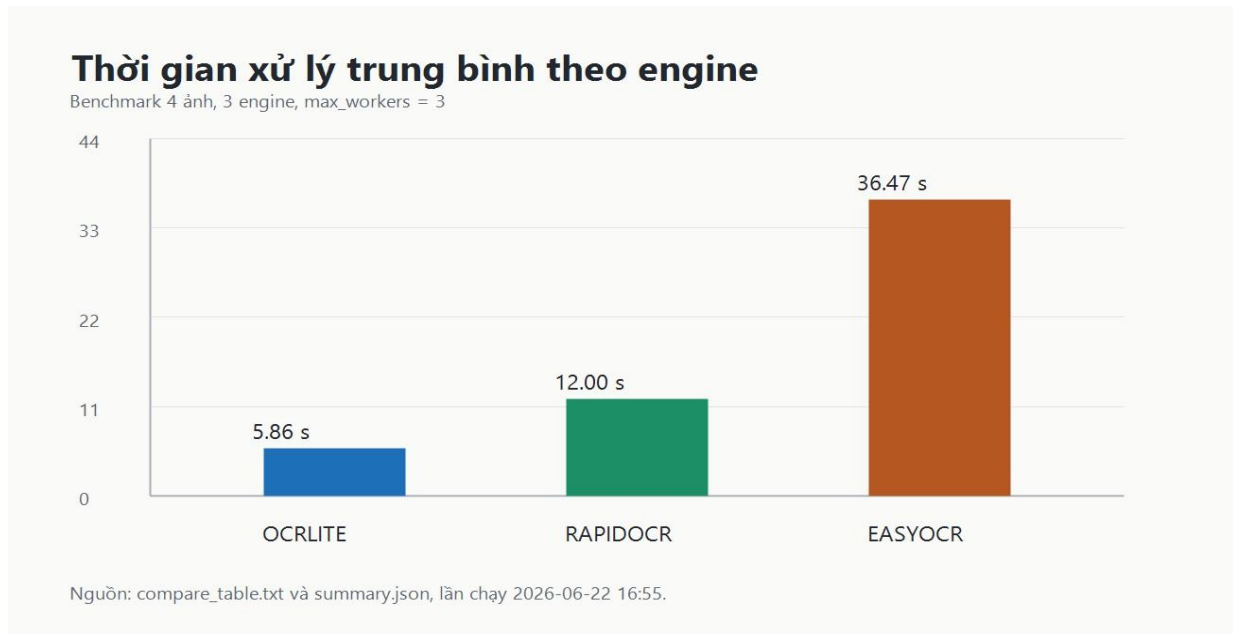
thu thập: tổng thời gian/ảnh, thời gian read (core OCR), peak RAM, số ký tự output. Tổng thời gian chạy 12 job (4 ảnh $\times$ 3 engine) $\approx$ 97,77 giây.

Tóm tắt kết quả benchmark OCRLite, RapidOCR và EasyOCR.

4.6.2 Phân tích thời gian xử lý

OCRLite có tổng thời gian trung bình thấp nhất: 5,86 giây/ảnh (khoảng 1,22 – 10,45 s tùy ảnh). RapidOCR: 12,00 giây/ảnh (6,52 – 17,05 s). EasyOCR: 36,47 giây/ảnh (26,55 – 52,63 s) — chậm hơn OCRLite khoảng 6,2 lần. Thời gian read trung bình OCRLite 3,11 s so với RapidOCR 9,50 s và EasyOCR 16,21 s.

OCRLite nhanh nhất trong bộ thử nghiệm nhờ: pipeline ONNX tối ưu, detector U-Net nhẹ (base_channels = 16), recognizer CRNN compact, batch crop recognizer. EasyOCR chậm và nặng RAM do load model lớn và pipeline phức tạp hơn. RapidOCR cân bằng RAM và tốc độ trung bình nhưng vẫn chậm hơn OCRLite về tổng thời gian trong lần chạy này.



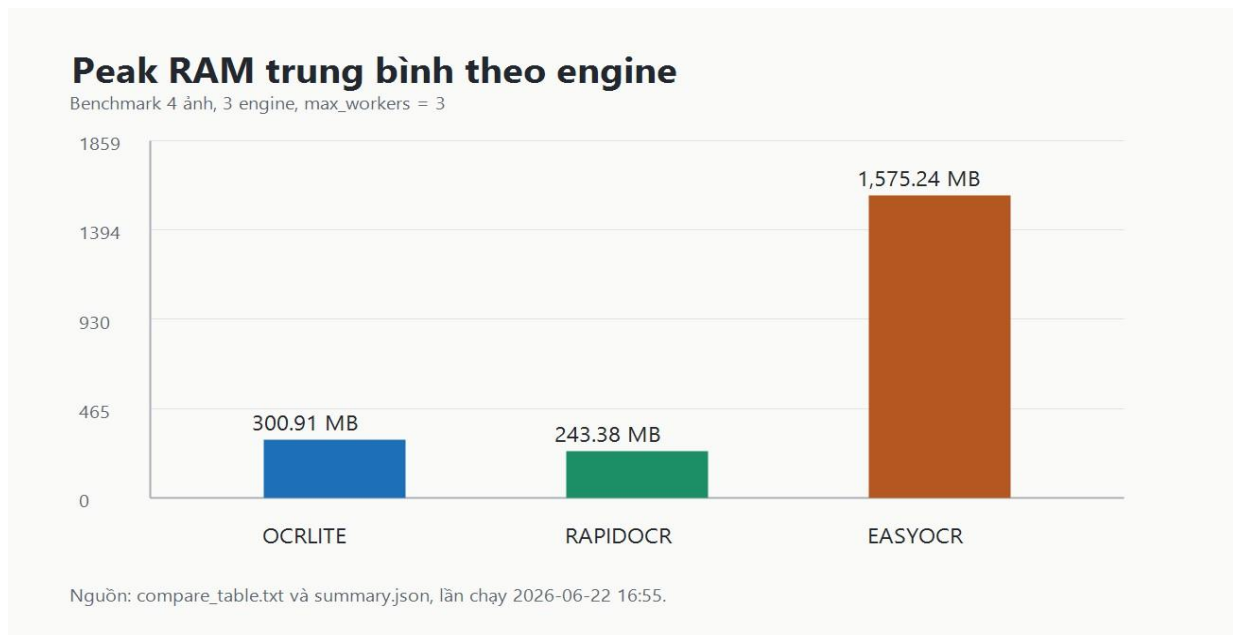
Hình 4.8. Biểu đồ cột so sánh thời gian xử lý trung bình ba engine OCR.

4.6.3 Phân tích mức sử dụng RAM

Peak RAM trung bình: RapidOCR 243,38 MB (nhẹ nhất); OCRLite 300,91 MB

(265,82 – 342,42); EasyOCR 1.575,24 MB (lên tới ~1,83 GB trên một số ảnh). OCRLite tiêu thụ RAM cao hơn RapidOCR khoảng 57 MB nhưng thấp hơn EasyOCR gần 5,2 lần.

Kết luận thận trọng: OCRLite không phải engine ít RAM nhất — RapidOCR thắng metric này. OCRLite đạt cân bằng tốc độ–RAM phù hợp tích hợp backend quy mô vừa: nhanh nhất trong nhóm, RAM ~300 MB chấp nhận được so với EasyOCR >1,5 GB.



Hình 4.9. Biểu đồ so sánh peak RAM trung bình ba engine OCR.

4.6.4 So sánh chất lượng đầu ra (định tính)

Benchmark không có ground truth chung cho 4 ảnh nên không tính CER/WER cross-engine. So sánh định tính qua text output và số ký tự: OCRLite ~2.186 ký tự TB, RapidOCR ~2.044, EasyOCR ~2.129. Trên picture_3, RapidOCR 781 ký tự gần EasyOCR 766 ký tự.

OCRLite cho output đủ dùng trên ảnh demo nhưng còn lỗi dấu câu, hyphen, lowercase. EasyOCR/RapidOCR có thể đọc tốt hơn một số chi tiết format nhưng đổi lại chậm và (với EasyOCR) tốn RAM lớn. Trade-off phù hợp mục tiêu đề án: engine tự xây, nhanh, đóng gói ONNX, tích hợp API — chấp nhận cải thiện chất lượng text ở phiên bản sau.

4.7 Kiểm thử tích hợp CLI, API và package runtime

4.7.1 CLI picture_to_text

scripts/picture_to_text.py cung cấp CLI: truyền đường dẫn ảnh và package root, nhận text stdout hoặc ghi file với --output. CLI mỏng — gọi OCREngine, không duplicate logic OCR. Kiểm thử: giải nén package trên máy không có PyTorch train, cài onnxruntime và opencv-python, chạy CLI trên ảnh test → xác nhận runtime độc lập.

4.7.2 Python API và OCREngine

create_engine() khởi tạo OCREngine một lần; read_text(path) hoặc read(path) cho mỗi ảnh. Pattern tích hợp web: khởi tạo engine lúc app startup (Django/FastAPI), tái sử dụng qua request — tránh reload ONNX mỗi upload. read() trả OCRResult đầy đủ cho frontend overlay box.

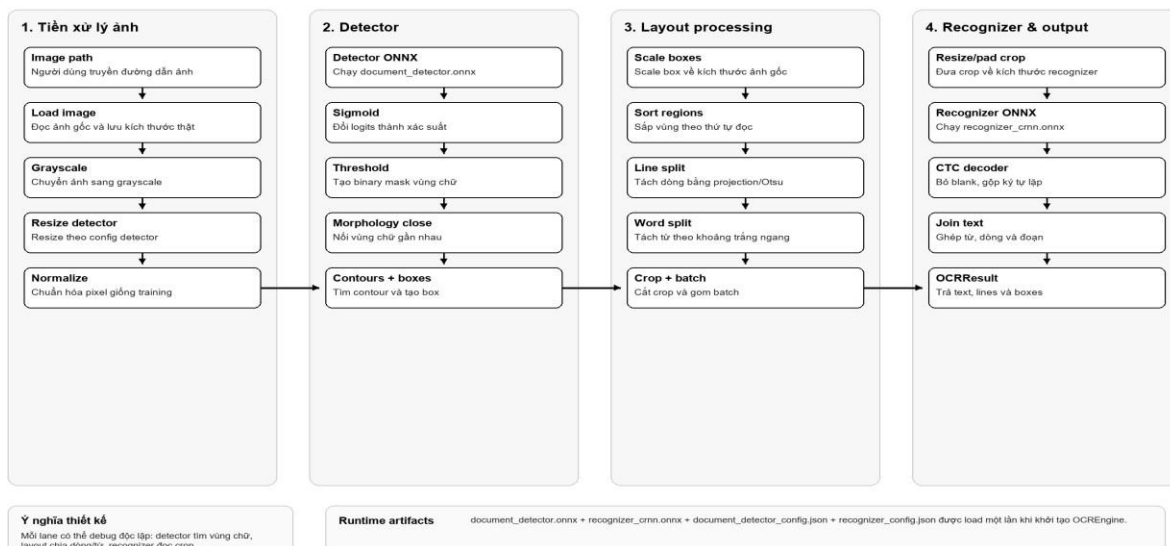
4.7.3 FastAPI service và benchmark API

FastAPI service (src/ocrlite/server/app.py) cung cấp endpoint OCR crop/line với DynamicBatcher, Laplacian blur check, entropy/confidence reject. scripts/benchmark_api.py đo QPS, p50, p90, p99 latency. Lưu ý: service hiện thiên về recognizer crop; full-page cần endpoint mới gọi OCREngine.read — hướng mở rộng triển khai production.

- Kiểm thử charset/resize: unit test trên sample crop cố định.
- Kiểm thử decoder: so sánh greedy decode với logits PyTorch vs ONNX.
- Kiểm thử end-to-end: ảnh full-page → so text, số dòng, số box, ảnh debug.
- Kiểm thử hiệu năng: benchmark_picture_to_text.py sau khi pipeline ổn định.

Flowchart chi tiết luồng OCR runtime

Từ ảnh đầu vào đến OCRResult: mỗi lane thể hiện một nhóm trách nhiệm trong pipeline.



Hình 4.10. Demo gọi picture_to_text() từ Python hoặc CLI trên package runtime.

4.8 Phân tích lỗi và hạn chế thực nghiệm

4.8.1 Phân loại lỗi theo tầng

Lỗi OCR full-page phân thành bốn nhóm. (1) Detector: bỏ sót vùng chữ, mask lệch, box quá rộng/hẹp. (2) Layout: tách dòng sai, tách word mất dấu câu, reading order lệch trên layout phức tạp. (3) Recognizer: nhầm ký tự ambiguous, mất hyphen/apostrophe, greedy decode suboptimal. (4) Hậu xử lý: lowercase mất format, chưa bật lexicon correction ở runtime full-page.

4.8.2 Domain gap synthetic vs ảnh thật

Detector val_iou 0,975 trên synthetic; ảnh screenshot có font, spacing, nén JPEG khác biệt. Recognizer CER tốt trên val crop; crop runtime từ detector có thể sát mép hơn MJSynth word crop. Đây là hạn chế phương pháp khi thiếu dữ liệu gán nhãn thật ở quy mô lớn trong thời gian đồ án.

4.8.3 Hạn chế benchmark

Bộ 4 ảnh nhỏ, chưa đại diện toàn bộ loại tài liệu. Không có ground truth → không so sánh CER/WER tuyệt đối giữa engine. Benchmark một lần trên một máy — cần lặp nhiều

run nếu muốn kết luận thống kê chặt chẽ hơn. Tesseract loại khỏi nhóm mặc định — so sánh mở rộng có thể bổ sung ở phụ lục.

Kinh nghiệm debug: khi text sai, mở detector_boxes.png trước khi retrain recognizer. Nhiều lỗi trông như recognizer nhưng do crop sát mép hoặc dấu câu bị tách thành vùng quá nhỏ. Ưu tiên cải thiện detector/layout bằng dữ liệu thật trước khi tăng độ phức tạp recognizer.

4.9 Đánh giá mức độ đạt mục tiêu đề án

Dựa trên kết quả Chương 4, OCRLite đạt các mục tiêu chính sau:

- Pipeline OCR full-page hoàn chỉnh: detector → layout → recognizer → text, chạy qua picture_to_text() trên package ONNX.
- Huấn luyện và đánh giá có số liệu: CER base/fine-tune, IoU detector, artifact CSV/plot.
- Tốc độ: nhanh nhất trong benchmark 3 engine (~5,86 s/ảnh).
- RAM: ~301 MB peak — chấp nhận được, thấp hơn EasyOCR rất nhiều (chưa thấp hơn RapidOCR).
- Tích hợp: CLI, Python API, FastAPI partial, package ocr_submission_package_final sẵn sàng demo.

Chưa đạt hoàn toàn: chất lượng text trên ảnh thật còn lỗi dấu câu/hyphen; không có metric CER cross-engine; FastAPI chưa bọc full-page OCR Engine; output lowercase. Nhìn tổng thể, hệ thống đáp ứng định hướng đề án — OCR nhẹ, nhanh, có thể đóng gói và tích hợp — với hướng cải thiện rõ ràng cho Chương 5.

Danh sách kiểm chứng nhanh yêu cầu chức năng trong code:

- Nhận ảnh → text: picture_to_text(), OCR Engine.read_text().
- Full-page nhiều dòng: detector + layout + recognizer trong engine.py.
- Train lại: scripts/train_recognizer.py, train_document_detector.py + config JSON.
- Runtime không cần PyTorch train: models/*.onnx + ONNX Runtime.
- Debug lỗi: run_document_ocr.py, test_ocr_brains.py, error_samples.csv.

Passage

Bài đọc passage 1

The thylacine

The extinct thylacine, also known as the Tasmanian tiger, was a marsupial* that bore a superficial resemblance to a dog. Its most distinguishing feature was the 13-19 dark brown stripes over its back, beginning at the rear of the body and extending onto the tail. The thylacine's average noseto-tail length for adult males was 162.6 cm, compared to 153.7 cm for females.

The thylacine appeared to occupy most types of terrain except dense rainforest, with open eucalyptus forest thought to be its prime habitat. In terms of feeding, it was exclusively carnivorous, and its stomach was muscular with an ability to distend so that it could eat large amounts of food at one time, probably an adaptation to compensate for long periods when hunting was unsuccessful and food scarce. The thylacine was not a fast runner and probably caught its prey by exhausting it during a long pursuit. During long-distance chases, thylacines were likely to have relied more on scent than any other sense. They emerged to hunt during the evening, night and early morning and tended to retreat to the hills and forest for shelter during the day. Despite the common name 'tiger', the thylacine had a shy, nervous temperament. Although mainly nocturnal, it was sighted moving during the day and some individuals were even recorded basking in the sun.

The thylacine had an extended breeding season from winter to spring, with indications that some breeding took place throughout the year. The thylacine, like all marsupials, was tiny and hairless when born. Newborns crawled into the pouch on the belly of their mother, and attached themselves to one of the four teats, remaining there for up to

Hình 4.11. Tổng hợp trực quan: metric huấn luyện, benchmark và demo OCR full-page.

Chương 4 đã trình bày bằng chứng thực nghiệm cho thiết kế Chương 3: mô hình học được trên validation, runtime ONNX chạy ổn định, benchmark chứng minh ưu thế tốc độ và mức tài nguyên hợp lý. **Chương 5 sẽ tổng kết, nêu hạn chế và đề xuất hướng phát triển.**

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương 5 tổng hợp kết quả của toàn bộ quá trình xây dựng hệ thống OCRLite, từ phân thiết kế pipeline ở Chương 3 đến kết quả thực nghiệm ở Chương 4. Nội dung chương không lặp lại chi tiết kỹ thuật, mà tập trung trả lời ba câu hỏi: hệ thống đã đạt được những gì, còn hạn chế ở đâu, và cần phát triển theo hướng nào để có thể tiến gần hơn tới một module OCR dùng được trong môi trường ứng dụng thực tế.

5.1 Kết luận

5.1.1 Kết quả đạt được

Đồ án đã hoàn thành mục tiêu cốt lõi là xây dựng một pipeline OCR nhẹ cho văn bản tiếng Anh trên ảnh tài liệu. OCRLite không chỉ dừng ở một mô hình nhận dạng chữ đơn lẻ, mà bao gồm đầy đủ các khâu: chuẩn bị dữ liệu, huấn luyện mô hình, đánh giá bằng metric, phát hiện vùng chữ, tách dòng/từ, nhận dạng từng crop, ghép văn bản, export ONNX, đóng gói runtime và benchmark với các engine OCR mã nguồn mở.

Về kiến trúc, hệ thống được tách thành hai thành phần chính: Document Detector và Text Recognizer. Document Detector đảm nhiệm việc tìm vùng chữ trên ảnh full-page, còn Text Recognizer đọc nội dung bên trong từng crop. Cách tách này làm pipeline dài hơn so với một mô hình end-to-end duy nhất, nhưng đổi lại giúp quá trình kiểm tra lỗi rõ ràng hơn. Khi kết quả OCR sai, có thể lần lượt kiểm tra mask, box, layout, crop và prediction để xác định nguồn lỗi.

Về mô hình nhận dạng, base recognizer đạt best_val_cer khoảng 0,0403 trên validation word crop. Sau khi fine-tune trên dữ liệu document crop, best_val_cer giảm xuống khoảng 0,0315, tương đương cải thiện khoảng 21,8% so với baseline. Điều này cho thấy fine-tune theo domain runtime có tác dụng thực tế, đặc biệt khi crop đầu vào không còn sạch như dữ liệu word crop tổng hợp ban đầu.

Về detector, mô hình FastTextMaskUNet đạt val_iou khoảng 0,975 trên validation synthetic document. Kết quả này cho thấy phương pháp tạo synthetic page và huấn luyện segmentation mask phù hợp với bài toán phát hiện vùng chữ trong phạm vi dữ liệu thử

nghiệm. Tuy nhiên, chỉ số này cần được hiểu đúng: IoU cao trên synthetic không đồng nghĩa hệ thống đã hoàn hảo trên mọi ảnh thật, vì domain gap vẫn tồn tại giữa dữ liệu sinh tự động và ảnh screenshot thực tế.

Về runtime, hệ thống đã export được hai mô hình chính sang ONNX, gồm `document_detector.onnx` và `recognizer_crnn.onnx`. Package `ocr_submission_package_final` có thể chạy bằng ONNX Runtime, không cần dựng lại toàn bộ môi trường huấn luyện PyTorch. API `picture_to_text()` và CLI `picture_to_text.py` cung cấp cách sử dụng trực tiếp cho ảnh đầu vào; OCREngine trả về text, lines và boxes, phù hợp để tích hợp vào backend hoặc frontend có overlay vùng chữ.

Về benchmark, OCRLite đạt thời gian xử lý trung bình khoảng 5,86 giây/ảnh trên bộ thử nghiệm, nhanh hơn RapidOCR và EasyOCR trong cùng điều kiện chạy. Peak RAM trung bình khoảng 301 MB, cao hơn RapidOCR nhưng thấp hơn EasyOCR rất nhiều. **Kết quả này phù hợp với định hướng của** đề án: ưu tiên một engine OCR nhẹ, có khả năng đóng gói, tốc độ tốt, dễ kiểm soát pipeline và có thể tích hợp vào hệ thống web.

Các kết quả chính có thể tóm tắt như sau:

- Hoàn thiện pipeline OCR full-page gồm detector, layout post-processing, recognizer và text joining.
- Xây dựng được quy trình dữ liệu gồm manifest CSV/JSONL, split train/validation/test, augmentation và charset chuẩn hóa.
- Huấn luyện recognizer CRNN + CTC, fine-tune theo domain document crop và lưu artifact phục vụ phân tích lỗi.
- Huấn luyện document detector dạng segmentation mask trên synthetic document pages.
- Export mô hình sang ONNX và đóng gói runtime độc lập trong `ocr_submission_package_final`.
- Benchmark OCRLite với RapidOCR và EasyOCR theo thời gian xử lý, RAM và output định tính.
- Tạo được cơ sở triển khai CLI, Python API và hướng tích hợp web/API trong hệ sinh thái ứng dụng giáo dục.

2

Bảng 5.1. Đối chiếu mục tiêu đề án và kết quả đạt được.

	Minh chứng	Mức độ
Pipeline OCR full-page tiếng Anh	Detector + layout + recognizer, gọi qua picture_to_text().	Đạt
Huấn luyện và đánh giá bằng metric chuẩn	CER fine-tune khoảng 0,0315; detector đạt IoU khoảng 0,975 trên validation synthetic.	Đạt
Export ONNX và runtime độc lập	Package ocr_submission_package_final chứa ONNX model, config, CLI và Python API.	Đạt
OCR nhẹ, tốc độ cao	OCRLite nhanh nhất trong benchmark 3 engine, trung bình khoảng 5,86 giây/ảnh.	Đạt
Tích hợp ứng dụng web/API	Có CLI, Python API và FastAPI ở mức một phần; full-page endpoint cần hoàn thiện thêm.	Đạt một phần
Chất lượng đủ dùng so với engine mã nguồn mở	Đọc được phần lớn ảnh demo, nhưng vẫn còn lỗi dấu câu, hyphen và lowercase.	Đạt một phần

1

8

Từ các kết quả trên, có thể kết luận rằng OCRLite đã đáp ứng được phạm vi của một đề án tốt nghiệp theo hướng xây dựng hệ thống: có mô hình, có pipeline runtime, có số liệu thực nghiệm, có benchmark, có package demo và có phân tích lỗi. Giá trị chính của đề án nằm ở việc hoàn thiện một vòng train-evaluate-deploy tương đối đầy đủ, thay vì chỉ trình bày một kiến trúc mô hình trên lý thuyết.

1

5.1.2 Hạn chế

Bên cạnh các kết quả đạt được, OCRLite vẫn nên được xem là một nền tảng thử nghiệm có thể mở rộng, chưa phải một hệ OCR production-ready cho mọi loại tài liệu. Các hạn chế chính được chia thành bốn nhóm sau.

Thứ nhất, phạm vi ngôn ngữ còn hẹp. Hệ thống hiện tập trung vào tiếng Anh, charset chủ yếu gồm chữ cái Latin, chữ số, khoảng trắng và một số dấu câu thông dụng. Hệ thống chưa xử lý tiếng Việt có dấu, đa ngôn ngữ, chữ viết tay hoặc các loại font đặc biệt.

Thứ hai, dữ liệu detector vẫn phụ thuộc nhiều vào synthetic document pages. Cách

sinh dữ liệu này giúp huấn luyện nhanh và kiểm soát ground truth tốt, nhưng chưa bao phủ đầy đủ đặc điểm của ảnh thật như nén ảnh, font lạ, nền nhiễu, nghiêng nhẹ, bảng biểu, nhiều cột hoặc bố cục có hình ảnh xen kẽ.

Thứ ba, chất lượng output trên ảnh thật vẫn còn lỗi chi tiết. Các lỗi điển hình gồm mất apostrophe, mất hyphen, nhận nhầm ký tự mảnh, không giữ chữ hoa và lowercase toàn bộ output. Fine-tune đã cải thiện CER validation nhưng chưa loại bỏ hoàn toàn lỗi định tính trong demo full-page.

Thứ tư, đánh giá cross-engine chưa có ground truth chung. Benchmark hiện tập trung vào tốc độ, RAM và quan sát định tính trên một tập ảnh nhỏ. Vì chưa có ground truth cho toàn bộ ảnh benchmark, báo cáo chưa thể kết luận CER/WER tuyệt đối giữa OCRLite, RapidOCR và EasyOCR.

Những hạn chế trên không phủ nhận kết quả của đồ án, mà giúp xác định đúng ranh giới của hệ thống. Các chỉ số như CER và IoU phản ánh năng lực của từng mô hình trong điều kiện validation tương ứng; khi ghép thành OCR full-page, kết quả cuối cùng còn phụ thuộc vào layout, crop, threshold, contour và quy tắc ghép văn bản.

5.2 Hướng phát triển

Hướng phát triển của OCRLite nên đi theo nguyên tắc cải thiện dữ liệu và layout trước, sau đó mới tăng độ phức tạp mô hình. Một recognizer lớn hơn có thể cải thiện một số lỗi ký tự, nhưng nếu detector hoặc layout tạo crop sai thì chất lượng cuối cùng vẫn bị giới hạn.

5.2.1 Bổ sung dữ liệu thật và fine-tune detector

Ưu tiên cao nhất là bổ sung tập ảnh tài liệu thật có gán nhãn. Dữ liệu có thể gồm screenshot bài đọc, slide bài giảng, trang sách quét và tài liệu đơn cột/nhiều cột. Với mỗi ảnh, cần có box hoặc mask vùng chữ để fine-tune detector. Cách làm này giúp giảm domain gap giữa synthetic page và ảnh thật, đồng thời cải thiện chất lượng box trước khi đưa crop sang recognizer.

5.2.2 Nâng cấp layout analysis

2

Layout hiện tại dựa trên threshold, morphology, contour, histogram và khoảng trắng. Cách tiếp cận này phù hợp với bài đọc một cột, nhưng chưa đủ mạnh cho bảng biểu, nhiều cột, header/footer hoặc thứ tự đọc phức tạp. Hướng **phát triển có thể là bổ sung** rule-based nâng cao hoặc thử nghiệm mô hình layout analysis chuyên dụng như LayoutLM/PubLayNet trong trường hợp có dữ liệu phù hợp.

5.2.3 Cải thiện recognizer và hậu xử lý văn bản

Recognizer cần được bổ sung dữ liệu có nhiều dấu câu, hyphen, apostrophe, chữ hoa và các token gần với văn bản trong tài liệu giáo dục. Ngoài greedy CTC decode, có thể thử beam search CTC hoặc kết hợp language model nhẹ. Hậu xử lý cũng cần được thiết kế thận trọng: lexicon correction có thể giảm lỗi chính tả nhưng cũng có nguy cơ sửa sai tên riêng, số liệu hoặc thuật ngữ kỹ thuật.

5.2.4 Mở rộng tiếng Việt và đa ngôn ngữ

Để hỗ trợ tiếng Việt, cần mở rộng charset, bổ sung dữ liệu có dấu, cân nhắc phân bố ký tự hiếm và đánh giá lại kích thước mô hình. Tiếng Việt có dấu làm số lớp ký tự tăng lên và yêu cầu dữ liệu đa dạng hơn so với tiếng Anh lowercase. Đây là hướng phù hợp nếu OCRLite được tích hợp vào nền tảng học tập tại Việt Nam.

5.2.5 Hoàn thiện tích hợp sản phẩm và tối ưu hiệu năng

Ở mức sản phẩm, cần bổ sung endpoint full-page OCR cho FastAPI hoặc backend web, trả về không chỉ text mà cả lines, boxes, confidence và ảnh debug nếu cần. Với hệ thống có nhiều người dùng, nên đưa OCR vào worker queue để tránh block request đồng bộ. Về hiệu năng, có thể thử TensorRT FP16 trên GPU NVIDIA hoặc quantization INT8 cho CPU, nhưng chỉ nên tối ưu mạnh sau khi chất lượng layout và crop đã ổn định.

5.3 Tài liệu tham khảo

- [1] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," ICML, 2006.
- [2] B. Shi, X. Bai, and C. Yao, "An End-to-End Trainable Neural Network for Image-based

14

18

Sequence Recognition and Its Application to Scene Text Recognition," IEEE TPAMI, 2017.

[3] A. Gupta, A. Vedaldi, and A. Zisserman, "Synthetic Data for Text Localisation in Natural

Images," CVPR, 2016.

[4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015.

[5] Microsoft, "ONNX Runtime Documentation," <https://onnxruntime.ai/docs/>.

[6] Open Neural Network Exchange, "ONNX: Open Neural Network Exchange," <https://onnx.ai/>.

[7] PyTorch Contributors, "PyTorch Documentation," <https://docs.pytorch.org/docs/2.12/index.html>

[8] OpenCV Team, "OpenCV Documentation," <https://docs.opencv.org/>.

[9] Jaidev AI, "EasyOCR," GitHub repository, <https://github.com/JaidevAI/EasyOCR>.

[10] RapidAI, "RapidOCR," GitHub repository, <https://github.com/RapidAI/RapidOCR>.

[11] R. Smith, "An Overview of the Tesseract OCR Engine," ICDAR, 2007.

[12] OCRLite project artifacts: configs, reports, benchmark summary, package ocr_submission_package_final.

Tóm lại, OCRLite đã đạt được mục tiêu xây dựng một pipeline OCR nhẹ, có thể huấn luyện, đánh giá và triển khai bằng ONNX. Hệ thống đã có bằng chứng thực nghiệm, package runtime và hướng tích hợp ứng dụng rõ ràng. Những hạn chế còn lại chủ yếu nằm ở dữ liệu thật, layout phức tạp và chất lượng nhận dạng chi tiết trên ảnh thực tế; đây cũng là cơ sở cho các hướng phát triển tiếp theo.

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế và xây dựng hệ thống OCR nhúng OCR LITE nhận dạng văn bản Tiếng Anh trên ảnh tài liệu

Sinh viên: + Tường Ng. Quốc Thắng MSSV: 22119231
 + MSSV:
 Hướng dẫn: Th. & Đào Trọng Hiền

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần không định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Được phép bảo vệ khóa luận

Tp. HCM, ngày ... tháng năm 20...

Người nhận xét

(Ký và ghi rõ họ tên)

Đào Trọng Hiền

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau